

GRADO EN INGENIERÍA MULTIMEDIA



VNIVERSITAT
DE VALÈNCIA

TRABAJO FIN DE GRADO

DONA'M MÓN: APLICACIÓN MÓVIL PARA
VISIBILIZAR LA VIDA Y OBRA DE MUJERES EN
PUNTOS GEOLOCALIZADOS DE VALENCIA.

AUTOR:

IVANA STANISLAVOVA IVANOVA

TUTOR:

MANUEL PÉREZ AIXENDRI



VNIVERSITAT
DE VALÈNCIA



Escola Tècnica Superior
d'Enginyeria **ETSE-UV**

TRABAJO FIN DE GRADO

DONA'M MÓN: APLICACIÓN MÓVIL PARA
VISIBILIZAR LA VIDA Y OBRA DE MUJERES EN
PUNTOS GEOLOCALIZADOS DE VALENCIA.

AUTOR: IVANA STANISLAVOVA IVANOVA

TUTOR: MANUEL PÉREZ AIXENDRI

Declaración de autoría:

Yo, Ivana Stanislavova Ivanova, declaro la autoría del Trabajo Fin de Grado titulado “DONA’M MÓN: Aplicación móvil para visibilizar la vida y obra de mujeres en puntos geolocalizados de Valencia.” y que el citado trabajo no infringe las leyes en vigor sobre propiedad intelectual. El material no original que figura en este trabajo ha sido atribuido a sus legítimos autores.

Valencia, 20 de julio de 2025

Fdo: Ivana Stanislavova Ivanova

Resumen:

Este Trabajo de Fin de Grado pretende dar visibilidad a mujeres históricamente relevantes en Valencia, cuyas aportaciones, aunque presentes en el espacio urbano, son en gran parte desconocidas por la ciudadanía. Esta falta de reconocimiento limita la construcción de referentes para las futuras generaciones y evidencia la necesidad de herramientas innovadoras que unan tecnología, educación e igualdad.

Con este objetivo se ha desarrollado *DONA'm MÓN*, una aplicación móvil que combina geolocalización, realidad aumentada y contenidos multimedia para rescatar y poner en valor el legado de estas mujeres. La aplicación permite explorar lugares emblemáticos mediante un mapa interactivo, consultar información detallada sobre las protagonistas y sus ubicaciones asociadas, y seguir rutas temáticas donde los usuarios marcan su progreso mediante el escaneo de códigos QR. Además, cuando se encuentran cerca de un lugar, pueden acceder a experiencias en realidad aumentada con modelos 3D interactivos y audios explicativos, fomentando así el aprendizaje activo y la exploración urbana desde una perspectiva inclusiva.

El desarrollo se ha realizado con React Native para la interfaz, Django y PostgreSQL para el backend y AR.js con A-Frame para la RA, priorizando compatibilidad y escalabilidad. Esta solución no solo digitaliza el patrimonio femenino, sino que también lo vincula con el territorio, transformando la forma en que se vive y se transmite la historia.

Abstract:

This Final Degree Project aims to give visibility to historically significant women in Valencia, whose contributions, although present in the urban space, remain largely unknown to the public. This lack of recognition limits the creation of role models for future generations and highlights the need for innovative tools that combine technology, education, and equality.

To achieve this goal, *DONA'm MÓN* has been developed, a mobile application that combines geolocation, augmented reality, and multimedia content to rescue and highlight the legacy of these women. The application allows users to explore iconic locations through an interactive map, access detailed information about the protagonists and their associated sites, and follow thematic routes where they can track their progress by scanning QR codes. Additionally, when users are near a location, they can access augmented reality experiences with interactive 3D models and explanatory audio, thus promoting active learning and urban exploration from an inclusive perspective.

The development was carried out using React Native for the interface, Django and PostgreSQL for the backend, and AR.js with A-Frame for augmented reality, prioritizing compatibility and scalability. This solution not only digitizes female heritage but also connects it to the territory, transforming the way history is experienced and transmitted.

Resum:

Aquest Treball de Fi de Grau pretén donar visibilitat a dones històricament rellevants a València, les aportacions de les quals, encara que presents en l'espai urbà, són en gran part desconegudes per la ciutadania. Aquesta manca de reconeixement limita la construcció de referents per a les futures generacions i evidencia la necessitat d'eines innovadores que unisquen tecnologia, educació e igualtat.

Amb aquest objectiu s'ha desenvolupat *DONA'm MÓN*, una aplicació mòbil que combina geolocalització, realitat augmentada i continguts multimèdia per rescatar i posar en valor el llegat d'aquestes dones. L'aplicació permet explorar llocs emblemàtics mitjançant un mapa interactiu, consultar informació detallada sobre les protagonistes i les seues ubicacions associades, i seguir rutes temàtiques on els usuaris marquen el seu progrés mitjançant l'escaneig de codis QR. A més, quan es troben prop d'un lloc, poden accedir a experiències en realitat augmentada amb models 3D interactius i àudios explicatius, fomentant així l'aprenentatge actiu i l'exploració urbana des d'una perspectiva inclusiva.

El desenvolupament s'ha realitzat amb React Native per a la interfície, Django i PostgreSQL per al backend i AR.js amb A-Frame per a la RA, prioritzant compatibilitat i escalabilitat. Aquesta solució no sols digitalitza el patrimoni femení, sinó que també el vincula amb el territori, transformant la manera en què es viu i es transmet la història.

Agradecimientos:

Quiero agradecer, en primer lugar, a mis padres, Polina y Stanislav, por haber sacrificado siempre todo por mí, por apoyarme en todas mis decisiones, por estar a mi lado hasta cuando me pongo insufrible por el estrés. No sería nada sin vosotros y sin duda no hubiera podido llegar hasta aquí, os quiero. También agradecer al resto de mi familia, en Alcalá de Henares y en Bulgaria, por celebrar mis logros desde la distancia.

A mi tutor Manolo, por confiar en mí desde el principio y ser el mejor de los guías, gracias por todos los consejos, por cada una de las reuniones y por siempre tener un momento para ayudar y transmitir conocimiento. Y a mis compañeros y profesores del IRTIC, por no solo haberme acogido, si no por haberme brindado el mejor de los ambientes de trabajo.

A mi pareja, Salva, por estar conmigo desde antes incluso de saber que bachillerato debería hacer hasta el día de hoy, en el que termino el máster, por haber crecido juntos académicamente y como personas, por siempre estar a mi lado en las buenas y en las malas. Gracias de corazón.

A mis amigos del grado, del máster y de Gandia, por todos los buenos momentos.

A todas las mujeres que han luchado por hacerse ver en este mundo y por las que no pudieron.

Índice general

1. Introducción	27
1.1. Introducción	27
1.2. Motivación	27
1.3. Objetivos	28
1.3.1. Alineación con los Objetivos de Desarrollo Sostenible (ODS)	29
1.4. Organización de la memoria	30
2. Estado del arte	33
2.1. Historia, evolución e impacto de las aplicaciones móviles	33
2.1.1. Origen y evolución de los dispositivos móviles	33
2.1.2. Ecosistemas y sistemas operativos móviles	34
2.1.3. Tipos de aplicaciones móviles	34
2.1.4. Impacto económico y social	35
2.2. Evolución de la Realidad Aumentada en el Contexto de las Aplicaciones Móviles	36
2.2.1. Fundamentos y primeras ideas (1960–1990)	36
2.2.2. Definición formal y primeros prototipos (1990–2000)	37
2.2.3. Herramientas abiertas y salto a lo portátil (1999–2010)	37
2.2.4. Consolidación y tendencias actuales (2010–2025)	38
2.3. La visibilización de las mujeres en el espacio urbano: una cuestión de justicia histórica	39
2.3.1. Referentes femeninos en la ciudad: biografías y puntos de interés	40
2.4. Análisis de aplicaciones similares	42
2.4.1. Pokémon GO	42
2.4.2. Streetmuseum	42
2.4.3. CultuAR	43
2.4.4. Historypin	43
2.5. Tecnologías	44
2.5.1. Plataformas de desarrollo de aplicaciones móviles	44
2.5.2. Geolocalización	46
2.5.3. Realidad Aumentada (RA)	47
2.5.4. Base de Datos	49
2.5.5. Arquitectura de persistencia y gestión de datos	49
3. Requisitos, especificaciones, coste, riesgos, viabilidad	53
3.1. Requisitos	53
3.1.1. Requisitos funcionales	53
3.1.2. Requisitos funcionales	53

3.1.3.	Requisitos no funcionales	55
3.2.	Especificaciones	57
3.3.	Ciclo de vida del software	59
3.3.1.	Ventajas del modelo en cascada	59
3.3.2.	Justificación de su elección	59
3.4.	Planificación temporal	60
3.4.1.	Descomposición de tareas por fases	60
3.4.2.	Estimación temporal	63
3.4.3.	Diagrama de Gantt	64
3.5.	Estimación de costes	66
3.5.1.	Costes directos de personal	66
3.5.2.	Asignación temporal por perfil	66
3.5.3.	Costes directos de material	67
3.5.4.	Costes indirectos	68
3.5.5.	Coste total del proyecto	68
3.6.	Viabilidad del proyecto	68
3.6.1.	Viabilidad económica	68
3.6.2.	Viabilidad legal	69
3.7.	Análisis de riesgos	69
3.7.1.	Principales riesgos identificados	70
3.7.2.	Matriz de riesgos	71
3.7.3.	Resumen de evaluación de riesgos	71
3.7.4.	Estrategias de mitigación	72
3.7.5.	Planes de contingencia	72
4.	Análisis	73
4.1.	Introducción	73
4.2.	Diagramas de casos de uso	73
4.3.	Especificación de casos de uso	76
4.3.1.	Caso de uso 1: Login de usuario	76
4.3.2.	Caso de uso 2: Registro de usuario	77
4.3.3.	Caso de uso 3: Ver mapa con puntos de interés	78
4.3.4.	Caso de uso 4: Ver listado de puntos ordenado por cercanía	79
4.3.5.	Caso de uso 5: Ver ficha del lugar	80
4.3.6.	Caso de uso 6: Ver rutas y consultar puntos de una ruta	81
4.3.7.	Caso de uso 7: Escanear QR de punto de ruta	82
4.3.8.	Caso de uso 8: Ver puntos RA disponibles	83
4.3.9.	Caso de uso 9: Visualizar contenido RA	84
4.3.10.	Caso de uso 10: Editar perfil de usuario	85
4.3.11.	Caso de uso 11: Ver historial de lugares visitados y rutas completadas	86
4.3.12.	Caso de uso 12: Cerrar sesión	87
4.3.13.	Caso de uso 13: Iniciar sesión como administrador	88
4.3.14.	Caso de uso 14: Gestionar mujeres históricas	89
4.3.15.	Caso de uso 15: Gestionar puntos geolocalizados	90
4.3.16.	Caso de uso 16: Gestionar rutas temáticas	91
4.3.17.	Caso de uso 17: Consultar lista de usuarios	92
4.3.18.	Caso de uso 18: Gestionar materiales multimedia	93
4.3.19.	Caso de uso 19: Gestionar áreas de investigación	94
4.4.	Diagramas de Actividad	95

4.5. Diagramas de Estado	98
5. Diseño	103
5.1. Diseño de la interfaz	103
5.1.1. Justificación de la Paleta Cromática	103
5.1.2. Justificación de la Tipografía	104
5.1.3. Wireframes	104
5.2. Diseño de base de datos	105
5.2.1. Tablas principales y justificación de diseño	106
5.2.2. Relaciones entre tablas	107
5.2.3. Esquema de tablas	108
5.3. Diseño de la arquitectura del frontend	110
5.3.1. Diagrama de clases del frontend	110
5.4. Diagramas de secuencia	112
5.4.1. Diagrama de secuencia del caso de uso 1: Login de usuario	112
5.4.2. Diagrama de secuencia del caso de uso 2: Registro de usuario	113
5.4.3. Diagrama de secuencia del caso de uso 3: Ver mapa con puntos de interés	113
5.4.4. Diagrama de secuencia del caso de uso 4: Ver listado de puntos ordenado por cercanía	114
5.4.5. Diagrama de secuencia del caso de uso 5: Ver ficha del lugar	114
5.4.6. Diagrama de secuencia del caso de uso 6: Ver rutas y consultar puntos de una ruta	115
5.4.7. Diagrama de secuencia del caso de uso 7: Escanear QR de punto de ruta	115
5.4.8. Diagrama de secuencia del caso de uso 8: Ver puntos RA disponibles	116
5.4.9. Diagrama de secuencia del caso de uso 9: Visualizar contenido RA	116
5.4.10. Diagrama de secuencia del caso de uso 10: Editar perfil de usuario	117
5.4.11. Diagrama de secuencia del caso de uso 11: Ver historial de lugares visitados y rutas completadas	117
5.4.12. Diagrama de secuencia del caso de uso 12: Cerrar sesión	118
5.4.13. Diagrama de secuencia del caso de uso 13: Iniciar sesión como ad- ministrador	118
5.4.14. Diagrama de secuencia del caso de uso 14: Gestionar mujeres históricas	119
5.4.15. Diagrama de secuencia del caso de uso 15: Gestionar puntos geolo- calizados	119
5.4.16. Diagrama de secuencia del caso de uso 16: Gestionar rutas temáticas	120
5.4.17. Diagrama de secuencia del caso de uso 17: Consultar lista de usuarios	120
5.4.18. Diagrama de secuencia del caso de uso 18: Gestionar materiales multimedia	121
5.4.19. Diagrama de secuencia del caso de uso 19: Gestionar áreas de in- vestigación	121
6. Implementación y pruebas	123
6.1. Implementación del backend	123
6.1.1. Herramientas, frameworks y lenguajes utilizados	123
6.1.2. Estructura del proyecto y organización del código	123
6.1.3. Modelado de datos	124
6.1.4. Gestión de imágenes	127
6.1.5. Paginación de la API	127

6.1.6.	Seguridad y permisos	128
6.1.7.	Gestión de variables de entorno y configuración segura	128
6.1.8.	Serialización y exposición de la API REST	129
6.1.9.	Vistas y endpoints de la API	131
6.1.10.	Administración y gestión de datos	132
6.1.11.	Cambio de contraseña del usuario administrador de Django	133
6.1.12.	Gestión de migraciones y consistencia de la base de datos	134
6.1.13.	Integración con el frontend y gestión de CORS	135
6.1.14.	Integración y despliegue con Docker	135
6.1.15.	Pruebas automáticas	136
6.1.16.	Internacionalización y localización	137
6.1.17.	Gestión de archivos estáticos	137
6.1.18.	Gestión de emails y notificaciones	138
6.1.19.	Internacionalización y localización	138
6.2.	Implementación del frontend	139
6.2.1.	Herramientas, frameworks y lenguajes utilizados	139
6.2.2.	Estructura del proyecto y organización del código	139
6.2.3.	Navegación y estructura de pantallas	140
6.2.4.	Gestión de usuarios y autenticación	141
6.2.5.	Visualización de mujeres y lugares	141
6.2.6.	Detalle de lugar y gestión de visitas	142
6.2.7.	Mapas y geolocalización	143
6.2.8.	Gestión de rutas temáticas	145
6.2.9.	Historial de lugares y rutas visitadas	145
6.2.10.	Sistema de filtros avanzado y gestión de estado	146
6.2.11.	Sincronización de estados de visita	146
6.2.12.	Notificaciones y proximidad	147
6.2.13.	Realidad aumentada	147
6.2.14.	Gestión de estilos y diseño	147
6.2.15.	Gestión de almacenamiento local	148
6.2.16.	Integración con la API y manejo de errores	148
6.2.17.	Pruebas y depuración	148
6.3.	Implementación de Realidad Aumentada	149
6.3.1.	Herramientas, frameworks y tecnologías utilizadas	149
6.3.2.	Arquitectura y estructura del proyecto AR	149
6.3.3.	Diseño de la experiencia interactiva	149
6.3.4.	Gestión de la tipografía en textos 3D	153
6.3.5.	Formato y optimización de modelos 3D	154
6.3.6.	Implementación del sistema de tracking en AR	156
6.3.7.	Despliegue y distribución	156
6.3.8.	Consideraciones técnicas y limitaciones	157
7.	Pruebas	159
7.1.	Pruebas funcionales	159
7.2.	Pruebas de Usabilidad	163
7.2.1.	Metodología	164
7.2.2.	Cálculo del SUS	165
7.2.3.	Resultados del cuestionario	165
7.2.4.	Visualización de resultados por pregunta	165

7.2.5. Puntuaciones SUS	168
7.2.6. Comentarios adicionales de los usuarios	170
7.3. Hardware empleado en las Pruebas de rendimiento	171
7.4. Pruebas de Rendimiento del Backend	172
7.4.1. Metodología	172
7.4.2. Resultados y Análisis	172
7.4.3. Pruebas de Rendimiento del Frontend	175
7.5. Pruebas de Rendimiento de la Realidad Aumentada	177
7.5.1. Metodología	177
7.5.2. Resultados	178
7.5.3. Análisis	179
8. Resultados	181
9. Conclusiones	191
9.1. Revisión de costes	191
9.1.1. Comparativa gráfica	192
9.1.2. Reflexión final	192
9.2. Conclusiones	192
9.2.1. Conclusiones técnicas	193
9.2.2. Conclusiones personales	193
9.3. Trabajo futuro	193
A. Apéndice	197
A.1. Ejemplos del lenguaje de marcado Latex	197
Bibliografía	198

Índice de figuras

2.1. Evolución histórica de los dispositivos móviles [1].	34
2.2. Primer sistema RA: "Sword of Damocles"(1968) [2]	36
2.3. Sistema KARMA para asistencia con RA (1993)	37
2.4. ARToolKit y marcadores visuales (1999)	38
2.5. HoloLens 2 de Microsoft en labores de asistencia remota y formación.	39
2.6. Boceto inicial del mapa de Valencia con los puntos geolocalizados de las mujeres referentes incluidos en la aplicación.	41
2.7. Interfaz de Pokémon GO mostrando un entorno real aumentado con la aparición de un Pokémon virtual y también el entorno virtual del juego.	42
2.8. Ejemplo de Streetmuseum: visualización de una escena histórica superpuesta sobre el entorno actual de la ciudad.	43
2.9. Captura de CultuAR mostrando información sobre un monumento en el entorno real.	43
2.10. Interfaz de Historypin donde se visualizan fotografías históricas asociadas a ubicaciones reales en el mapa y su detalle de contenido.	44
3.1. Representación del modelo de desarrollo en cascada.	60
3.2. Diagrama de Gantt detallado del proyecto.	65
3.3. Matriz de priorización de riesgos utilizada	71
4.1. Diagrama de casos de uso: Administrador	74
4.2. Diagrama de casos de uso: Usuario no registrado	74
4.3. Diagrama de actividad de usuario no registrado	95
4.4. Diagrama de actividad de usuario registrado	96
4.5. Diagrama de actividad del panel de administración	97
4.6. Diagrama de estado de usuario no registrado	98
4.7. Diagrama de estado de usuario registrado 1	99
4.8. Diagrama de estado de usuario registrado 2	100
4.9. Diagrama de estado del panel de administración	101
5.1. Colores de la paleta cromática con código hexadecimal y RGB.	104
5.2. Ejemplo de las tipografías San Francisco y Roboto.	104
5.3. Wireframes de las pantallas principales de <i>DONA'm MÓN</i>	105
5.4. Diagrama de clases de la base de datos	109
5.5. Diagrama de clases del frontend React Native: estructura de componentes y relaciones principales.	111
5.6. Diagrama de secuencia del caso de uso 1: Login de usuario	112
5.7. Diagrama de secuencia del caso de uso 2: Registro de usuario	113
5.8. Diagrama de secuencia del caso de uso 3: Ver mapa con puntos de interés	113

5.9. Diagrama de secuencia del caso de uso 4: Ver listado de puntos ordenado por cercanía	114
5.10. Diagrama de secuencia del caso de uso 5: Ver ficha del lugar	114
5.11. Diagrama de secuencia del caso de uso 6: Ver rutas y consultar puntos de una ruta	115
5.12. Diagrama de secuencia del caso de uso 7: Escanear QR de punto de ruta	115
5.13. Diagrama de secuencia del caso de uso 8: Ver puntos RA disponibles	116
5.14. Diagrama de secuencia del caso de uso 9: Visualizar contenido RA	116
5.15. Diagrama de secuencia del caso de uso 10: Editar perfil de usuario	117
5.16. Diagrama de secuencia del caso de uso 11: Ver historial de lugares visitados y rutas completadas	117
5.17. Diagrama de secuencia del caso de uso 12: Cerrar sesión	118
5.18. Diagrama de secuencia del caso de uso 13: Iniciar sesión como administrador	118
5.19. Diagrama de secuencia del caso de uso 14: Gestionar mujeres históricas	119
5.20. Diagrama de secuencia del caso de uso 15: Gestionar puntos geolocalizados	119
5.21. Diagrama de secuencia del caso de uso 16: Gestionar rutas temáticas	120
5.22. Diagrama de secuencia del caso de uso 17: Consultar lista de usuarios	120
5.23. Diagrama de secuencia del caso de uso 18: Gestionar materiales multimedia	121
5.24. Diagrama de secuencia del caso de uso 19: Gestionar áreas de investigación	121
6.1. Vista de listado del modelo Mujer en Django Admin	133
6.2. Formulario de edición para el modelo Mujer	133
6.3. Recopilación de archivos estáticos con <code>collectstatic</code> en Django	138
6.4. Cuatro modelos generados automáticamente por Meshy a partir de una imagen de entrada.	155
6.5. Modelo seleccionado de los cuatro generados, ya con la textura aplicada por Meshy.	155
6.6. Imagen original de Manuela Solís que se utiliza como entrada en Meshy para la generación automática de modelos 3D.	155
7.1. Usuarios probando la aplicación DONA'm MÓN en el IVAM	164
7.2. Distribución de respuestas a la Pregunta 1	166
7.3. Distribución de respuestas a la Pregunta 2	166
7.4. Distribución de respuestas a la Pregunta 3	166
7.5. Distribución de respuestas a la Pregunta 4	166
7.6. Distribución de respuestas a la Pregunta 5	167
7.7. Distribución de respuestas a la Pregunta 6	167
7.8. Distribución de respuestas a la Pregunta 7	167
7.9. Distribución de respuestas a la Pregunta 8	167
7.10. Distribución de respuestas a la Pregunta 9	168
7.11. Distribución de respuestas a la Pregunta 10	168
7.12. Puntuaciones SUS por usuario.	169
7.13. Distribución de puntuaciones SUS.	170
7.14. Comparación de tiempos promedio por endpoint	174
7.15. Comparación del percentil 95 por endpoint	174
7.16. Comparación de tiempos máximos por endpoint	175
7.17. Aplicación DONA'm MÓN ejecutándose en Expo Go con el panel <i>Performance Monitor</i>	176
7.18. Captura de los registros obtenidos en el dispositivo durante la prueba	177
7.19. Evolución del FPS durante la interacción con elementos AR	179

8.1. Pantalla de bienvenida y notificación de proximidad a un lugar.	181
8.2. Mapa con lugares visitados (verde), no visitados (rosa) y ubicación actual.	182
8.3. Detalle de un lugar: información de la mujer, el lugar y distancia.	182
8.4. Listado de lugares y buscador por nombre de lugares y mujeres.	183
8.5. Filtros aplicables sobre el mapa y el listado de lugares.	183
8.6. Detalles de un lugar en la aplicación.	184
8.7. Opciones de interacción con el lugar: compartir.	184
8.8. Visualización de imágenes en modal.	185
8.9. Página de rutas y ejemplos de rutas incompleta y completada.	185
8.10. Pantalla de realidad aumentada.	186
8.11. Experiencia de realidad aumentada (1).	186
8.12. Experiencia de realidad aumentada (2).	187
8.13. Experiencia de realidad aumentada (3).	187
8.14. Pantalla de perfil del usuario.	188
8.15. Historial de lugares visitados.	188
8.16. Historial de rutas completadas.	189
9.1. Comparación entre el coste estimado y el coste real	192

Índice de cuadros

2.1. Comparación entre tipos de aplicaciones móviles	35
2.2. Comparativa entre SQLite, MySQL, PostgreSQL, MariaDB y MongoDB [3, 4, 5, 6].	51
3.1. Estimación por tres valores de las tareas del proyecto	63
3.2. Perfiles y costes de personal (según cotización SS 2025)	66
3.3. Asignación de tareas y costes por perfil	67
3.4. Elementos utilizados y coste amortizado durante el desarrollo (80 días) . .	67
3.5. Resumen de costes del proyecto	68
3.6. Principales riesgos identificados	70
3.7. Resumen cualitativo de evaluación de riesgos	71
4.1. Caso de uso 1 - Login de usuario	76
4.2. Caso de uso 2 - Registro de usuario	77
4.3. Caso de uso 3 - Ver mapa con puntos de interés	78
4.4. Caso de uso 4 - Ver listado de puntos ordenado por cercanía	79
4.5. Caso de uso 1.5 - Ver ficha del lugar	80
4.6. Caso de uso 6 - Ver rutas y consultar puntos de una ruta	81
4.7. Caso de uso 7 - Escanear QR de punto de ruta	82
4.8. Caso de uso 8 - Ver puntos RA disponibles	83
4.9. Caso de uso 9 - Visualizar contenido RA	84
4.10. Caso de uso 10 - Editar datos del perfil	85
4.11. Caso de uso 11 - Ver historial de lugares visitados y rutas completadas . .	86
4.12. Caso de uso 12 - Cerrar sesión	87
4.13. Caso de uso 13 - Iniciar sesión como administrador	88
4.14. Caso de uso 14 - Gestionar mujeres históricas	89
4.15. Caso de uso 15 - Gestionar puntos geolocalizados	90
4.16. Caso de uso 16 - Gestionar rutas temáticas	91
4.17. Caso de uso 17 - Consultar lista de usuarios	92
4.18. Caso de uso 18 - Gestionar materiales multimedia	93
4.19. Caso de uso 19 - Gestionar áreas de investigación	94
5.1. Esquema resumido de las tablas principales de la base de datos	108
7.2. Respuestas del cuestionario SUS por usuario	165
7.3. Perfil de los participantes y puntuación SUS final	169
7.4. Especificaciones del hardware utilizado en las pruebas	171
7.5. Resultados bajo carga normal (10 usuarios, 100 peticiones por endpoint) .	172
7.6. Resultados bajo carga de estrés (50 usuarios, 1000 peticiones por endpoint)	173
7.7. Tiempos de carga y eventos críticos	178

9.1. Ampliación de jornadas y costes adicionales	191
--	-----

Capítulo 1

Introducción

1.1. Introducción

En la actualidad, la tecnología se ha convertido en una herramienta clave para transformar la forma en la que interactuamos con nuestro entorno, permitiendo nuevas maneras de aprender, explorar y conectarnos con la historia y la cultura. Sin embargo, la construcción de los relatos históricos y culturales ha tendido a otorgar mayor protagonismo a figuras masculinas, dejando en la sombra numerosas contribuciones femeninas de gran relevancia en ámbitos como la ciencia, el arte, la política o la educación.

Este Trabajo de Fin de Grado se centra en el desarrollo de una aplicación móvil interactiva basada en geolocalización y realidad aumentada (RA), que tiene como objetivo visibilizar el legado de mujeres con relevancia histórica que han nacido, vivido, desarrollado parte de su labor o tenido relevancia en Valencia. Mediante esta aplicación, los usuarios podrán desbloquear contenido educativo y multimedia al visitar puntos de interés geolocalizados, conectando los lugares emblemáticos de la ciudad con las historias de estas mujeres, conociendo su vida y obra.

La aplicación lleva por nombre *DONA'm MÓN*, una frase en valenciano con doble significado en su primera palabra: “dona” puede leerse tanto como sustantivo (“mujer”) o como forma verbal (“dame”). Así que el título leído como “Dame mundo”, esconde un juego de palabras que representa el propósito del proyecto: ofrecer a las mujeres este espacio real y simbólico en el mundo, un lugar en el mapa donde su historia pueda ser descubierta y reconocida.

DONA'm MÓN ofrece una experiencia educativa y lúdica que invita a reflexionar sobre la igualdad de género. Este proyecto pretende ser un puente entre el pasado y el presente, ayudando a los usuarios a redescubrir la ciudad desde esta nueva perspectiva inclusiva.

1.2. Motivación

El desarrollo de este proyecto surge de una doble motivación: por un lado, mi interés personal en el desarrollo de tecnologías con un propósito educativo y social, y por otro, el deseo de poner en valor las aportaciones de las mujeres a lo largo de la historia.

Durante mi recorrido académico como estudiante de Ingeniería Multimedia y en mi máster actual en Tecnologías web, Aplicaciones móviles y Computación en la nube, he desarrollado un fuerte interés por aplicar el conocimiento técnico en proyectos que tengan un impacto positivo en la sociedad, orientados a transformar la manera en la que accedemos y transmitimos el conocimiento.

Además, como mujer en STEM, considero importante que este proyecto consiga ayudar en la visibilización del papel femenino en todas las áreas del saber y a toda clase de público. Aunque se han producido avances significativos en materia de igualdad, los datos actuales evidencian que la brecha persiste: solo el 33 % de los investigadores a nivel mundial son mujeres, y en campos como la inteligencia artificial esta cifra desciende al 22 % [7]. En el contexto español, apenas el 24 % de las cátedras universitarias están ocupadas por mujeres, según el informe “Científicas en Cifras 2021” del Ministerio de Ciencia e Innovación [8]. Estas cifras reflejan una infrarrepresentación que no solo afecta a la ciencia, sino también a otros ámbitos clave del desarrollo cultural y social.

Valencia, como muchas otras ciudades, alberga una historia rica y compleja en la que también han participado numerosas mujeres valiosas, desde científicas y artistas hasta activistas y educadoras. Muchas de ellas siguen siendo poco conocidas fuera de círculos especializados. Este proyecto representa una oportunidad para darles mayor protagonismo, aprovechando tecnologías como la realidad aumentada y la geolocalización para generar experiencias de aprendizaje activas, accesibles e innovadoras.

Por todo esto, el objetivo de este trabajo no es solo técnico, sino también ético y cultural: contribuir, desde la tecnología, a construir una memoria más justa e inclusiva.

1.3. Objetivos

El objetivo principal de este proyecto es desarrollar una aplicación móvil interactiva que utilice tecnologías de geolocalización, realidad aumentada (RA) y una base de datos centralizada para visibilizar y divulgar las historias de mujeres destacadas en la ciudad de Valencia, promoviendo su reconocimiento histórico y cultural a través de una experiencia inmersiva, educativa y accesible para los usuarios.

Descomponiendo este objetivo principal, se definen los siguientes objetivos específicos:

- **Investigación y selección de mujeres invisibilizadas:**
 - Identificar figuras femeninas históricas relacionadas con Valencia que hayan destacado en campos como la ciencia, el arte, la política o los derechos sociales.
 - Asociar a cada figura un punto geolocalizado en la ciudad con un vínculo significativo con su historia.
- **Creación y gestión de una base de datos con PostgreSQL:**
 - Diseñar y desarrollar una base de datos relacional en PostgreSQL, estructurada para almacenar de forma eficiente información sobre las mujeres seleccionadas, sus ubicaciones geográficas y el contenido multimedia vinculado.
 - Implementar un backend utilizando el framework Django, asegurando la integridad, consistencia y seguridad de los datos mediante su ORM, y exponiendo

una API RESTful que permita su acceso y manipulación desde la aplicación móvil.

■ **Desarrollo de funcionalidades tecnológicas:**

- Implementar un sistema de geolocalización que permita a los usuarios desbloquear contenido interactivo al acercarse a los puntos de interés.
- Integrar elementos de realidad aumentada desarrollados con A-Frame, que permitan visualizar objetos, imágenes o recreaciones 3D relacionadas con las mujeres seleccionadas al enfocar con la cámara del dispositivo móvil.
- Implementar un sistema de notificaciones que alerte a los usuarios al aproximarse a un punto de interés relevante, mejorando la interacción con el entorno urbano y fomentando la exploración activa.

■ **Diseño de una experiencia de usuario intuitiva y atractiva:**

- Crear una interfaz accesible y adaptada al entorno móvil con React Native y Expo Go, que facilite la navegación y el uso de las funcionalidades principales de la aplicación.
- Incluir un mapa interactivo con los puntos de interés marcados, accesibles según la ubicación del usuario.

■ **Generación de contenido multimedia:**

- Diseñar materiales interactivos y educativos como textos, audios, imágenes, vídeos y modelos 3D que permitan a los usuarios conocer la vida y las contribuciones de las mujeres seleccionadas.
- Desarrollar un sistema de recompensas o logros para incentivar la exploración de todos los puntos geolocalizados.

■ **Compatibilidad multiplataforma y pruebas de usabilidad:**

- Garantizar que la aplicación sea funcional en dispositivos Android e iOS mediante el uso de herramientas multiplataforma como React Native y Expo Go.
- Realizar pruebas de usabilidad con un grupo de usuarios para evaluar la experiencia de uso y verificar que se cumplen los objetivos educativos y tecnológicos planteados.

1.3.1. Alineación con los Objetivos de Desarrollo Sostenible (ODS)

DONA'm MÓN se relaciona de forma directa con algunos de los Objetivos de Desarrollo Sostenible propuestos por la ONU dentro de la Agenda 2030, especialmente aquellos centrados en la igualdad, la educación, la sostenibilidad urbana y el bienestar. A través de una experiencia tecnológica que combina cultura, memoria y participación ciudadana, el proyecto contribuye a acercar estos objetivos al contexto local. [9, 10, 11]

- **ODS 5: Igualdad de género.** Este objetivo busca eliminar todas las formas de discriminación y violencia contra mujeres y niñas, y promover su plena participación en todos los ámbitos. *DONA'm MÓN* se centra precisamente en recuperar y visibilizar figuras femeninas que han sido olvidadas o ignoradas. Reivindicar sus historias

es una forma de justicia simbólica y de construir referentes para las nuevas generaciones.

- **ODS 4: Educación de calidad.** Este ODS promueve el acceso a una educación inclusiva, equitativa y de calidad. La aplicación funciona como una herramienta educativa alternativa que permite aprender fuera del aula, en el espacio urbano, utilizando recursos multimedia y tecnologías actuales. Es una forma distinta de acercarse a la historia local desde una perspectiva crítica y feminista.
- **ODS 11: Ciudades y comunidades sostenibles.** Uno de los enfoques de este objetivo es garantizar que las ciudades sean más inclusivas y accesibles. *DONA 'm MÓN* propone una nueva forma de recorrer Valencia, resignificando lugares que muchas veces pasan desapercibidos. También invita a caminar y descubrir el entorno, fomentando un uso más consciente del espacio público.
- **ODS 3: Salud y bienestar.** Aunque no es el objetivo principal del proyecto, se puede decir que la aplicación promueve indirectamente hábitos saludables, ya que para desbloquear los contenidos de las rutas es necesario moverse físicamente por la ciudad. Esto puede motivar a las personas usuarias a caminar, explorar y mantenerse activas mientras aprenden.

1.4. Organización de la memoria

Esta memoria se estructura en ocho capítulos principales y una bibliografía final. A continuación se describen brevemente los contenidos de cada uno de ellos:

Capítulo 1 – Introducción: contextualiza el trabajo realizado, presentando la motivación personal y social del proyecto, así como los objetivos generales y específicos que se pretenden alcanzar. También incluye esta sección de organización de la memoria para orientar al lector sobre la estructura del documento.

Capítulo 2 – Estado del arte: recoge un análisis comparativo de aplicaciones similares ya existentes, con el objetivo de identificar puntos fuertes y carencias que puedan aprovecharse como referencia para el desarrollo del proyecto. Además, se detallan las tecnologías empleadas, justificando su elección en función de las necesidades de la aplicación.

Capítulo 3 – Requisitos, especificaciones, costes, riesgos y viabilidad: define los requisitos funcionales y no funcionales del sistema, junto con sus especificaciones técnicas. También se lleva a cabo una estimación de los costes asociados, se identifican posibles riesgos y se valora la viabilidad del proyecto desde una perspectiva técnica y económica.

Capítulo 4 – Análisis: describe el análisis de la solución propuesta, incluyendo aspectos como la estructura general de la aplicación, la lógica funcional y la interacción entre sus componentes principales.

Capítulo 5 – Diseño: presenta el diseño de la aplicación a diferentes niveles (arquitectura, interfaz de usuario, estructura de datos, etc.), sirviendo como puente entre el análisis conceptual y la posterior implementación técnica.

Capítulo 6 – Implementación: documenta el proceso de desarrollo, indicando cómo se ha llevado a cabo la implementación de la solución y qué herramientas se han utilizado.

Capítulo 7 – Pruebas: describe las pruebas realizadas para verificar el correcto funcionamiento del sistema, abarcando pruebas funcionales, de rendimiento y de usabilidad.

Capítulo 8 – Conclusiones: ofrece una reflexión final sobre los resultados obtenidos, revisa los costes reales frente a los estimados, valora el cumplimiento de los objetivos y propone posibles mejoras o líneas de trabajo futuro.

Bibliografía: recoge todas las fuentes utilizadas para la elaboración del trabajo, tanto en su fase de investigación como en el desarrollo e implementación del proyecto.

Capítulo 2

Estado del arte

2.1. Historia, evolución e impacto de las aplicaciones móviles

2.1.1. Origen y evolución de los dispositivos móviles

Para entender el desarrollo de las aplicaciones móviles, es útil conocer primero la evolución histórica de los dispositivos móviles, la cual se puede observar en la Figura 2.1. La telefonía móvil nace en 1946 con el sistema móvil de AT&T, que permitía realizar llamadas desde vehículos utilizando dispositivos de más de 30 kg. Este sistema inicial, basado en la tecnología Push-To-Talk y gestionado por operadoras humanas, tenía una cobertura muy limitada y altos costes de uso [12].

Durante las décadas siguientes se sucedieron avances importantes: la introducción del sistema RCC en 1960, el Improved Mobile Telephone Service de AT&T en 1965, y finalmente, la llegada del primer teléfono móvil portátil, el Motorola DynaTAC, en 1983. Sin embargo, no fue hasta la década de 1990 cuando los dispositivos comenzaron a incorporar funcionalidades propias de asistentes personales, como los Apple Newton o los Palm Pilot, marcando así el inicio de las primeras “aplicaciones” de uso cotidiano, como agendas, calendarios y juegos como Snake de Nokia [13].

El cambio de paradigma llegó en 2007 con la aparición del iPhone, que introdujo una interfaz multitáctil intuitiva y eliminó el teclado físico, inaugurando así la era del smartphone moderno. Un año más tarde, Apple lanzó la App Store, permitiendo a desarrolladores externos distribuir aplicaciones fácilmente. Ese mismo año, Google presentó Android junto con el HTC Dream, diversificando el ecosistema y facilitando el acceso masivo al desarrollo y consumo de aplicaciones móviles [14].



Figura 2.1: Evolución histórica de los dispositivos móviles [1].

2.1.2. Ecosistemas y sistemas operativos móviles

El auge de las aplicaciones móviles está estrechamente vinculado al desarrollo de los sistemas operativos (SO) móviles. Durante los primeros años del siglo XXI, Symbian OS dominaba el mercado, especialmente en terminales de Nokia. Sin embargo, fue rápidamente desplazado por iOS (2007) y Android (2008), que ofrecían interfaces gráficas avanzadas, tiendas de aplicaciones centralizadas y acceso optimizado a hardware [15].

Actualmente, Android posee una cuota de mercado superior al 70 % a nivel global, mientras que iOS concentra cerca del 28 % del mercado, especialmente en países desarrollados [16]. Otros sistemas operativos como Windows Phone, BlackBerry OS o Tizen han desaparecido o mantienen una presencia marginal. La consolidación de este duopolio ha propiciado estándares de desarrollo bien definidos, facilitando la expansión del mercado de aplicaciones.

Cabe destacar que la fragmentación del ecosistema Android sigue representando un reto técnico. Según un estudio de Android Police [17], el tiempo medio de soporte de actualizaciones es de 21 meses en dispositivos Android, frente a los 41 meses de Apple, lo que obliga a los desarrolladores a implementar múltiples versiones y adaptaciones para garantizar la compatibilidad.

2.1.3. Tipos de aplicaciones móviles

Las aplicaciones móviles pueden clasificarse en función de su arquitectura en cinco categorías principales:

- **Aplicaciones web:** ejecutadas dentro del navegador, sin necesidad de instalación. Utilizan tecnologías como HTML5, CSS y JavaScript. Son portables pero con acceso

limitado al hardware [18].

- **Aplicaciones web progresivas (PWA):** ofrecen una experiencia cercana a las nativas, pueden instalarse desde el navegador y ejecutarse sin conexión, gracias a tecnologías como los service workers.
- **Aplicaciones híbridas:** encapsulan una web app dentro de una aplicación nativa mediante motores como Cordova o Ionic. Permiten cierto acceso a funciones del dispositivo, pero con rendimiento inferior a las apps nativas.
- **Aplicaciones nativas:** desarrolladas específicamente para cada plataforma, usando lenguajes como Swift (iOS), Kotlin o Java (Android). Ofrecen el mejor rendimiento, pero requieren múltiples desarrollos independientes.
- **Aplicaciones nativas multiplataforma:** permiten usar un único lenguaje y base de código para generar versiones para iOS y Android. Ejemplos incluyen React Native, Flutter y Xamarin, ampliamente utilizados en el desarrollo moderno [19].

Cada enfoque presenta ventajas y limitaciones en términos de rendimiento, coste, mantenimiento, compatibilidad y acceso a recursos del sistema, como puede observarse en el Cuadro 2.1.

Característica	App Nativa	App Híbrida	App Web Progresiva	App Web
Experiencia de usuario	EXCELENTE	EXCELENTE	BUENA	NORMAL
Velocidad y rendimiento	MUY RÁPIDA	MUY RÁPIDA	BUENA	NORMAL
Seguridad	ALTA	ALTA	BUENA	BUENA
App Stores	SÍ	SÍ	NO	NO
Función offline	SÍ	SÍ	NO	NO
Tiempo de desarrollo	ALTO	MEDIO	BAJO	BAJO
Mantenimiento	ALTO	MEDIO	BAJO	BAJO
Coste de desarrollo	ALTO	MEDIO	MEDIO	MEDIO

Cuadro 2.1: Comparación entre tipos de aplicaciones móviles

2.1.4. Impacto económico y social

El mercado de aplicaciones móviles ha alcanzado dimensiones económicas y sociales extraordinarias. En 2023 se descargaron más de 257 mil millones de aplicaciones a nivel mundial, con un gasto dentro de estas y suscripciones que superó los 170 mil millones de dólares [20]. El tiempo medio diario de uso del móvil ha aumentado a 5 horas, consolidando a las aplicaciones como principales mediadoras de nuestra relación con el entorno digital [21].

Además, su impacto trasciende lo económico. Las aplicaciones han revolucionado sectores clave como la salud (mHealth), la educación (mLearning), el transporte, el comercio electrónico, la cultura y la participación ciudadana. Por ejemplo, en el ámbito de la salud, aplicaciones como MySugr o Glucose Buddy permiten a pacientes con diabetes gestionar su enfermedad mediante el seguimiento de glucosa, dieta y ejercicio. En educación, plataformas como Duolingo o Khan Academy han universalizado el acceso al aprendizaje, permitiendo a millones de personas adquirir nuevas habilidades desde cualquier lugar [22].

En cuanto a la accesibilidad, los dispositivos móviles permiten una conexión constante con contenidos digitales, lo que favorece procesos de inclusión digital en poblaciones diversas. Esta capacidad de mediación sociotécnica hace que las aplicaciones móviles sean herramientas importantes para la innovación social [22].

2.2. Evolución de la Realidad Aumentada en el Contexto de las Aplicaciones Móviles

La Realidad Aumentada (RA) es una tecnología que permite combinar contenido digital generado por ordenador con la percepción del mundo físico, proporcionando una experiencia enriquecida y contextual. Aunque su auge reciente está vinculado al ecosistema móvil, su desarrollo abarca más de medio siglo, con avances teóricos, técnicos y aplicados que han configurado su estado actual como tecnología madura.

2.2.1. Fundamentos y primeras ideas (1960–1990)

Los antecedentes de la RA preceden a la era digital. En 1962, Morton Heilig presentó el Sensorama, un sistema inmersivo que combinaba estímulos visuales, sonoros y olfativos para simular experiencias sensoriales integradas, aunque sin capacidades de interacción. A nivel tecnológico, el verdadero precursor de la RA fue el Head-Mounted Display (HMD) diseñado por Ivan Sutherland en 1968, apodado “Sword of Damocles”, como se puede observar en la Figura 2.2. Este sistema óptico-mecánico, con seis grados de libertad, permitió visualizar objetos generados por ordenador superpuestos al entorno real [23].



Figura 2.2: Primer sistema RA: "Sword of Damocles"(1968) [2]

No obstante, el concepto formal de RA se desarrolló más tarde. En 1992, Caudell y Mizell acuñaron el término Augmented Reality en un contexto industrial, para referirse al

uso de displays que asistían a operarios durante tareas complejas. Este enfoque pragmático orientó la investigación hacia sistemas útiles para tareas del mundo real, diferenciándose de la realidad virtual (RV), que propone la inmersión total en entornos digitales.

2.2.2. Definición formal y primeros prototipos (1990–2000)

Ronald Azuma estableció en 1997 la definición académica más citada de RA, basada en tres criterios fundamentales: (1) combinación de elementos reales y virtuales, (2) interacción en tiempo real y (3) registro espacial tridimensional preciso [23]. En paralelo, Paul Milgram y Fumio Kishino introdujeron el Continuo Realidad-Virtualidad, que posiciona la RA entre el entorno físico puro y la RV completa, permitiendo caracterizar distintos grados de mezcla entre ambos mundos [24].



Figura 2.3: Sistema KARMA para asistencia con RA (1993)

Durante esta década se desarrollaron los primeros sistemas funcionales de RA. En 1993, el sistema KARMA, creado por Steve Feiner y su equipo, permitía la superposición de información contextual durante tareas físicas [25]. En 1997, se presentó el prototipo MARS, un sistema de RA móvil diseñado para guiar a turistas mediante visualización aumentada en tiempo real. Estas iniciativas fueron posibles gracias al desarrollo de dispositivos portables y técnicas de tracking visual.

2.2.3. Herramientas abiertas y salto a lo portátil (1999–2010)

Uno de los hitos más relevantes en esta etapa fue el desarrollo de ARToolKit por Hirokazu Kato y Mark Billinghurst en 1999. Esta librería de código abierto permitió detectar marcadores visuales en video y superponer contenido digital 3D en tiempo real [26]. Supuso una democratización de la RA, facilitando la experimentación por parte de investigadores y desarrolladores independientes.

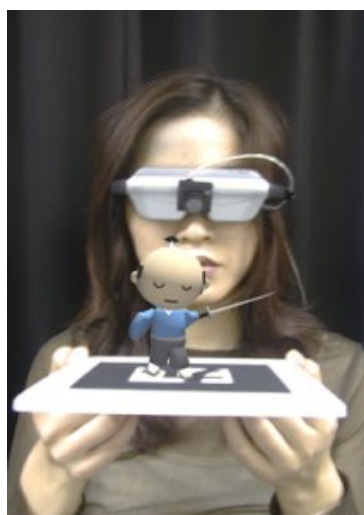


Figura 2.4: ARToolKit y marcadores visuales (1999)

Entre 2001 y 2004, surgieron los primeros sistemas de RA sobre dispositivos móviles, como PDAs y teléfonos inteligentes de primera generación. En 2003, Wagner y Schmalstieg presentaron un sistema de RA completamente autónomo en un dispositivo de mano [27]. Otros experimentos destacados fueron ARQuake (2000), un videojuego de RA en primera persona, y Invisible Train (2004), una aplicación colaborativa con varios usuarios simultáneos.

2.2.4. Consolidación y tendencias actuales (2010–2025)

Durante la última década, la Realidad Aumentada ha pasado de ser una tecnología emergente a consolidarse como una herramienta madura y transversal en múltiples sectores. El desarrollo de kits como ARToolKit, ARCore (Google) y ARKit (Apple) permitió la integración de funcionalidades de RA en dispositivos móviles de forma nativa, facilitando su adopción masiva y el desarrollo de aplicaciones accesibles sin necesidad de hardware especializado [28, 29].

Uno de los catalizadores clave de esta expansión fue el éxito comercial de Pokémon GO (2016), que demostró el potencial de la RA móvil como forma de interacción lúdica y geolocalizada [30]. Paralelamente, surgieron dispositivos avanzados como Microsoft HoloLens y Magic Leap One, que ofrecieron experiencias inmersivas mediante visores ópticos, reconocimiento espacial y control gestual [31, 32].

En el plano técnico, los avances en visión por computador y en algoritmos de localización y mapeo simultáneo (SLAM) mejoraron notablemente la precisión del registro espacial tridimensional [33]. También se consolidaron nuevas formas de interacción como la RA tangible y la RA colaborativa, que permiten manipular objetos físicos para modificar contenidos digitales o interactuar con varios usuarios en tiempo real [34, 35].

Actualmente, la RA se aplica con éxito en campos como la educación, la salud, la industria, la arquitectura o el comercio, y su desarrollo se orienta hacia una fusión más natural entre el mundo físico y el digital mediante técnicas de spatial computing y nuevos dispositivos como el Apple Vision Pro (2024) [36].

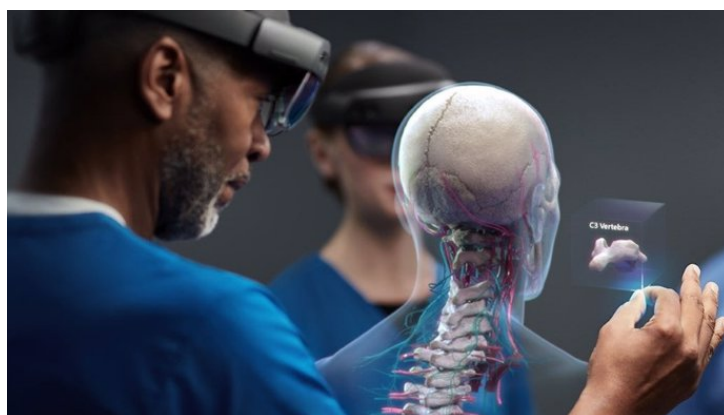


Figura 2.5: HoloLens 2 de Microsoft en labores de asistencia remota y formación.

2.3. La visibilización de las mujeres en el espacio urbano: una cuestión de justicia histórica

La representación de las mujeres en el espacio urbano continúa siendo escasa. Según el proyecto europeo “Women’s Legacy” [37], impulsado desde la Conselleria d’Educació, Cultura i Esport de la Generalitat Valenciana, solo un 10 % de las calles con nombres de personas en muchas ciudades españolas están dedicadas a mujeres, y este patrón se repite en monumentos, placas conmemorativas y espacios simbólicos.

A pesar de esta disparidad, en los últimos años han surgido iniciativas locales e institucionales orientadas a revertir esta situación. En Valencia, destaca el proyecto “Dones de Ciència” [38], una colaboración entre la Universitat Politècnica de València (UPV) y Las Naves, que ha creado más de 30 murales urbanos en centros educativos para homenajear a mujeres científicas de todo el mundo. Esta intervención artística no solo embellece el entorno, sino que actúa como una herramienta pedagógica para inspirar a las nuevas generaciones. Asimismo, el Ayuntamiento de València ha promovido campañas de renombramiento de calles con nombres femeninos y la inclusión de mujeres en los itinerarios culturales y turísticos de la ciudad [39].

Por otra parte, diversas asociaciones vecinales y colectivos feministas han impulsado rutas autogestionadas por la ciudad que visibilizan las huellas femeninas en el espacio urbano. Entre ellas destaca la Ruta Violeta de Mujeres en Valencia", organizada por la Asociación Por Ti Mujer, que recorre espacios emblemáticos ligados a la historia de las mujeres en la ciudad, promoviendo un reconocimiento público de sus contribuciones sociales y culturales [40].

Asimismo, la Asamblea Feminista de València ha organizado la Ruta pels espais del Patronato de Protección a la Mujer", que recorre espacios vinculados a esta institución franquista, la cual operó durante gran parte del siglo XX bajo la tutela de la Iglesia y el Estado. Su función no era tanto proteger a mujeres víctimas de violencia como controlar y castigar a aquellas que transgredían los roles tradicionales de género. Esta ruta permite evidenciar cómo las estructuras institucionales legitimaron la exclusión, la represión y la estigmatización de mujeres consideradas "desviadas" por la moral conservadora, visibilizando así dinámicas de resistencia y memoria colectiva en la ciudad [41].

Además, la “Ruta de Memorias Lesbianas”, también organizada por la Asamblea Femi-

nista de València, recorre lugares significativos para la comunidad lésbica, con el objetivo de visibilizar los derechos y experiencias de las mujeres lesbianas en la Valencia de los años 70, 80 y 90, destacando la importancia de reconocer y valorar estas luchas en la construcción de la historia urbana [42].

Estas rutas permiten redescubrir barrios desde una óptica feminista, incorporando historias personales, luchas sociales y legados invisibilizados. En este sentido, la plataforma DONA'm MÓN podría ser un recurso útil a implementar durante estas rutas, integrando información geolocalizada, recursos multimedia y facilitando la participación activa de las personas en la construcción de esta memoria feminista urbana.

2.3.1. Referentes femeninos en la ciudad: biografías y puntos de interés

La aplicación propuesta busca poner en valor la figura de mujeres que, desde distintos campos del saber y la acción, han contribuido de manera decisiva al progreso de los derechos, el pensamiento y la cultura. Aunque muchas de ellas han nacido o trabajado en Valencia, su legado trasciende el ámbito local y constituye una aportación fundamental a la historia del feminismo y la igualdad. A través de la geolocalización de lugares clave asociados a sus vidas y obras, se propone una experiencia que va más allá de la información enciclopédica, para convertirse en un ejercicio de reconocimiento social y emocional de su presencia en la ciudad.

Una de las figuras destacadas es Concepción Aleixandre Ballester (1862–1952), una de las primeras mujeres médicas en España y pionera en el ámbito de la ginecología. Su activismo feminista se centró en la defensa del acceso de las mujeres a la educación y al ejercicio profesional de la medicina, algo profundamente innovador en su tiempo. Estudió en la Universidad de Valencia, donde enfrentó obstáculos institucionales por su condición de mujer, y desarrolló una intensa carrera médica y científica. El edificio histórico de la Facultad de Medicina de la Universitat de València es un lugar simbólico para recordar su figura y su lucha por la igualdad en el ámbito sanitario [43].

Otra mujer fundamental en la historia reciente de Valencia es Carmen Alborch (1947–2018). Fue ministra de Cultura, escritora, catedrática de Derecho Mercantil y una de las voces más influyentes del feminismo institucional en España. Su obra literaria, como *Solas* (1999), promovió un feminismo humanista accesible y empático. En Valencia, dejó una impronta profunda como directora del Instituto Valenciano de Arte Moderno (IVAM), donde impulsó una política cultural abierta, plural y comprometida con la igualdad. El IVAM, por tanto, no solo representa su labor como gestora cultural, sino también su apuesta por un feminismo integrador desde las instituciones [44].

En el ámbito de los derechos civiles, Clara Campoamor (1888–1972) ocupa un lugar esencial por su incansable lucha por el sufragio femenino, que logró defender con éxito en las Cortes Constituyentes de 1931. Si bien no tuvo una vinculación directa con Valencia, su legado ideológico inspiró a generaciones de mujeres en toda España, incluida la Comunidad Valenciana. La existencia de centros educativos como el CEIP Clara Campoamor subraya la importancia de su figura como referente pedagógico y su impacto en la formación de una conciencia feminista desde edades tempranas [45].

Retrocediendo a la Edad Media, encontramos a Isabel de Villena (1430–1490), abadesa del Real Monasterio de la Trinidad y considerada la primera escritora en lengua valenciana. Su obra *Vita Christi* constituye una relectura humanista del relato evangélico desde una

perspectiva femenina, resaltando la figura de la Virgen y otras mujeres bíblicas. Isabel de Villena transformó el espacio monástico en un lugar de producción intelectual y espiritual liderado por mujeres, algo excepcional en su época. El Monasterio de la Trinidad, donde vivió y escribió, es un lugar clave para reivindicar la aportación femenina a la literatura y la espiritualidad valencianas [46].

En la medicina, Manuela Solís Clarás (1888–1936) destaca por ser una de las primeras mujeres médicas de la ciudad. Ejerció en el Hospital Provincial de Valencia y fue una firme defensora del acceso de las mujeres a la formación sanitaria. Además de su práctica médica, se implicó en la divulgación científica y en la promoción de redes de apoyo entre profesionales sanitarias, en un momento en que la medicina era un espacio profundamente masculinizado [47]. El Hospital Provincial es, por tanto, un espacio idóneo para recordar su legado.

La aplicación también incluye representaciones artísticas contemporáneas de mujeres de gran relevancia internacional. Por ejemplo, Mae Jemison, primera mujer afroamericana en viajar al espacio, es homenajeada con un mural en el CEIP Torrefiel. Su figura simboliza la superación de barreras raciales y de género en los campos de la ciencia y la exploración espacial. Del mismo modo, Vandana Shiva, ecofeminista india y activista medioambiental, cuenta con un mural en el CEIP Ballester Fandos. Su lucha por la justicia ambiental, la soberanía alimentaria y los derechos de las mujeres rurales conecta con un feminismo global que también debe tener presencia en las narrativas educativas locales [48].

“DONA’m MÓN” busca ser un vehículo para visibilizar la presencia de estas figuras en el espacio urbano, contribuyendo así a una representación más justa y equitativa de la historia y promoviendo un futuro en el que la diversidad y la igualdad sean valores fundamentales.

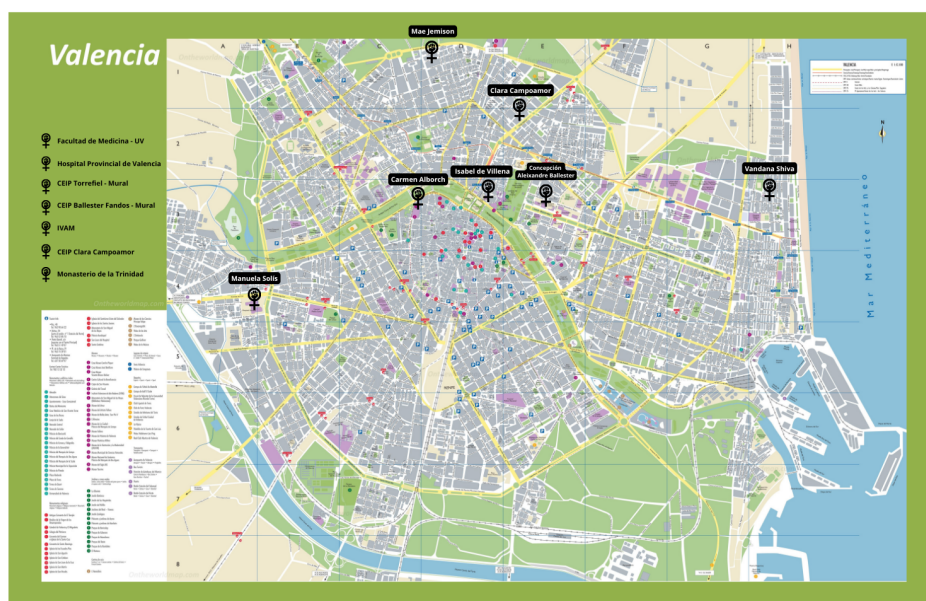


Figura 2.6: Boceto inicial del mapa de Valencia con los puntos geolocalizados de las mujeres referentes incluidos en la aplicación.

2.4. Análisis de aplicaciones similares

En el desarrollo de “DONA’*m* MÓN”, resulta esencial analizar aplicaciones existentes que, de manera similar, combinan geolocalización y realidad aumentada (RA) para ofrecer experiencias inmersivas y educativas. Este análisis nos permite identificar características, estrategias y tecnologías clave que podrían ser adaptadas para cumplir con los objetivos específicos del proyecto. A continuación, se presentan algunas aplicaciones relevantes cuyos enfoques, funcionalidades y tecnologías han inspirado aspectos concretos del diseño conceptual de este TFG.

2.4.1. Pokémon GO

Una de las referencias más notables es Pokémon GO, que revolucionó el mercado de aplicaciones móviles al combinar geolocalización y RA para motivar a los usuarios a explorar el mundo real. En este caso, los jugadores buscan y capturan criaturas virtuales que aparecen en puntos geográficos específicos. La aplicación también implementa un sistema de recompensas y niveles que fomenta la participación constante. Aunque el enfoque principal de Pokémon GO es el entretenimiento, su éxito demuestra cómo las tecnologías inmersivas pueden transformar la forma en que las personas interactúan con su entorno. En “DONA’*m* MÓN”, se busca adaptar esta dinámica de exploración y descubrimiento, pero con un enfoque educativo y cultural, utilizando los puntos de interés como portales hacia las historias de mujeres que dejaron una huella significativa en Valencia.

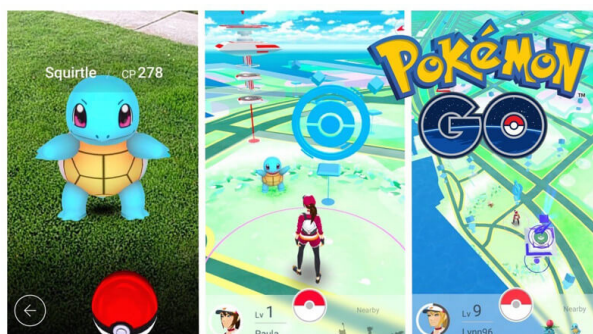


Figura 2.7: Interfaz de Pokémon GO mostrando un entorno real aumentado con la aparición de un Pokémon virtual y también el entorno virtual del juego.

2.4.2. Streetmuseum

Por otro lado, Streetmuseum es una aplicación que también aprovecha la RA, pero con un objetivo más centrado en la historia y la cultura. Esta herramienta permite a los usuarios visualizar imágenes y escenas históricas superpuestas al paisaje actual, ofreciendo una perspectiva del pasado en tiempo real. La capacidad de conectar visualmente el entorno moderno con eventos y figuras históricas es una inspiración directa para el proyecto. En nuestro proyecto, esta idea se adapta al permitir que los usuarios accedan a contenido multimedia –como imágenes, textos y modelos 3D– que contextualice la vida y obra de las mujeres en cada ubicación geolocalizada.



Figura 2.8: Ejemplo de Streetmuseum: visualización de una escena histórica superpuesta sobre el entorno actual de la ciudad.

2.4.3. CultuAR

CultuAR, desarrollada por la empresa española AR Vision, es una aplicación que combina la geolocalización con la realidad aumentada para ofrecer visitas culturales interactivas en más de 200 municipios. Mediante la cámara del dispositivo, los usuarios pueden visualizar reconstrucciones históricas, personajes del pasado o información multimedia superpuesta al entorno real. Una de sus características más valiosas es la capacidad de personalizar rutas culturales por municipios, integrando elementos inmersivos que enriquecen la experiencia turística y patrimonial. Esta aplicación destaca por su integración fluida con el entorno urbano y su enfoque didáctico, lo que representa una clara inspiración para *DONA'm MÓN* en cuanto a accesibilidad y diseño de experiencias geolocalizadas con RA.



Figura 2.9: Captura de CultuAR mostrando información sobre un monumento en el entorno real.

2.4.4. Historypin

Finalmente, Historypin se presenta como una plataforma colaborativa que permite a los usuarios subir y explorar fotografías, eventos y recuerdos históricos asociados a ubicaciones específicas. Lo más destacable de esta aplicación es su enfoque participativo, donde los usuarios pueden contribuir activamente al contenido disponible. Este modelo de colaboración inspira la posibilidad de que, en futuras versiones de *DONA'm MÓN*, los

usuarios también puedan agregar información o enriquecer las narrativas existentes sobre mujeres de Valencia, convirtiendo la aplicación en una herramienta dinámica y viva.

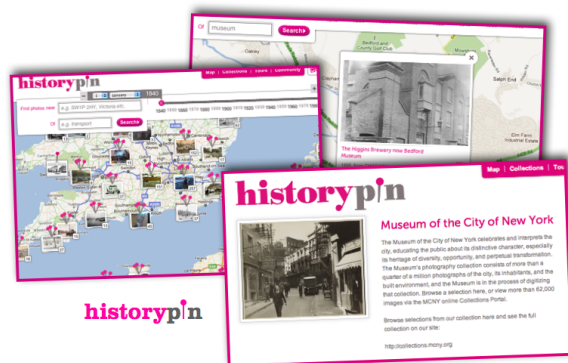


Figura 2.10: Interfaz de Historypin donde se visualizan fotografías históricas asociadas a ubicaciones reales en el mapa y su detalle de contenido.

Estas cuatro aplicaciones ilustran diversas maneras de utilizar la tecnología para crear experiencias inmersivas e interactivas. Cada una de ellas presenta fortalezas únicas que se alinean, en mayor o menor medida, con los objetivos de *DONA'm MÓN*. De Pokémon GO tomamos la dinámica de exploración y gamificación; de Streetmuseum, la idea de superponer contenido histórico en el entorno actual; de CultuAR, el uso efectivo de RA para la divulgación cultural local; y de Historypin, el potencial de colaboración comunitaria. Esta combinación de estrategias y conceptos pretende hacer de *DONA'm MÓN* una aplicación única que conecte a los usuarios con la historia de manera inclusiva, educativa e innovadora.

2.5. Tecnologías

En este apartado se analizan las tecnologías disponibles y las soluciones existentes en el mercado que pueden ser aplicadas al desarrollo de la aplicación móvil. Este análisis incluye una revisión de herramientas de geolocalización, realidad aumentada, bases de datos y frameworks de desarrollo multiplataforma. La selección de las tecnologías se justifica considerando la funcionalidad, compatibilidad y eficiencia necesarias para cumplir con los objetivos del proyecto.

2.5.1. Plataformas de desarrollo de aplicaciones móviles

Una de las decisiones fundamentales en cualquier proyecto de ingeniería de software es la elección de la tecnología base sobre la cual se construirá la aplicación. Esta decisión afecta directamente a la arquitectura general, la experiencia de usuario, la mantenibilidad del código y la escalabilidad futura del sistema. En el contexto de este proyecto, se evaluaron varias opciones para el desarrollo de una aplicación móvil multiplataforma que integrase geolocalización, funcionalidades de realidad aumentada, y una interfaz moderna, accesible y responsiva.

Unity. Unity es un motor de desarrollo multiplataforma ampliamente utilizado en los ámbitos del videojuego, simulación y experiencias interactivas en tiempo real. Su potencia radica en su motor gráfico, que permite renderizar contenido 3D de alta calidad, así como en su flexibilidad para implementar lógica de aplicación mediante scripts en C#. Unity ofrece soporte directo para tecnologías de realidad aumentada a través de módulos como AR Foundation, que actúan como capa de abstracción sobre ARKit (iOS) y ARCore (Android), permitiendo así desarrollar una única base de código RA para múltiples plataformas.

En lo relativo a interfaces de usuario, Unity ha evolucionado en los últimos años. Su sistema tradicional basado en el Canvas permitía crear interfaces gráficas bidimensionales, pero con limitaciones de estilo y escalabilidad. Como respuesta a estas deficiencias, se introdujo UI Toolkit, un nuevo sistema de interfaz basado en un modelo declarativo, inspirado en tecnologías web como HTML y CSS. UI Toolkit permite separar la lógica de presentación del comportamiento, definiendo componentes visuales reutilizables y estilos en archivos dedicados. A pesar de estas mejoras, el desarrollo de interfaces ricas, formularios interactivos y navegación compleja continúa siendo menos eficiente que en frameworks específicos para aplicaciones móviles.

React Native. React Native es un framework desarrollado por Meta (anteriormente Facebook) para el desarrollo de aplicaciones móviles multiplataforma utilizando JavaScript y el modelo de componentes de React. Permite crear interfaces de usuario nativas reutilizando componentes declarativos que se renderizan mediante puentes hacia los componentes nativos de Android e iOS. Una de las grandes ventajas de React Native es su arquitectura reactiva, que permite gestionar estados complejos de forma eficiente, manteniendo la UI sincronizada con los datos de la aplicación.

En combinación con Expo, un conjunto de herramientas que facilita la creación y prueba de aplicaciones React Native, es posible desarrollar aplicaciones sin necesidad de compilar ni configurar código nativo. Esto permite utilizar herramientas como Expo Go para probar la aplicación directamente en un dispositivo físico mediante código QR, lo que acelera significativamente el flujo de desarrollo. React Native es especialmente potente para construir interfaces modernas, personalizadas y adaptables, con soporte para navegación compleja, formularios, animaciones y estilos dinámicos.

Flutter. Flutter es un framework desarrollado por Google para construir aplicaciones multiplataforma desde una única base de código. Utiliza el lenguaje Dart, y su arquitectura se basa en un sistema de renderizado propio que dibuja todos los elementos de la UI desde cero, lo que garantiza uniformidad visual entre plataformas. Flutter ofrece una amplia colección de widgets personalizables y anima la creación de interfaces complejas de forma declarativa y eficiente.

Aunque Flutter no fue la opción elegida para este proyecto, se considera una alternativa competitiva frente a React Native. Sin embargo, su adopción requiere dominar el lenguaje Dart, y su integración con tecnologías de realidad aumentada aún está en evolución en comparación con la madurez del ecosistema de Unity o la compatibilidad simplificada que ofrece React Native con herramientas web como AR.js y A-Frame.

Desarrollo de interfaz de usuario y elección final. Una parte crucial del proyecto es la interfaz gráfica de usuario, dado que la aplicación está destinada a un público general y debe facilitar la navegación intuitiva, la visualización de datos geográficos y el acceso a contenidos de realidad aumentada. Si bien Unity, especialmente con UI Toolkit, ofrece

un entorno funcional para construir interfaces gráficas, su enfoque está más orientado a interacciones en entornos tridimensionales y menos a la gestión de flujos de navegación o formularios comunes en aplicaciones móviles.

Por el contrario, React Native destaca precisamente en este tipo de interfaz. Su modelo declarativo, su sistema de componentes reutilizables, su integración con herramientas modernas de desarrollo (como React Navigation, Redux o Context API), y su compatibilidad con Expo, permiten construir aplicaciones móviles escalables y con una experiencia de usuario optimizada.

Por estos motivos, y tras un análisis técnico detallado, se decidió utilizar React Native con Expo como entorno principal de desarrollo para esta aplicación, reservando el uso de tecnologías gráficas (como la realidad aumentada) a componentes embebidos que se integran dentro de esta arquitectura general. Esta elección permite combinar la robustez de React para la UI con la flexibilidad de integrar experiencias multimedia mediante tecnologías web.

2.5.2. Geolocalización

La geolocalización es uno de los pilares fundamentales de la aplicación, ya que permite situar puntos de interés específicos en un mapa, facilitando que los usuarios exploren la ciudad de Valencia mientras descubren la historia de mujeres destacadas. Para implementar esta funcionalidad, existen diversas herramientas ampliamente utilizadas en el desarrollo de aplicaciones móviles:

- **Google Maps API:** Es una solución robusta y versátil que permite integrar mapas interactivos, obtener datos de ubicación en tiempo real y personalizar la visualización según las necesidades del proyecto. Ofrece documentación detallada y soporte técnico, lo que facilita su integración. Sin embargo, su principal desventaja es que su plan gratuito únicamente se ofrece durante un periodo de prueba de corta duración y su coste se incrementa en proyectos con un gran volumen de usuarios o solicitudes [49].
- **Mapbox:** Alternativa potente que combina funcionalidades avanzadas, un alto nivel de personalización estética y una política de precios más flexible que Google Maps API, ya que su plan gratuito no tiene un límite de tiempo. Es especialmente útil en aplicaciones que requieren un diseño visual distintivo. No obstante, su integración puede resultar más compleja para desarrolladores con poca experiencia [50].
- **OpenStreetMap (OSM):** Opción de código abierto que permite el acceso y uso gratuito de datos cartográficos, además de ofrecer un alto nivel de personalización. Esta plataforma colaborativa es especialmente adecuada para proyectos con presupuestos limitados o académicos, ya que no implica costes de licencia. Aunque carece de algunas funcionalidades avanzadas nativas, como el enrutamiento en tiempo real o servicios de geolocalización enriquecidos, su flexibilidad y libertad de uso la convierten en una solución funcional y accesible [51].
- **Mapas nativos del dispositivo con React Native:** Esta opción permite aprovechar las capacidades de geolocalización del propio sistema operativo del dispositivo móvil (iOS o Android) mediante bibliotecas como react-native-maps. Es una solución eficiente y ligera, que facilita la integración con el hardware del dispositivo y proporciona una experiencia fluida. Además, evita costes derivados de servicios ex-

ternos, lo que la convierte en una alternativa atractiva para proyectos con recursos limitados [52].

En este proyecto se ha optado por utilizar mapas nativos con React Native, ya que permiten una integración directa con la aplicación móvil, ofrecen un rendimiento óptimo, y eliminan la dependencia de servicios externos de pago. Esta elección responde a criterios de eficiencia, sostenibilidad económica y control sobre el desarrollo.

2.5.3. Realidad Aumentada (RA)

Una vez definida la arquitectura general de la aplicación y confirmado el uso de React Native como entorno principal de desarrollo, fue necesario analizar detalladamente qué tecnologías de realidad aumentada (RA) podían integrarse de forma eficiente, respetando las limitaciones del ecosistema de desarrollo adoptado. Uno de los objetivos fundamentales era conservar la compatibilidad con Expo Go, evitando procesos de compilación nativa que dificultasen el flujo de desarrollo iterativo. Bajo esta premisa, se realizó un estudio de las distintas alternativas existentes, evaluando tanto su madurez técnica como su grado de integración con React Native.

Realidad aumentada con tecnologías nativas. Las soluciones nativas ofrecen, en general, el mayor grado de precisión, rendimiento y acceso a capacidades avanzadas del dispositivo. Entre estas destacan ARKit (para dispositivos iOS) y ARCore (para Android), los kits de desarrollo oficial proporcionados por Apple y Google, respectivamente. Ambas plataformas permiten el seguimiento del entorno, la detección de planos horizontales y verticales, la oclusión de objetos virtuales con respecto al entorno físico, el anclaje espacial y la iluminación adaptativa. Estas herramientas están diseñadas para crear experiencias inmersivas de alta calidad y son ampliamente utilizadas en entornos profesionales y comerciales.

Sin embargo, una de sus limitaciones fundamentales es su naturaleza monoplataforma. Para desarrollar una aplicación de RA que funcione tanto en Android como en iOS, sería necesario implementar dos versiones independientes, o bien utilizar un entorno unificador como Unity, que permite compilar una única base de código para múltiples sistemas operativos.

Unity, a través de su módulo AR Foundation, permite abstraer las diferencias entre ARKit y ARCore, y desarrollar experiencias RA multiplataforma con una arquitectura común. Unity es particularmente potente en términos de renderizado gráfico, gestión de modelos tridimensionales, simulación física y animación en tiempo real, por lo que constituye una de las herramientas más robustas para el desarrollo de RA avanzada.

No obstante, su integración con aplicaciones desarrolladas en React Native es compleja. El enfoque convencional implicaría desarrollar toda la aplicación en Unity, lo cual, como se ha discutido anteriormente, penaliza la usabilidad y escalabilidad de la interfaz. Por este motivo, se exploró también la posibilidad de utilizar Unity como una librería nativa embebida dentro de una aplicación React Native.

Unity como librería nativa integrada en React Native. Unity permite exportar un proyecto como una librería nativa —un archivo .aar para Android o .framework para iOS— que puede integrarse dentro de una aplicación móvil desarrollada con otras tecnologías, como React Native. Este enfoque, conocido como Unity as a Library, ofrece la posibilidad de desarrollar únicamente la parte de RA en Unity y exponerla como un

módulo invocable desde la aplicación principal.

Mediante este patrón arquitectónico, es posible mantener la lógica general y la interfaz de usuario desarrolladas en React Native, y delegar en Unity únicamente el renderizado y gestión de la escena RA. La comunicación entre ambos entornos se realiza mediante puentes nativos que deben implementarse en Java/Kotlin (para Android) y Objective-C/Swift (para iOS).

Este enfoque ha sido demostrado en proyectos como react-native-unity, que ofrecen ejemplos funcionales de integración, así como en la documentación oficial de Unity (Unity as a Library). No obstante, esta integración presenta múltiples desafíos técnicos:

- Es necesario ejectar el proyecto Expo, abandonando así la posibilidad de utilizar Expo Go y el flujo de desarrollo simplificado que ofrece.
- La configuración del proyecto requiere conocimientos avanzados en desarrollo nativo para cada plataforma.
- La integración y sincronización de eventos entre Unity y React Native debe implementarse manualmente.

Por tanto, aunque Unity como librería nativa representa una solución muy potente desde el punto de vista gráfico y funcional, su coste de integración y mantenimiento dentro de una arquitectura basada en React Native la convierte en una opción poco viable para este proyecto, cuyo foco está en la accesibilidad, rapidez de desarrollo y compatibilidad multiplataforma sin compilación nativa.

Realidad aumentada en React Native con tecnologías web. En busca de una alternativa más ligera y plenamente compatible con React Native y Expo, se optó por explorar las soluciones basadas en tecnologías web. Una de las más destacadas es AR.js, una librería de código abierto que permite implementar experiencias de RA directamente en navegadores móviles, sin necesidad de acceso a capas nativas. AR.js se basa en WebGL, Three.js y A-Frame, y permite implementar RA sobre marcadores visuales (marker-based AR) mediante la detección de patrones gráficos como Hiro, Kanji o diseños personalizados.

Complementando AR.js, el framework A-Frame facilita la construcción declarativa de escenas 3D y RA mediante HTML. Permite importar modelos .glb, añadir iluminación, cámaras, interacciones y animaciones sin necesidad de escribir código complejo de bajo nivel. Esta aproximación reduce significativamente la barrera de entrada al desarrollo de RA, y se adapta perfectamente al modelo de desarrollo web moderno.

Las escenas construidas con A-Frame y AR.js pueden ser desplegadas como aplicaciones web estáticas (por ejemplo, en GitHub Pages), e integradas dentro de la aplicación React Native utilizando el componente WebView, que actúa como un navegador embebido. Este enfoque permite mantener la lógica general y la interfaz de la aplicación dentro del ecosistema de React Native, mientras que la experiencia de RA se encapsula como una unidad independiente, reutilizable y fácilmente actualizable.

Viro React. Otra alternativa evaluada fue Viro React, un framework orientado a la creación de experiencias inmersivas en RA y realidad virtual directamente en React Native. Viro permite cargar modelos 3D, definir cámaras, aplicar animaciones, detectar superficies, y renderizar objetos virtuales en función del entorno físico. Su enfoque declarativo está alineado con la filosofía de React y ofrece un entorno de desarrollo intuitivo para experiencias visuales.

No obstante, Viro no es compatible con Expo Go, ya que depende de módulos nativos que requieren compilar la aplicación. Para utilizarlo, es necesario ejectar el proyecto y utilizar herramientas como EAS Build para generar versiones personalizadas de la app. Esta necesidad de compilación rompe el flujo de desarrollo ágil basado en Expo y contradice uno de los principios técnicos fundamentales del proyecto: evitar configuraciones nativas complejas para preservar la portabilidad y escalabilidad del sistema.

Después de un análisis riguroso, se optó por utilizar AR.js combinado con A-Frame, integrados en la aplicación React Native mediante un componente WebView. Esta solución permite cumplir los objetivos funcionales de la aplicación (visualización de modelos 3D sobre marcadores físicos), respetando al mismo tiempo los requisitos técnicos del entorno (compatibilidad con Expo, desarrollo ágil, y arquitectura limpia).

La elección de esta tecnología responde a criterios técnicos sólidos:

- Portabilidad total: la experiencia RA se ejecuta en el navegador y es accesible desde cualquier dispositivo moderno.
- Desacoplamiento arquitectónico: la escena RA es independiente del código base de la aplicación.
- Facilidad de mantenimiento: se pueden actualizar las escenas de RA sin necesidad de recompilar la app.
- Compatibilidad con Expo Go: se preserva el flujo de desarrollo iterativo y multiplataforma.

En resumen, esta arquitectura permite combinar las ventajas del desarrollo móvil moderno (mediante React Native) con la flexibilidad de tecnologías web para RA, consiguiendo una solución técnica equilibrada, funcionalmente completa y sostenible a largo plazo.

2.5.4. Base de Datos

2.5.5. Arquitectura de persistencia y gestión de datos

En el desarrollo de esta aplicación se ha optado por una arquitectura desacoplada que separa claramente el frontend y el backend. El frontend se implementa en React Native, lo que permite el despliegue multiplataforma en dispositivos móviles, mientras que el backend se desarrolla en Django, un framework web de alto nivel que facilita la creación de aplicaciones robustas y seguras [53].

Persistencia de datos en entornos relacionales y no relacionales

La persistencia de datos es un aspecto fundamental en el desarrollo de aplicaciones que requieren almacenar información de manera duradera. Existen dos enfoques principales para gestionar esta persistencia: las bases de datos relacionales, que estructuran la información en tablas con relaciones definidas mediante claves, y las bases de datos no relacionales, que permiten una mayor flexibilidad al almacenar los datos en formatos como documentos, grafos o pares clave-valor [3].

Las bases de datos relacionales son especialmente adecuadas cuando los datos presentan una estructura definida, relaciones entre entidades y necesidad de integridad referen-

cial, mientras que las no relacionales ofrecen ventajas en contextos donde la escalabilidad horizontal, la alta disponibilidad o la variabilidad en los esquemas de datos son prioridades [54].

Django y la arquitectura MVT

Django se basa en la arquitectura Modelo–Vista–Plantilla (MVT), un patrón similar al clásico MVC (Model–View–Controller), adaptado al ecosistema web de Python [55]. Este patrón organiza el código en tres componentes:

- **Modelo (Model):** Define la estructura de los datos y su comportamiento a través de clases Python, que se traducen automáticamente en tablas mediante el ORM (Object-Relational Mapping) de Django.
- **Vista (View):** Gestiona la lógica de negocio y responde a las solicitudes HTTP, procesando la información que llega desde el frontend.
- **Plantilla (Template):** Presenta la información al usuario utilizando HTML enriquecido con etiquetas dinámicas, aunque en este proyecto su papel es mínimo, ya que la interfaz se renderiza completamente en React Native.

La interacción entre Django y la base de datos está mediada por su ORM, lo que permite manipular los datos mediante objetos Python en lugar de consultas SQL explícitas. Esta capa de abstracción incrementa la seguridad —previniendo ataques como la inyección SQL— y facilita el mantenimiento del código [55].

Sistema de Gestión de Bases de Datos: PostgreSQL

Para este proyecto se ha elegido PostgreSQL como sistema de gestión de bases de datos relacional (SGBDR), debido a su robustez, características avanzadas y excelente compatibilidad con Django [4]. PostgreSQL es conocido por su cumplimiento estricto de los estándares SQL, su capacidad para manejar tipos de datos complejos (incluyendo JSON, arrays y tipos geoespaciales), y su rendimiento superior en consultas complejas. Estas características resultan fundamentales para gestionar tanto la información relacionada con las figuras históricas como los datos de geolocalización, multimedia y analíticas del uso de la aplicación.

La configuración entre Django y PostgreSQL se realiza mediante el uso de controladores compatibles, lo que asegura una integración fluida y un rendimiento óptimo. Además, PostgreSQL ofrece extensiones avanzadas como PostGIS para el manejo de datos geoespaciales, lo que puede resultar útil para futuras mejoras en las funcionalidades de geolocalización.

Comparativa entre sistemas de bases de datos

A continuación se muestra una tabla comparativa entre los principales sistemas de gestión de bases de datos considerados:

Característica	SQLite	MySQL	PostgreSQL	MariaDB	MongoDB
Tipo	Relacional	Relacional	Relacional	Relacional	No relacional
Escalabilidad	Baja	Alta	Muy alta	Alta	Muy alta
Rendimiento	Medio	Alto	Muy alto	Muy alto	Alto
Integridad referencial	Sí	Sí	Sí	Sí	No
Compatibilidad Django	Nativa	Completa	Excelente	Completa	Parcial (con plugins)
Flexibilidad de esquema	Baja	Media	Alta	Media	Muy alta
Facilidad de configuración	Muy fácil	Media	Media	Media	Media
Tipos de datos avanzados	No	Limitado	Excelente	Limitado	Excelente
Extensiones geoespaciales	No	Limitado	Excelente (PostGIS)	Limitado	Bueno
Comunidad y soporte	Moderada	Muy activa	Muy activa	Activa	Muy activa

Cuadro 2.2: Comparativa entre SQLite, MySQL, PostgreSQL, MariaDB y MongoDB [3, 4, 5, 6].

Consideraciones sobre otras tecnologías: MariaDB, SQLite, MySQL, MongoDB y Cassandra

Aunque inicialmente se contempló el uso de MySQL, una solución madura y ampliamente utilizada, finalmente se optó por PostgreSQL debido a sus características avanzadas y su mejor soporte para tipos de datos complejos [4]. PostgreSQL ofrece ventajas significativas como el soporte nativo para JSON, arrays, tipos geoespaciales mediante PostGIS, y un motor de consultas más sofisticado que resulta especialmente útil para aplicaciones que manejan datos diversos y complejos.

MariaDB, una bifurcación de MySQL con mejoras en rendimiento y licencias completamente abiertas, también fue considerada, pero PostgreSQL superó esta opción por su ecosistema más rico de extensiones y su mejor integración con Django [5]. SQLite, si bien está soportada nativamente por Django y resulta útil en fases de desarrollo o en aplicaciones de bajo requerimiento, no ofrece la escalabilidad ni la gestión de concurrencia necesarias para este proyecto [56].

En cuanto a las bases de datos no relacionales, cabe mencionar tecnologías como MongoDB y Cassandra. MongoDB, basada en documentos JSON, es ampliamente utilizada en aplicaciones que requieren estructuras de datos flexibles y cambios frecuentes en el esquema [6]. Cassandra, por su parte, está orientada a entornos distribuidos con grandes volúmenes de datos y alta disponibilidad [57]. Sin embargo, el enfoque relacional de PostgreSQL fue considerado más adecuado para las necesidades estructuradas y normalizadas del presente trabajo, ofreciendo además la flexibilidad necesaria para futuras expansiones del sistema.

Capítulo 3

Requisitos, especificaciones, coste, riesgos, viabilidad

3.1. Requisitos

3.1.1. Requisitos funcionales

Los requisitos funcionales describen las acciones que el sistema debe poder realizar para satisfacer las necesidades del usuario y del administrador. En este proyecto, se centran en el uso de geolocalización, realidad aumentada, gestión de contenido y experiencia interactiva.

Lista de requisitos funcionales:

3.1.2. Requisitos funcionales

- RF1: La aplicación debe permitir a los usuarios visualizar un mapa interactivo con los puntos geolocalizados de interés.
- RF2: El sistema debe detectar la ubicación actual del usuario mediante GPS.
- RF3: Al acercarse físicamente a un punto geolocalizado, la aplicación debe desbloquear el contenido de RA asociado.
- RF4: Cada punto geolocalizado debe estar vinculado a una mujer histórica, con contenido educativo multimedia.
- RF5: La aplicación debe mostrar una ficha informativa con texto, imágenes y/o audio sobre cada figura femenina.
- RF6: La app debe permitir visualizar elementos de realidad aumentada (como imágenes, objetos 3D o figuras animadas).
- RF7: La lista de puntos y el mapa deben permitir filtrar los lugares por el ámbito de investigación de las mujeres representadas.
- RF8: El usuario debe poder ver qué puntos ha desbloqueado y cuáles no.
- RF9: La aplicación debe permitir consultar la lista completa de mujeres disponibles con sus respectivas ubicaciones.
- RF10: El usuario debe recibir una notificación cuando esté cerca de un punto no visitado.

- RF11: Debe existir un sistema de rutas que conecten diferentes puntos de interés temáticamente.
- RF12: Al pulsar sobre un punto en el mapa o la lista, debe abrirse su ficha detallada, desde la cual el usuario podrá marcar el lugar como visitado o no.
- RF13: El usuario debe disponer de un historial donde se registre la fecha y hora de cada punto visitado.
- RF14: El usuario debe disponer de un historial donde se registre la fecha y hora de cada ruta completada.
- RF15: La aplicación debe ser funcional tanto en dispositivos Android como iOS.
- RF16: Debe existir un panel de administración web para gestionar los puntos, contenidos y figuras históricas.
- RF17: El administrador debe poder añadir, editar o eliminar mujeres, ubicaciones y materiales multimedia desde el panel.
- RF18: El sistema debe permitir importar imágenes, vídeos y modelos 3D desde el panel de administración.
- RF19: El administrador debe poder gestionar áreas de investigación y rutas temáticas desde el panel de administración.
- RF20: La experiencia de RA debe ofrecer retroalimentación visual y sonora.
- RF21: El sistema debe enviar datos al backend para mantener sincronizado el progreso del usuario.
- RF22: La aplicación debe permitir a los usuarios registrarse y acceder mediante un sistema de inicio de sesión.
- RF23: Los usuarios deben poder iniciar sesión con un nombre de usuario y contraseña.
- RF24: La aplicación debe mantener la sesión iniciada hasta que el usuario decida cerrarla.
- RF25: El usuario debe poder editar los datos de su perfil desde la aplicación.
- RF26: El administrador debe poder acceder, desde el panel de gestión, a la lista de usuarios registrados.
- RF27: Al iniciar la aplicación, debe mostrarse una pantalla de bienvenida (splash screen) con el logotipo oficial antes de acceder a la pantalla principal.
- RF28: El administrador debe poder visualizar el perfil de cada usuario, incluyendo su progreso (puntos visitados).
- RF29: El sistema debe permitir al administrador eliminar usuarios o restablecer sus datos si es necesario.
- RF30: Al escanear un código QR de un lugar, se debe actualizar el progreso de rutas temáticas dentro de la aplicación.
- RF31: El sistema debe permitir filtrar los puntos por estado de visita (visitados o no visitados).
- RF32: Los filtros por estado de visita y por ámbito deben poder aplicarse de forma combinada.

- RF33: El listado de puntos debe estar ordenado por cercanía al usuario.
- RF34: La distancia en tiempo real entre el usuario y cada punto debe mostrarse junto a cada entrada del listado.
- RF35: La lista de puntos debe mostrar de forma resumida información del lugar y de la mujer representada.
- RF36: Desde la ficha detallada de un punto debe poder compartirse su contenido por redes sociales.
- RF37: El usuario debe poder acceder a una sección de rutas temáticas disponibles y ver los puntos que componen cada una.
- RF38: Los puntos de una ruta no pueden marcarse como visitados manualmente: sólo se desbloquean escaneando el código QR correspondiente.
- RF39: La ruta temática se debe marcar como completa y registrar en el historial del usuario al visitar todos los puntos
- RF40: Debe existir una pestaña específica con los puntos que disponen de realidad aumentada, accesibles si el usuario está dentro de un radio determinado.
- RF41: Al seleccionar un punto con RA disponible, debe abrirse la cámara y mostrarse el modelo correspondiente.
- RF42: El usuario debe poder consultar una sección de perfil donde ver su historial de lugares visitados y rutas completadas, y cerrar sesión.
- RF43: El sistema debe implementar un contexto global de filtros que permita mantener el estado entre diferentes pantallas.
- RF44: La aplicación debe calcular distancias precisas utilizando la fórmula de Haversine[58].
- RF45: El mapa debe sincronizar automáticamente los filtros aplicados en la pantalla de listado.
- RF46: El sistema debe permitir visualizar imágenes ampliadas en modal desde la pantalla de detalle.
- RF47: La aplicación debe procesar automáticamente las áreas de investigación del backend para crear una estructura plana de lugares.
- RF48: El sistema debe mantener un cache local de estado de visitas utilizando AsyncStorage.
- RF49: La aplicación debe mostrar indicadores visuales de filtros activos en tiempo real.
- RF50: El sistema debe permitir reiniciar el progreso de rutas completadas.
- RF51: El usuario debe poder buscar mujeres y lugares por nombre en tiempo real.

3.1.3. Requisitos no funcionales

Los requisitos no funcionales indican cómo debe comportarse el sistema más allá de lo que hace: establecen criterios de calidad, restricciones técnicas o normativas, y condiciones necesarias para su correcto desarrollo, despliegue y uso.

Requisitos no funcionales del producto

Estos requisitos definen las propiedades internas del sistema y sus comportamientos esperados, como rendimiento, usabilidad, seguridad o compatibilidad técnica.

- RNF1: La aplicación debe cargarse completamente en menos de 5 segundos desde su apertura.
- RNF2: El sistema debe ser capaz de gestionar al menos 50 usuarios simultáneos sin caída de rendimiento, con capacidad estimada para 100+ usuarios.
- RNF3: Los modelos 3D deben estar optimizados para dispositivos móviles, con un tamaño inferior a 20 MB por unidad.
- RNF4: La aplicación debe implementar optimizaciones de rendimiento como cache local de tokens, serialización eficiente de datos y consultas optimizadas al backend para minimizar el uso de recursos.
- RNF5: La interfaz debe ser intuitiva y permitir su uso sin formación previa.
- RNF6: La tipografía y botones deben adaptarse automáticamente a diferentes resoluciones de pantalla.
- RNF7: La aplicación debe ofrecer un modo accesible con audio narrado.
- RNF8: La navegación debe ser posible con una sola mano (diseño mobile-first).
- RNF9: Compatible con Android 10+ y iOS 13+.
- RNF10: El panel de administración debe ser accesible desde navegadores modernos (Chrome, Firefox, Safari, Edge).
- RNF11: Las contraseñas de los usuarios deben almacenarse cifradas en la base de datos.
- RNF12: La comunicación entre cliente y servidor debe estar cifrada mediante HTTPS.
- RNF13: El backend debe estar desarrollado en Django y utilizar PostgreSQL como sistema gestor de bases de datos.
- RNF14: La experiencia de realidad aumentada debe estar desarrollada con A-Frame y AR.js, desplegadas en Vercel e integradas a través de WebViews nativos.
- RNF15: La arquitectura debe ser modular y permitir añadir nuevos puntos o contenidos sin modificar el código base.
- RNF16: Se debe estructurar el código y documentarlo adecuadamente para facilitar mantenimiento y ampliación.

Requisitos no funcionales de la organización

Estos requisitos están relacionados con decisiones internas del equipo desarrollador o institución que afectan a los métodos de trabajo, tecnologías permitidas o estructura del proyecto.

- RNF17: El sistema debe permitir su despliegue en un servidor que soporte Django y PostgreSQL.
- RNF18: La aplicación utiliza el proveedor de mapas nativo del dispositivo (Google Maps en Android, Apple Maps en iOS).
- RNF19: El panel de administración debe incluir autenticación de administradores con control de permisos.

RNF20: El proyecto debe desarrollarse siguiendo buenas prácticas de ingeniería del software: control de versiones, pruebas, documentación técnica y separación por capas.

Requisitos no funcionales externos

Son aquellos impuestos por normativas legales, estándares externos o políticas públicas que el sistema debe cumplir.

RNF21: El acceso a los datos de usuario debe estar restringido y protegido contra accesos no autorizados.

RNF22: El almacenamiento de datos personales (nombre, email, localización) debe hacerse de forma segura.

RNF23: El sistema debe mantener un throughput mínimo de 8 requests por segundo bajo carga normal.

RNF24: Los endpoints de la API deben responder en menos de 100ms bajo condiciones normales de uso.

3.2. Especificaciones

Una vez definidos los requisitos funcionales del sistema, es posible descomponerlos en funcionalidades concretas que formarán parte de la aplicación y su panel de gestión. A continuación se detalla el conjunto de acciones que los usuarios (tanto visitantes como administradores) podrán realizar.

Funcionalidades relacionadas con la geolocalización y el mapa

F1. Visualizar un mapa interactivo con los puntos de interés distribuidos por Valencia.

F2. Detectar en tiempo real la ubicación actual del usuario.

F3. Calcular y mostrar la distancia a cada punto desde la ubicación actual del usuario.

F4. Notificar al usuario cuando se acerque a un punto no visitado.

F5. Mostrar puntos cercanos en función de la ubicación.

Funcionalidades educativas y de contenido

F6. Acceder a la ficha de una mujer histórica al llegar a un punto.

F7. Mostrar contenido multimedia asociado: texto, imágenes, audios o vídeos.

F8. Consultar un listado general de todas las mujeres incluidas en la app.

F9. Visualizar imágenes ampliadas en modal al hacer tap sobre ellas.

F10. Compartir información de lugares vía aplicaciones nativas del dispositivo.

Funcionalidades de usuario

F11. Registrar un nuevo usuario mediante nombre de usuario y contraseña.

- F12. Iniciar sesión en la aplicación.
- F13. Mantener la sesión activa entre usos.
- F14. Permitir al usuario editar su nombre de usuario.
- F15. Cerrar sesión manualmente.
- F16. Filtrar lugares por área de investigación y estado de visita.
- F17. Buscar mujeres y lugares por nombre en tiempo real.
- F18. Sincronizar estado de filtros entre diferentes pantallas de la aplicación.
- F19. Consultar el historial de puntos visitados, con fecha y hora.
- F20. Consultar el historial de rutas completadas, con fecha y hora.

Funcionalidades del panel de administración

- F21. Iniciar sesión como administrador desde el backend web.
- F22. Añadir nuevas figuras históricas con nombre, descripción y material multimedia.
- F23. Asociar cada mujer con un punto geolocalizado.
- F24. Cargar imágenes, audios, vídeos y modelos 3D desde el panel.
- F25. Editar o eliminar puntos o figuras ya creadas.
- F26. Ver la lista de usuarios registrados.
- F27. Eliminar usuarios si fuese necesario.

Funcionalidades generales

- F28. Adaptar la interfaz a distintos tamaños de pantalla.
- F29. Navegar por la app de forma intuitiva y con accesibilidad mobile-first.
- F30. Garantizar el uso fluido y sin errores tanto en Android como en iOS.

Funcionalidades relacionadas con rutas y QRs

- F31. Ver una lista de rutas temáticas con sus nombres y descripciones.
- F32. Consultar los puntos que forman parte de cada ruta.
- F33. Escanear códigos QR para desbloquear puntos de una ruta.
- F34. Reiniciar el progreso de rutas ya completadas para repetir la experiencia.

Funcionalidades de realidad aumentada (RA)

- F35. Ver una lista de los puntos con contenido RA disponible.
- F36. Mostrar únicamente los puntos con RA que estén a una distancia determinada del usuario.

F37. Al seleccionar uno de estos puntos, abrir automáticamente la cámara y cargar el modelo RA asociado.

3.3. Ciclo de vida del software

Para el desarrollo del proyecto **DONA'm MÓN**, se ha adoptado el **modelo de desarrollo en cascada**, una metodología secuencial en la que cada fase del ciclo de vida del software debe completarse completamente antes de pasar a la siguiente. Este enfoque estructurado es especialmente útil en proyectos en los que los requisitos están claramente definidos desde el inicio, como es el caso de esta aplicación, cuya funcionalidad ha sido minuciosamente detallada y delimitada en la fase de análisis.

3.3.1. Ventajas del modelo en cascada

- **Claridad y estructura:** permite dividir el proyecto en fases bien diferenciadas, con objetivos y entregables específicos en cada una.
- **Planificación precisa:** facilita la estimación de tiempos y recursos al establecer un flujo lineal de trabajo sin iteraciones constantes.
- **Documentación completa:** promueve una elaboración detallada de documentación desde las primeras etapas del proyecto.
- **Control de calidad:** permite realizar revisiones sistemáticas al finalizar cada fase, asegurando la solidez del sistema antes de avanzar.

3.3.2. Justificación de su elección

El modelo en cascada ha resultado adecuado para este proyecto debido a la existencia de una base de requisitos bien definidos y una visión clara de las funcionalidades esperadas desde el principio. La aplicación combina tecnologías como geolocalización, escaneo de códigos QR y realidad aumentada, pero todas estas funcionalidades están organizadas en módulos claramente separados, lo que ha permitido planificar su desarrollo de forma lineal. Además, el hecho de que el proyecto haya sido desarrollado por una sola persona favorece un enfoque ordenado y controlado, minimizando el riesgo de desviaciones.

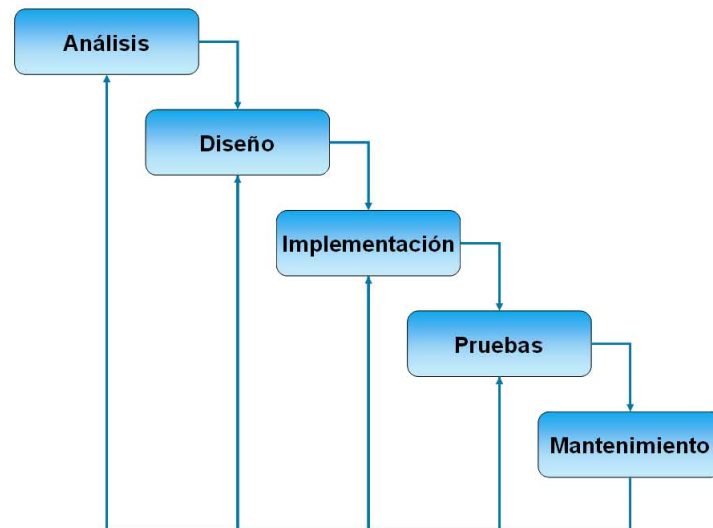


Figura 3.1: Representación del modelo de desarrollo en cascada.

3.4. Planificación temporal

3.4.1. Descomposición de tareas por fases

La planificación se ha dividido en siete bloques principales: **Inicio**, **Estado del arte**, **Análisis**, **Diseño**, **Implementación**, **Pruebas** y **Documentación**, en concordancia con el modelo incremental utilizado.

1. Inicio

- Estudio del problema que resuelve la app
- Definición de objetivos del TFG
- Planificación inicial del proyecto

2. Estado del arte

- Revisión de apps similares educativas, con RA y mapas
- Estudio de mujeres y sus lugares en Valencia
- Estudio de frameworks de RA, mapas y escáneres QR
- Selección de tecnologías definitivas (React Native, Django, AFrame)

3. Análisis

- Definición detallada de requisitos funcionales y no funcionales
- Modelado inicial del sistema: navegación y casos de uso
- Modelado entidad-relación de la base de datos

- Definición de métricas y rutas

4. Diseño

- Diseño de la arquitectura frontend y backend
- Diseño de interfaz móvil y estilo visual
- Diseño de pantallas clave: login, listado, mapa, RA, etc.

5. Implementación backend

- Creación del proyecto Django y configuración Docker
- Creación de modelos de datos y migraciones
- Implementación de API REST para usuarios, lugares y rutas
- Sistema de login y sesiones
- Registro de visitas, logros y progreso

6. Implementación frontend

- Configuración base y navegación
- Registro e inicio de sesión de usuarios
- Implementación del listado y filtros por ámbito y visitas
- Implementación del historial de usuario
- Implementación de mapa con geolocalización
- Escaneo de QR y navegación a detalle
- Sincronización de datos con backend

7. Implementación de realidad aumentada

- Configuración de AR.js y marcador personalizado
- Activación de RA según la ubicación (menos de 1km)
- Visualización de modelo 3D y contenido RA educativo

8. Pruebas y validación

- Pruebas funcionales de cada módulo
- Pruebas completas con usuarios externos
- Corrección de errores y mejoras detectadas

9. Documentación

- Redacción progresiva de la memoria (paralela a todo el desarrollo)
- Revisión completa, maquetación y anexos finales

3.4.2. Estimación temporal

La técnica de estimación utilizada es la media beta de probabilidad. Para cada tarea se ha asignado un tiempo optimista (TO), pesimista (TP) y más probable (TM), y se ha calculado la duración estimada (TE) con la siguiente fórmula:

$$TE = \frac{TO+4TM+TP}{6}$$

Cuadro 3.1: Estimación por tres valores de las tareas del proyecto

Tarea	TO	TP	TM	TE
Estudio del problema	1	3	2	2.00
Definición de objetivos	1	2	1	1.17
Planificación inicial	1	2	1	1.17
Revisión apps similares	2	4	3	3.00
Estudio lugares mujeres	1	3	2	2.00
Frameworks RA y QR	2	4	3	3.00
Selección tecnologías	1	2	1	1.17
Requisitos funcionales	2	4	3	3.00
Casos de uso y navegación	1	3	2	2.00
Modelo E-R BD	1	3	2	2.00
Def. métricas y rutas	1	2	1	1.17
Diseño arquitectura	2	4	3	3.00
Diseño interfaz visual	2	5	3	3.17
Pantallas clave	2	4	3	3.00
Django y Docker	1	3	2	2.00
Modelos y migraciones	1	3	2	2.00
API usuarios y lugares	3	5	4	4.00
Login y sesiones	1	3	2	2.00
Registro visitas y logros	1	3	2	2.00
Conf. navegación base	1	3	2	2.00
Registro e inicio sesión	1	3	2	2.00
Listado y filtros	2	4	3	3.00
Historial de usuario	1	3	2	2.00
Mapa geolocalizado	2	4	3	3.00
Escaneo QR	1	3	2	2.00
Sincronización datos	1	3	2	2.00
Config. AR.js y marcador	1	3	2	2.00
Desbloqueo por distancia	1	3	2	2.00
Visualización RA	2	4	3	3.00
Pruebas funcionales	2	4	3	3.00
Pruebas con usuarios	1	3	2	2.00
Corrección errores	1	3	2	2.00
Redacción progresiva	4	6	5	5.00
Revisión y entrega final	2	4	3	3.00
Total estimado	73	127	96	100.00 días

3.4.3. Diagrama de Gantt

Para representar la planificación temporal del proyecto de forma visual, se ha elaborado un diagrama de Gantt (véase la Figura 3.2) utilizando Microsoft Project, teniendo en cuenta las dependencias y la posibilidad de solapar tareas como la documentación progresiva.

A pesar de que la estimación por tres valores del proyecto arroja una duración total de aproximadamente **100 días** (equivalente a **800 horas**), el desarrollo real se ha planificado a lo largo de **80 días laborables** (como puede observarse en el diagrama de Gantt), lo que supone un total de **640 horas efectivas** de trabajo, distribuidas en jornadas de **8 horas al día**.

Cabe destacar que:

- El proyecto se ha realizado en paralelo con el *Máster en Tecnologías Web, Computación en la Nube y Aplicaciones Móviles*.
- Además, durante este periodo se estaban llevando a cabo *prácticas extracurriculares en el Instituto de Robótica y Tecnologías de la Información y la Comunicación (IRTIC)*, donde también se ha avanzado el desarrollo del TFG.
- No se ha trabajado durante los fines de semana, por lo que los 80 días corresponden exclusivamente a días laborables.

La fecha de inicio del proyecto fue el **3 de marzo de 2025**, y la fecha de finalización fue el **20 de junio de 2025**.

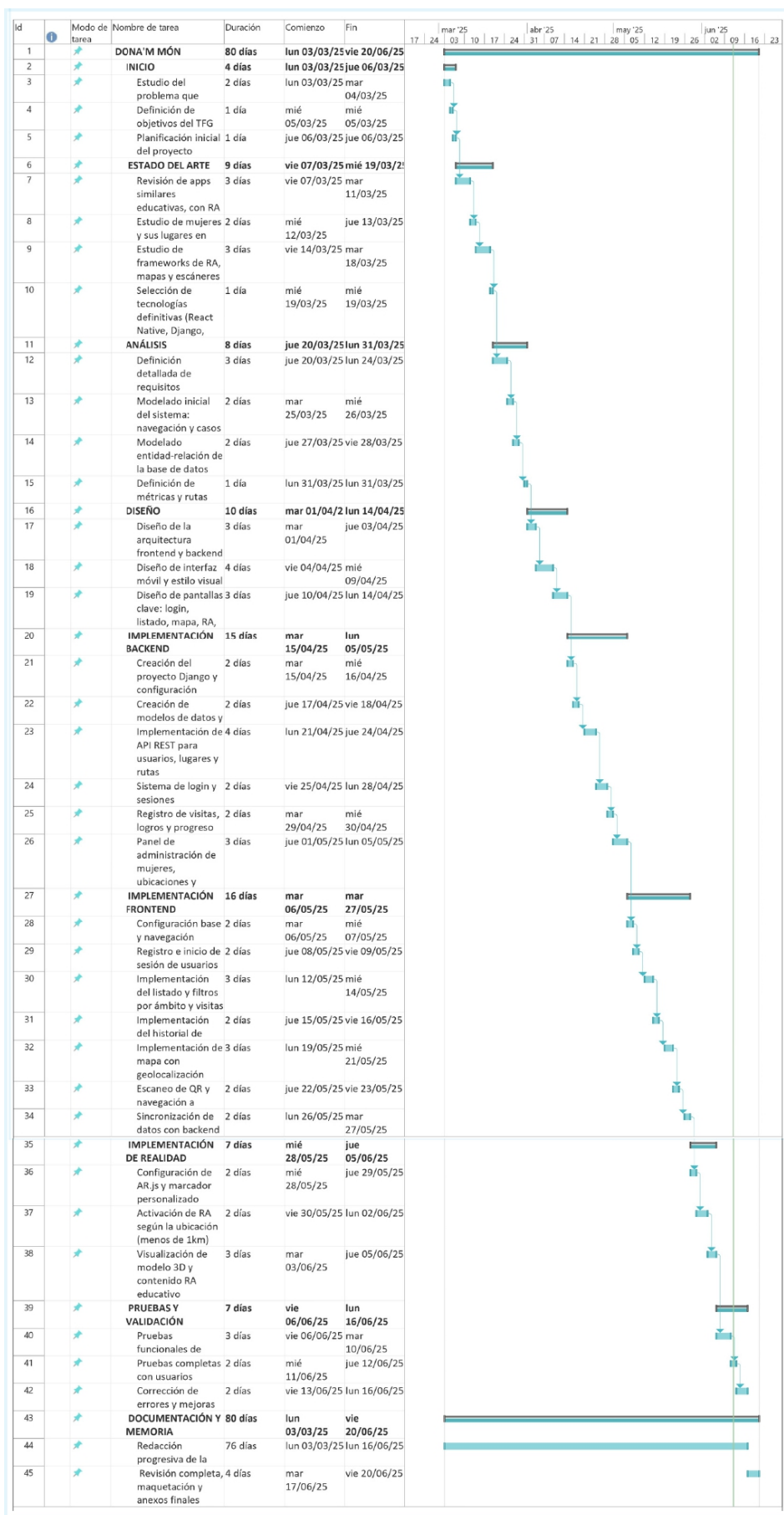


Figura 3.2: Diagrama de Gantt detallado del proyecto.

3.5. Estimación de costes

La estimación económica del proyecto se ha realizado distinguiendo entre:

- **Costes directos de personal:** incluyen el salario bruto anual para cada perfil profesional, junto con los costes de Seguridad Social correspondientes (aproximadamente un 30 % adicionales). Se han considerado perfiles distintos (Jefe de Proyecto, Desarrollador Backend, Desarrollador Frontend, Desarrollador de Realidad Tester/-QA y Modelador 3D), aunque el proyecto sea llevado a cabo por una sola persona.
- **Costes directos de material:** contemplan licencias, equipos y amortización del hardware y software utilizado.
- **Costes indirectos:** coeficiente adicional del 20 % aplicado sobre el total de los costes directos, para cubrir gastos generales asociados (electricidad, internet, etc.).

Este desglose permite dimensionar correctamente los recursos implicados y simular una estructura empresarial.

3.5.1. Costes directos de personal

Se han obtenido los sueldos medios anuales en España para cada perfil profesional a partir del portal *Glassdoor* [59, 60, 61, 62, 63, 64, 65]. A cada salario bruto se le añade un 30 % adicional correspondiente a cotizaciones sociales: contingencias comunes, desempleo, formación y Fogasa.

Se asume una jornada laboral de 8 h/día, 20 días laborables por mes y 11 meses de trabajo al año, siendo 220 días laborales al año. La planificación se basa en 80 días efectivos de trabajo entre el **3 de marzo y el 20 de junio de 2025**, distribuidos entre los distintos perfiles según su implicación en cada fase.

Cuadro 3.2: Perfiles y costes de personal (según cotización SS 2025)

Perfil	Bruto/año (€)	+31.4 % SS (€)	Total (€)	€/día	€/h
Jefe de Proyecto	40 500	12 717.00	53 217.00	270.92	33.87
Desarrollador Frontend	40 500	12 717.00	53 217.00	270.92	33.87
Desarrollador Backend	30 000	9 420.00	39 420.00	200.10	25.01
Desarrollador RA	23 000	7 222.00	30 222.00	153.68	19.21
Tester / QA	23 500	7 379.00	30 879.00	156.71	19.59
Modelador 3D	12 000	3 768.00	15 768.00	79.84	9.98
Diseñador gráfico	26 000	8 164.00	34 164.00	173.32	21.67

3.5.2. Asignación temporal por perfil

Se ha asignado a cada perfil el número estimado de jornadas necesarias para cubrir las tareas planificadas según el cronograma de trabajo. A continuación, se presenta la estimación de coste directo de personal en función de esas jornadas.

Cuadro 3.3: Asignación de tareas y costes por perfil

Perfil	Días	€/día	Tareas	Coste (€)
Jefe de Proyecto	50	270.92	Estudio inicial, definición de objetivos, planificación, selección de tecnologías, coordinación general	13,546.00
Desarrollador Frontend	16	270.92	Diseño arquitectura frontend, desarrollo UI, navegación, QR, filtros, mapa	4,334.72
Desarrollador Backend	15	200.10	Creación API Django, modelo de datos, autenticación, sincronización backend	3,001.50
Desarrollador RA	7	153.68	Integración AR.js, activación por proximidad, carga de modelos RA	1,075.76
Tester / QA	7	156.71	Pruebas funcionales, pruebas con usuarios, validación y corrección de errores	1,096.97
Modelador 3D	15	79.84	Diseño y preparación de modelos 3D para RA	1,197.60
Diseñador gráfico	20	173.32	Diseño interfaz, estilo visual, iconos y logotipos	3,466.40

3.5.3. Costes directos de material

Los costes directos de material comprenden los recursos físicos y licencias digitales utilizados durante el desarrollo del proyecto. Para estimar su impacto económico real, se ha calculado la amortización proporcional al tiempo de uso, aplicando la fórmula:

$$\text{Amortización} = \frac{\text{Precio unidad} \times \text{Días de uso}}{\text{Días de vida útil}}$$

Cuadro 3.4: Elementos utilizados y coste amortizado durante el desarrollo (80 días)

Elemento	Precio (€)	Vida útil	Uso (días)	Coste amortizado (€)
Portátil Asus ROG Strix G15	999.00	5 años (1825 días)	80	43.79
iPhone 11	749.00	4 años (1460 días)	80	41.03
Ratón Logitech G203	20.99	5 años (1825 días)	80	0.92
Visual Paradigm (mensual)	30.00	30 días	30	30.00
Microsoft 365 Business Standard	12.50	30 días	30	12.50
Visual Studio Code (profesional)	45.00	365 días	80	9.86
Docker Pro	99.00	365 días	80	21.70
GitHub Copilot	100.00	365 días	80	21.92
Overleaf Premium	140.00	365 días	80	30.68
Total amortizado				212.40 €

Cabe destacar que varias de estas herramientas (como Visual Studio Code, Docker o GitHub) disponen de versiones gratuitas o de acceso académico, pero se ha optado por reflejar el coste real de uso profesional para una estimación más realista.

3.5.4. Costes indirectos

Los costes indirectos se han estimado aplicando un coeficiente adicional del 20 % sobre el total de los costes directos (personal y material). Este porcentaje se considera representativo para cubrir gastos generales como el consumo eléctrico, conexión a internet, mantenimiento del equipo, uso de espacios compartidos y otros recursos no imputables directamente al desarrollo del proyecto, pero imprescindibles para su ejecución.

$$\text{Costes indirectos} = (\text{Costes directos de personal} + \text{Costes directos de material}) \times 0,20$$

$$\text{Costes indirectos} = (27\,718,95 + 212,40) \times 0,20 = 5\,586,27$$

3.5.5. Coste total del proyecto

A partir de la suma de los distintos componentes estimados (personal, material e indirectos), se obtiene el coste total simulado del desarrollo del proyecto en un entorno profesional.

Cuadro 3.5: Resumen de costes del proyecto

Concepto	Coste (€)
Costes directos de personal	27 718.95
Costes directos de material	212.40
Costes indirectos (20 %)	5 586.27
Coste total estimado	33 517.62

Esta estimación permite cuantificar de forma realista los recursos necesarios para ejecutar un proyecto de características similares en un entorno empresarial. Aunque el trabajo haya sido desarrollado por una única persona, se ha optado por realizar una simulación profesional con perfiles especializados para reflejar adecuadamente la complejidad técnica y el valor añadido del producto final.

3.6. Viabilidad del proyecto

3.6.1. Viabilidad económica

El análisis de costes realizado en la sección anterior ha permitido cuantificar los recursos necesarios para el desarrollo de la aplicación *DONA'm MÓN*. El coste total estimado, que incluye los costes directos de personal (27.718,95 €), costes directos de material (212,40 €) y un 20 % adicional en concepto de costes indirectos (5.586,27 €), asciende a **33.517,62 €**.

Dado que el proyecto se ha desarrollado sin fines comerciales, el desarrollo ha sido ejecutado por una única persona, utilizando licencias gratuitas, versiones académicas o software de código abierto. Por tanto, los costes no se han asumido en su totalidad. No obstante, se ha realizado una estimación económica completa con el objetivo de evaluar

su viabilidad en un contexto profesional o empresarial real, permitiendo así dimensionar correctamente los recursos necesarios en caso de una futura implementación a escala.

El presupuesto puede considerarse razonable si se contempla una posible financiación pública (subvenciones a proyectos culturales, feministas o tecnológicos), financiación universitaria, patrocinio institucional (ayuntamientos, universidades o fundaciones), o apoyo por parte de entidades interesadas en promover la visibilización de mujeres en entornos urbanos y educativos. Además, el impacto social y educativo del proyecto puede justificar su financiación mediante convocatorias de innovación social o género.

3.6.2. Viabilidad legal

Desde el punto de vista legal, el desarrollo de esta aplicación requiere considerar varias cuestiones relevantes:

- **Protección de datos personales (RGPD):** la aplicación permite el registro de usuarios y almacena información relacionada con su ubicación geográfica y su historial de visitas. Por ello, se deberá garantizar el cumplimiento del Reglamento General de Protección de Datos (RGPD), asegurando la obtención del consentimiento explícito del usuario, el almacenamiento seguro de datos y la posibilidad de acceder, rectificar o eliminar la información personal.
- **Política de privacidad y condiciones de uso:** será necesario incluir una política clara y accesible que informe al usuario sobre el tratamiento de sus datos, el uso de la geolocalización y la finalidad educativa de la aplicación.
- **Derechos de autor y licencias:** todos los modelos 3D, imágenes, textos y recursos utilizados deben contar con licencia libre (Creative Commons, dominio público) o haber sido creados expresamente para el proyecto. En caso contrario, sería necesario solicitar los permisos adecuados.
- **Política de uso de servicios externos:** la aplicación hace uso de tecnologías como *WebView* para integrar experiencias de realidad aumentada desarrolladas con A-Frame y AR.js, por lo que se han respetado sus licencias de uso (ambas de código abierto bajo licencia MIT), garantizando la legalidad del uso de estas herramientas dentro del proyecto.

No se han identificado restricciones legales insalvables que impidan la realización o publicación de la aplicación. En todo caso, se recomienda aplicar buenas prácticas en el desarrollo seguro, accesible y ético del software.

3.7. Análisis de riesgos

Durante el desarrollo del proyecto *DONA'm MÓN* se han identificado diversos riesgos que pueden afectar negativamente a su cumplimiento en tiempo, coste o calidad. Para su evaluación se ha utilizado un enfoque cualitativo que combina la estimación de la probabilidad de ocurrencia y el impacto en el proyecto, permitiendo establecer niveles de riesgo y su clasificación prioritaria.

3.7.1. Principales riesgos identificados

En la Tabla 3.6 se recogen los principales riesgos detectados, su probabilidad estimada, el impacto medido en días de retraso potencial, su nivel de riesgo ($NR = \text{Probabilidad} \times \text{Impacto}$), la clasificación resultante y la fase del proyecto en la que podrían manifestarse.

Cuadro 3.6: Principales riesgos identificados

Riesgo	Prob. (%)	Impacto (días)	NR	Clasificación	Fase
Retrasos en la integración de la RA con React Native	40	8	3.20	Inaceptable	Ejecución
Cambios en dependencias externas (Mapas, RA, etc.)	30	5	1.50	Alto	Ejecución
Falta de experiencia previa con tecnologías clave	35	6	2.10	Alto	Planificación/Ejecución
Definición ambigua del alcance inicial	25	7	1.75	Alto	Inicio
Estimaciones optimistas en tareas críticas	20	6	1.20	Moderado	Planificación
Problemas legales (datos, localización, RGPD)	15	5	0.75	Moderado	Inicio
Baja disponibilidad de recursos y tiempo personal	30	6	1.80	Alto	Ejecución
Errores funcionales en fases finales	10	5	0.50	Bajo	Cierre

A continuación se describen los riesgos con mayor nivel de prioridad:

1. **Definición ambigua del alcance del proyecto:** Un alcance mal delimitado puede derivar en objetivos poco claros y decisiones incorrectas en fases posteriores del desarrollo.

Probabilidad: Media — Impacto: Alto — Nivel de riesgo: Inaceptable

2. **Estimaciones temporales y de costes imprecisas:** Una planificación optimista puede provocar desviaciones significativas respecto al cronograma o sobrecostes imprevistos.

Probabilidad: Alta — Impacto: Alto — Nivel de riesgo: Inaceptable

3. **Falta de experiencia con tecnologías específicas:** La falta de familiaridad con herramientas clave como AR.js o A-Frame podría ralentizar la implementación de la funcionalidad de realidad aumentada.

Probabilidad: Media — Impacto: Alto — Nivel de riesgo: Inaceptable

4. **Retrasos en la ejecución de tareas clave:** La elevada carga de trabajo asumida por una sola persona incrementa la probabilidad de bloqueos o demoras.

Probabilidad: Alta — Impacto: Medio — Nivel de riesgo: Alto

5. **Dependencia de recursos externos:** El uso de librerías de terceros (como AR.js, Mapbox, A-Frame) puede verse afectado por cambios inesperados o falta de mantenimiento.

Probabilidad: Media — Impacto: Medio — Nivel de riesgo: Medio

6. **Problemas de compatibilidad entre componentes:** La integración entre tecnologías heterogéneas (React Native, Django, AR.js) podría causar errores o conflictos difíciles de prever.

Probabilidad: Baja — Impacto: Alto — Nivel de riesgo: Alto

7. **Riesgos legales:** A pesar de que no se almacena información sensible, el tratamiento de datos de geolocalización y el uso de usuarios registrados exige el cumplimiento de la normativa vigente en materia de protección de datos (RGPD).

Probabilidad: Baja — Impacto: Medio — Nivel de riesgo: Medio

3.7.2. Matriz de riesgos

La figura 3.3 muestra la matriz de evaluación de riesgos utilizada, que relaciona la probabilidad de ocurrencia con la magnitud del impacto. Esta herramienta facilita la clasificación de los riesgos y la priorización de acciones preventivas.

		IMPACTO EN EL PROYECTO		
		BAJO	MEDIO	ALTO
PROBABILIDAD DE QUE OCURRA (%)	ALTA [70 - 100]	RIESGO MEDIO	RIESGO ALTO	RIESGO INACEPTABLE
	MEDIA [35 - 70]	RIESGO BAJO	RIESGO ALTO	RIESGO INACEPTABLE
	BAJA [10 - 35]	RIESGO BAJO	RIESGO MEDIO	RIESGO ALTO

Figura 3.3: Matriz de priorización de riesgos utilizada

3.7.3. Resumen de evaluación de riesgos

Cuadro 3.7: Resumen cualitativo de evaluación de riesgos

Riesgo	Probabilidad	Impacto	Clasificación
Definición ambigua del alcance	Media	Alto	Inaceptable
Estimaciones temporales imprecisas	Alta	Alto	Inaceptable
Falta de experiencia con RA	Media	Alto	Inaceptable
Retrasos en tareas clave	Alta	Medio	Alto
Dependencia de recursos externos	Media	Medio	Medio
Problemas de compatibilidad tecnológica	Baja	Alto	Alto
Riesgos legales (RGPD)	Baja	Medio	Medio

3.7.4. Estrategias de mitigación

Para minimizar el impacto de los riesgos identificados en el proyecto, se han definido una serie de medidas preventivas orientadas a reducir su probabilidad de ocurrencia o su impacto en caso de materializarse. Estas estrategias se alinean con las buenas prácticas en la gestión de proyectos tecnológicos y tienen carácter proactivo.

- **Definición ambigua del alcance del proyecto:** Se ha trabajado desde el inicio con una delimitación clara del alcance y los objetivos, apoyada por una estructura modular de tareas y entregables. La revisión periódica del cronograma y del backlog funcional permite detectar desviaciones tempranas y corregir posibles ambigüedades.
- **Estimaciones temporales y de costes imprecisas:** La planificación se ha fundamentado en la técnica de estimación por tres valores (optimista, más probable y pesimista), reduciendo la subjetividad y permitiendo incorporar márgenes de seguridad en tareas críticas. Además, se han utilizado referencias de proyectos similares como guía de validación.
- **Falta de experiencia con tecnologías específicas:** Durante la fase de análisis técnico se han realizado pruebas de concepto con las tecnologías clave (AR.js, A-Frame, integración con React Native) para identificar dificultades de forma anticipada. Asimismo, se ha recurrido a documentación oficial, foros especializados y comunidades activas.
- **Retrasos en la ejecución de tareas clave:** Dado que el proyecto es desarrollado por una única persona, se han asignado holguras temporales en tareas críticas y puntos de control semanales para verificar el avance real. El uso de herramientas como Microsoft Project ha permitido detectar desviaciones y replanificar en tiempo real.
- **Dependencia de recursos externos:** Se ha priorizado el uso de herramientas open source con comunidades activas y mantenidas (por ejemplo, AR.js, OpenStreetMap). Además, se han definido posibles alternativas ante fallos o incompatibilidades en recursos externos.
- **Problemas de compatibilidad entre componentes:** La arquitectura del sistema se ha diseñado de forma desacoplada, favoreciendo la integración por medio de interfaces RESTful y estándares abiertos. Esto reduce la posibilidad de conflictos entre tecnologías heterogéneas.
- **Riesgos legales (RGPD):** Aunque no se almacenan datos sensibles, se garantiza el cumplimiento normativo mediante el uso de comunicaciones seguras (HTTPS), el consentimiento del usuario para acceder a la ubicación y la ausencia de almacenamiento persistente de información personal.

3.7.5. Planes de contingencia

Se han previsto planes de contingencia específicos para los riesgos más relevantes. Por ejemplo, si se detectan incompatibilidades graves entre tecnologías, se contemplaría el uso de una solución alternativa basada únicamente en mapas interactivos sin RA, manteniendo la funcionalidad básica de exploración de contenidos educativos geolocalizados.

Capítulo 4

Análisis

4.1. Introducción

En este capítulo se realiza el análisis del sistema desde el punto de vista de la Ingeniería del Software. El objetivo es especificar, de manera estructurada y rigurosa, cómo debe comportarse la aplicación y qué funcionalidades debe ofrecer a los distintos tipos de usuarios, siguiendo las metodologías y herramientas propias de la disciplina.

El análisis comienza con la identificación de los roles principales del sistema y las tareas asociadas a cada uno. En el caso de esta aplicación, los roles contemplados son: usuario registrado, usuario no registrado y administrador. Cada uno de ellos dispone de diferentes permisos y accesos a funcionalidades, siendo el administrador quien puede gestionar todos los contenidos y usuarios, mientras que los usuarios registrados acceden a la experiencia completa de la app.

A continuación, se presentan los diagramas de casos de uso, que permiten visualizar de forma global y por rol las interacciones posibles con el sistema. Posteriormente, se detallan los casos de uso más relevantes mediante tablas descriptivas, especificando actores, propósito, precondiciones, postcondiciones y flujos de eventos. Según la naturaleza de la aplicación, también se incluyen diagramas de actividad y de estados para ilustrar los procesos dinámicos y los posibles cambios de estado de los objetos principales.

4.2. Diagramas de casos de uso

A continuación se presentan los diagramas de casos de uso para los diferentes roles del sistema:

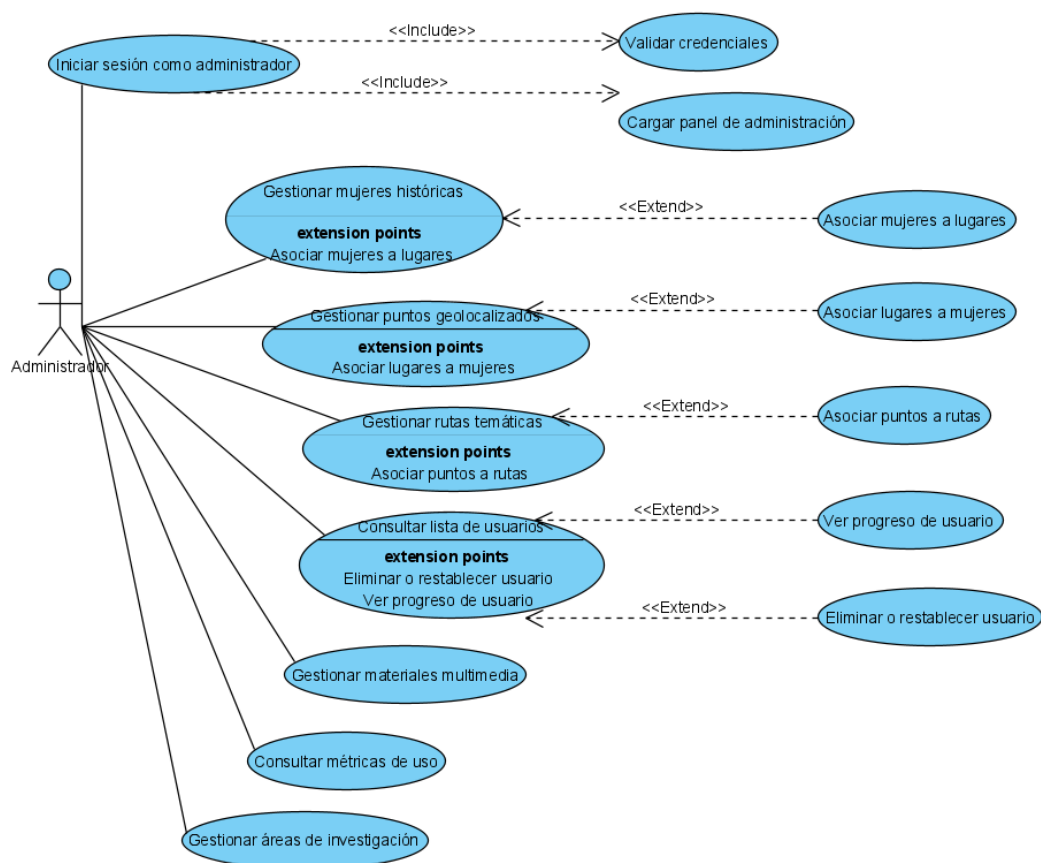


Figura 4.1: Diagrama de casos de uso: Administrador

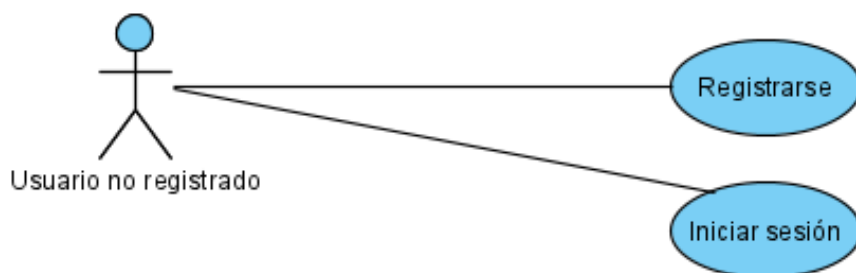


Figura 4.2: Diagrama de casos de uso: Usuario no registrado

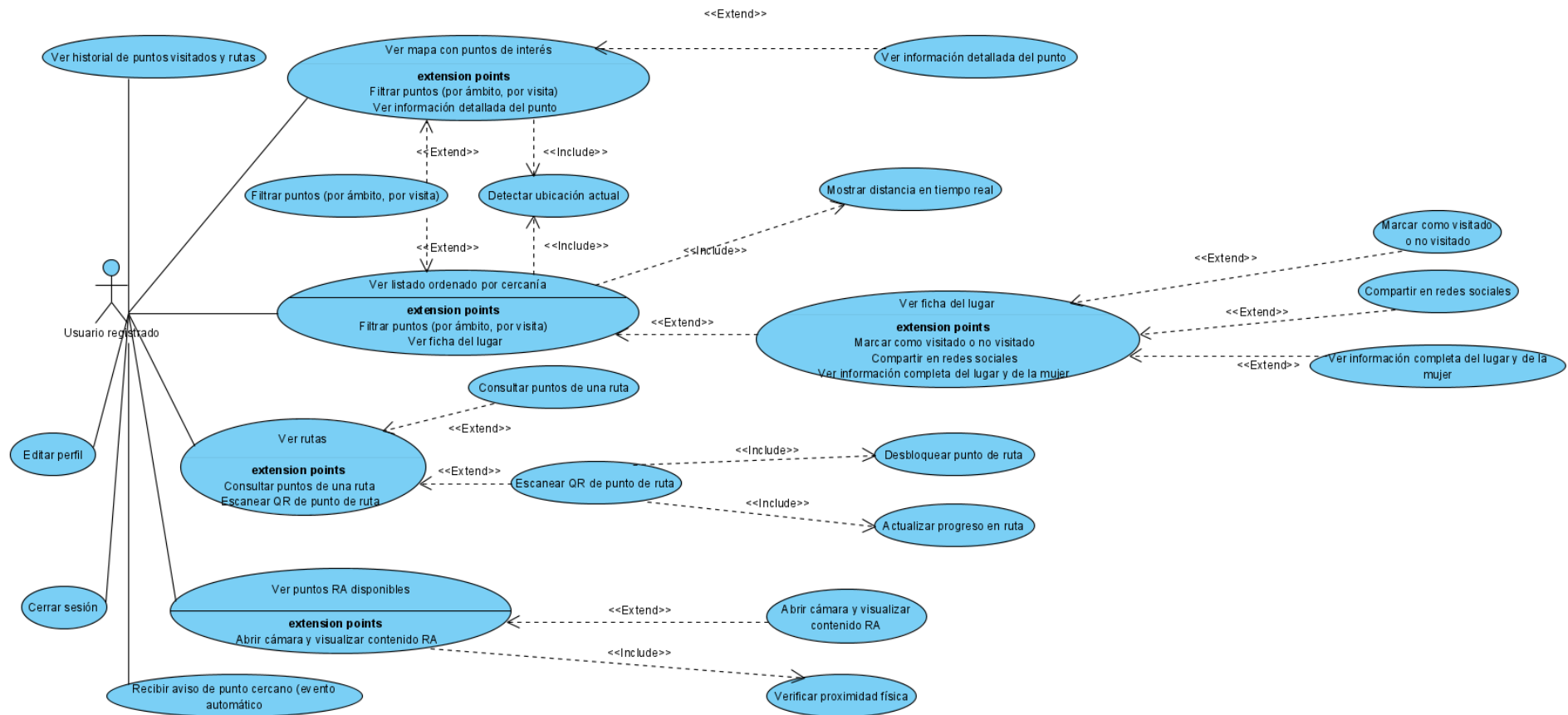


Figura 4.3: Diagrama de casos de uso: Usuario registrado

4.3. Especificación de casos de uso

A continuación se detallan los casos de uso principales de la aplicación:

4.3.1. Caso de uso 1: Login de usuario

Cuadro 4.1: Caso de uso 1 - Login de usuario

Nombre del caso de uso	Login de usuario
Actores	Usuario no registrado (principal), Sistema (secundario)
Propósito	Permitir que el usuario acceda a su cuenta personal mediante nombre de usuario y contraseña.
Resumen	Este caso de uso describe el proceso de inicio de sesión mediante credenciales para acceder a las funcionalidades personalizadas de la app.
Tipo	Primario
Referencias	RF23, RF24
Precondiciones	El usuario debe tener una cuenta creada previamente. El sistema debe estar operativo y conectado a internet.
Postcondiciones	El usuario inicia sesión correctamente y accede a la pantalla principal de la aplicación, manteniendo la sesión iniciada hasta que decida cerrarla.
Flujo de eventos principal	
Actor	Sistema
1. El usuario abre la aplicación DONA'm MÓN.	
	2. El sistema muestra la pantalla de login con los campos de email y contraseña.
3. El usuario introduce su email y contraseña.	
	4. El sistema valida las credenciales en la base de datos.
	5. Si las credenciales son válidas, se genera un token y se accede a la pantalla principal.
	6. Si las credenciales no son válidas, se muestra un mensaje de error y se vuelve al paso 2.

4.3.2. Caso de uso 2: Registro de usuario

Cuadro 4.2: Caso de uso 2 - Registro de usuario

Nombre del caso de uso	Registro de usuario
Actores	Usuario no registrado (principal), Sistema (secundario)
Propósito	Permitir que nuevos usuarios creen una cuenta personal mediante un formulario con nombre de usuario, correo electrónico, fecha de nacimiento y contraseña.
Resumen	Este caso de uso describe el proceso por el cual un usuario puede registrarse en la aplicación.
Tipo	Primario
Referencias	RF24, RNF21, RNF22
Precondiciones	El sistema debe estar operativo y conectado. El usuario no debe tener una cuenta registrada con ese correo o nombre de usuario.
Postcondiciones	El usuario queda registrado y accede automáticamente a la pantalla principal con sesión iniciada.
Flujo de eventos principal	
Actor	Sistema
1. El usuario pulsa sobre “Registrarse” en la pantalla de login.	
	2. El sistema muestra el formulario de registro con los campos de correo, nombre de usuario, fecha de nacimiento y contraseña.
3. El usuario introduce sus datos.	
	4. El sistema verifica que el correo y nombre de usuario no exista previamente en la base de datos.
	5. El sistema guarda los datos cifrados y crea la nueva cuenta.
	6. El sistema genera una sesión automática y redirige a la pantalla principal.
	7. Si el correo ya existe o hay errores, se muestra un mensaje al usuario.

4.3.3. Caso de uso 3: Ver mapa con puntos de interés

Cuadro 4.3: Caso de uso 3 - Ver mapa con puntos de interés

Nombre del caso de uso	Ver mapa con puntos de interés
Actores	Usuario registrado (principal), Sistema (secundario)
Propósito	Permitir al usuario visualizar un mapa interactivo con los puntos geolocalizados disponibles en la ciudad de Valencia.
Resumen	Este caso de uso describe la carga del mapa, la detección de la ubicación actual, la visualización de los puntos y la posibilidad de aplicar filtros o acceder a la ficha de un punto.
Tipo	Primario
Referencias	RF1, RF2, RF3, RF7, RF8, RF31, RF32
Precondiciones	El usuario debe haber iniciado sesión y tener permisos de geolocalización habilitados.
Postcondiciones	El usuario ve los puntos en el mapa y puede filtrar, recibir notificaciones o acceder a la ficha de un punto.
Flujo de eventos principal	
Actor	Sistema
1. El usuario accede a la pestaña del mapa.	
	2. El sistema obtiene la ubicación actual mediante GPS.
	3. El sistema carga los puntos geolocalizados en el mapa.
2. El usuario activa un filtro por ámbito o estado de visita.	
	4. El sistema actualiza la visualización del mapa según los filtros.
3. El usuario pulsa sobre un punto del mapa.	
	5. El sistema abre la ficha detallada del punto seleccionado.
	6. Si el usuario se aproxima a un punto no visitado, se lanza una notificación.

4.3.4. Caso de uso 4: Ver listado de puntos ordenado por cercanía

Cuadro 4.4: Caso de uso 4 - Ver listado de puntos ordenado por cercanía

Nombre del caso de uso	Ver listado de puntos ordenado por cercanía
Actores	Usuario registrado (principal), Sistema (secundario)
Propósito	Permitir al usuario consultar todos los puntos disponibles organizados automáticamente por distancia desde su ubicación actual.
Resumen	Este caso de uso permite al usuario visualizar un listado de lugares asociados a mujeres históricas, ordenados por cercanía, con filtros por ámbito y estado, y acceso a la ficha de cada uno.
Tipo	Primario
Referencias	RF7, RF8, RF9, RF31, RF32, RF33, RF34, RF35
Precondiciones	El usuario debe tener sesión iniciada y el GPS activado.
Postcondiciones	El usuario ve una lista actualizada y filtrada de puntos, con acceso a sus fichas.
Flujo de eventos principal	
Actor	Sistema
1. El usuario accede a la pestaña de listado.	
	2. El sistema detecta la ubicación actual del usuario.
	3. El sistema obtiene todos los puntos disponibles.
	4. El sistema calcula y muestra la distancia en tiempo real a cada punto.
	5. El sistema ordena el listado por cercanía ascendente.
2. El usuario aplica un filtro por ámbito o por estado de visita.	
	6. El sistema actualiza el listado aplicando los filtros seleccionados.
3. El usuario pulsa sobre un punto del listado.	
	7. El sistema abre la ficha detallada del punto.

4.3.5. Caso de uso 5: Ver ficha del lugar

Cuadro 4.5: Caso de uso 1.5 - Ver ficha del lugar

Nombre del caso de uso	Ver ficha del lugar
Actores	Usuario registrado (principal), Sistema (secundario)
Propósito	Permitir al usuario visualizar toda la información del lugar y de la figura femenina representada, incluyendo contenido educativo multimedia.
Resumen	Este caso de uso describe el acceso a la ficha completa de un punto, con texto, imágenes, biografía de la mujer, ubicación y la posibilidad de marcar como visitado o compartir por redes sociales.
Tipo	Primario
Referencias	RF4, RF5, RF12, RF13, RF36
Precondiciones	El usuario debe haber accedido desde el mapa o listado, seleccionando un punto.
Postcondiciones	El usuario ha consultado la información y puede haber marcado el punto como visitado o compartido el contenido.
Flujo de eventos principal	
Actor	Sistema
1. El usuario pulsa sobre un punto desde el mapa o el listado.	
	2. El sistema carga los datos completos del lugar y de la mujer histórica asociada.
	3. El sistema muestra el contenido educativo (texto, imágenes y/o audio).
2. El usuario pulsa “Marcar como visitado”.	
	4. El sistema actualiza el estado del punto como visitado y lo guarda en el historial.
3. El usuario pulsa “Compartir”.	
	5. El sistema lanza el menú para compartir el contenido en redes sociales.

4.3.6. Caso de uso 6: Ver rutas y consultar puntos de una ruta

Cuadro 4.6: Caso de uso 6 - Ver rutas y consultar puntos de una ruta

Nombre del caso de uso	Ver rutas y consultar puntos de una ruta
Actores	Usuario registrado (principal), Sistema (secundario)
Propósito	Permitir al usuario visualizar las rutas temáticas disponibles y consultar qué puntos forman parte de cada una.
Resumen	Este caso de uso describe cómo el usuario accede a la lista de rutas temáticas (por ejemplo, mujeres científicas), y cómo puede consultar los puntos que debe desbloquear escaneando su QR correspondiente.
Tipo	Primario
Referencias	RF11, RF37
Precondiciones	El usuario debe haber iniciado sesión.
Postcondiciones	El usuario ha accedido a la información de una ruta y conoce qué puntos contiene.
Flujo de eventos principal	
Actor	Sistema
1. El usuario accede a la pestaña de rutas.	
	2. El sistema muestra una lista de rutas temáticas disponibles, con su nombre y descripción.
2. El usuario pulsa sobre una de las rutas.	
	3. El sistema muestra los puntos geolocalizados que componen esa ruta.
	4. El sistema indica visualmente cuáles han sido desbloqueados y cuáles no.

4.3.7. Caso de uso 7: Escanear QR de punto de ruta

Cuadro 4.7: Caso de uso 7 - Escanear QR de punto de ruta

Nombre del caso de uso	Escanear QR de punto de ruta
Actores	Usuario registrado (principal), Sistema (secundario)
Propósito	Permitir al usuario desbloquear un punto dentro de una ruta temática mediante el escaneo de un código QR.
Resumen	Este caso de uso permite que el usuario avance en una ruta únicamente si ha escaneado correctamente el QR asociado a un punto. Una vez validado, se actualiza su progreso.
Tipo	Primario
Referencias	RF30, RF38, RF39
Precondiciones	El usuario debe haber iniciado sesión y tener la cámara habilitada. Debe haber accedido previamente a la ruta.
Postcondiciones	Si el código escaneado es válido, el punto se marca como desbloqueado en la ruta y se actualiza el historial del usuario.
Flujo de eventos principal	
Actor	Sistema
1. El usuario accede a una ruta y pulsa sobre un punto no desbloqueado.	
2. El usuario pulsa el botón "Escanear QR".	
	3. El sistema abre la cámara y espera a que el usuario enfoque el código QR.
3. El usuario escanea el QR.	
	4. El sistema valida que el QR corresponde al punto correcto.
	5. Si es válido, el sistema desbloquea el punto y lo marca como visitado.
	6. El sistema actualiza el progreso del usuario en esa ruta.
	7. Si el código QR no es válido, se muestra un mensaje de error.

4.3.8. Caso de uso 8: Ver puntos RA disponibles

Cuadro 4.8: Caso de uso 8 - Ver puntos RA disponibles

Nombre del caso de uso	Ver puntos RA disponibles
Actores	Usuario registrado (principal), Sistema (secundario)
Propósito	Mostrar al usuario solo los puntos que tienen contenido de realidad aumentada disponible y que están dentro del radio permitido.
Resumen	Este caso de uso describe cómo el sistema filtra automáticamente los puntos que disponen de RA y están dentro del rango de distancia del usuario para que puedan visualizarse.
Tipo	Primario
Referencias	RF40, RF41
Precondiciones	El usuario debe tener sesión iniciada, el GPS activado y conexión a internet.
Postcondiciones	El usuario visualiza un listado de puntos RA disponibles que puede seleccionar para abrir el contenido RA.
Flujo de eventos principal	
Actor	Sistema
1. El usuario accede a la pestaña de RA.	
	2. El sistema detecta la ubicación actual del usuario.
	3. El sistema filtra los puntos con contenido RA.
	4. El sistema calcula qué puntos RA están dentro del radio permitido.
	5. El sistema muestra el listado de puntos RA disponibles cercanos.

4.3.9. Caso de uso 9: Visualizar contenido RA

Cuadro 4.9: Caso de uso 9 - Visualizar contenido RA

Nombre del caso de uso	Visualizar contenido RA
Actores	Usuario registrado (principal), Sistema (secundario)
Propósito	Permitir al usuario visualizar los modelos 3D o elementos interactivos en RA sobre el entorno real desde la cámara del dispositivo.
Resumen	Este caso de uso describe el proceso de abrir la cámara desde un punto RA cercano y mostrar el contenido de realidad aumentada asociado, como un modelo 3D vinculado a una mujer histórica.
Tipo	Primario
Referencias	RF41, RF6, RF20
Precondiciones	El usuario debe haber accedido a la pestaña RA, seleccionado un punto válido y concedido permiso de cámara.
Postcondiciones	El usuario visualiza el contenido RA correctamente alineado sobre el marcador o la ubicación.
Flujo de eventos principal	
Actor	Sistema
1. El usuario pulsa sobre un punto RA disponible.	
	2. El sistema solicita permisos para acceder a la cámara.
	3. El sistema abre la cámara del dispositivo.
	4. El sistema detecta la ubicación o el marcador visual.
	5. El sistema carga el modelo RA correspondiente (imagen, animación o 3D).
	6. El sistema muestra el modelo RA en tiempo real superpuesto al entorno.

4.3.10. Caso de uso 10: Editar perfil de usuario

Cuadro 4.10: Caso de uso 10 - Editar datos del perfil

Nombre del caso de uso	Editar datos del perfil
Actores	Usuario registrado (principal), Sistema (secundario)
Propósito	Permitir al usuario modificar sus datos personales como nombre, correo o contraseña desde la sección de perfil.
Resumen	Este caso de uso describe cómo un usuario puede acceder a su perfil, modificar la información personal y guardar los cambios, los cuales se actualizan en la base de datos.
Tipo	Primario
Referencias	RF25, RF42
Precondiciones	El usuario debe tener sesión iniciada. Debe estar en la sección de perfil.
Postcondiciones	Los datos del perfil del usuario quedan actualizados correctamente en el sistema.
Flujo de eventos principal	
Actor	Sistema
1. El usuario accede a la pestaña de perfil.	
2. El usuario pulsa el botón "Editar perfil".	
3. El usuario modifica los campos deseados.	
4. El usuario pulsa el botón "Guardar cambios".	
	5. El sistema valida los nuevos datos.
	6. El sistema actualiza los datos del usuario en la base de datos.
	7. El sistema muestra un mensaje de confirmación.

4.3.11. Caso de uso 11: Ver historial de lugares visitados y rutas completadas

Cuadro 4.11: Caso de uso 11 - Ver historial de lugares visitados y rutas completadas

Nombre del caso de uso	Ver historial de lugares visitados y rutas completadas
Actores	Usuario registrado (principal), Sistema (secundario)
Propósito	Permitir al usuario consultar qué puntos ha visitado y qué rutas ha completado, con su respectiva fecha y hora.
Resumen	Este caso de uso permite al usuario ver un resumen personalizado de su progreso en la app, incluyendo puntos visitados, rutas completadas y fechas correspondientes.
Tipo	Primario
Referencias	RF14, RF42, RF13
Precondiciones	El usuario debe tener sesión iniciada.
Postcondiciones	El usuario visualiza su historial actualizado, sin modificar ningún dato.
Flujo de eventos principal	
Actor	Sistema
1. El usuario accede a la pestaña de perfil.	
2. El usuario pulsa sobre “Historial de visitas y rutas”.	
	3. El sistema recupera los datos del historial del usuario desde la base de datos.
	4. El sistema muestra la lista de puntos visitados con fecha y hora.
	5. El sistema muestra también las rutas completadas por el usuario.

4.3.12. Caso de uso 12: Cerrar sesión

Cuadro 4.12: Caso de uso 12 - Cerrar sesión

Nombre del caso de uso	Cerrar sesión
Actores	Usuario registrado (principal), Sistema (secundario)
Propósito	Permitir al usuario finalizar su sesión actual y cerrar su acceso a la app hasta el próximo inicio de sesión.
Resumen	Este caso de uso describe cómo el usuario puede cerrar manualmente su sesión desde la sección de perfil, eliminando su token de acceso y volviendo a la pantalla de login.
Tipo	Primario
Referencias	RF24, RF42
Precondiciones	El usuario debe tener sesión iniciada.
Postcondiciones	La sesión se cierra y el usuario vuelve a la pantalla de login.
Flujo de eventos principal	
Actor	Sistema
1. El usuario accede a la pestaña de perfil.	
2. El usuario pulsa el botón “Cerrar sesión”.	
	3. El sistema elimina el token de autenticación almacenado.
	4. El sistema redirige al usuario a la pantalla de login.

4.3.13. Caso de uso 13: Iniciar sesión como administrador

Cuadro 4.13: Caso de uso 13 - Iniciar sesión como administrador

Nombre del caso de uso	Iniciar sesión como administrador
Actores	Administrador (principal), Sistema (secundario)
Propósito	Permitir al administrador acceder al panel de administración web mediante autenticación segura.
Resumen	El administrador introduce sus credenciales (usuario y contraseña) en la pantalla de login del panel de administración. El sistema valida los datos y, si son correctos, permite el acceso a las funcionalidades administrativas.
Tipo	Primario
Referencias	RF16, RNF19
Precondiciones	El administrador debe estar registrado en el sistema y disponer de credenciales válidas.
Postcondiciones	El administrador accede al panel de administración con los permisos correspondientes.
Flujo de eventos principal	
Actor	Sistema
El administrador accede a la URL del panel de administración.	
El administrador introduce su usuario y contraseña.	
El administrador pulsa el botón de “Iniciar sesión”.	
	4. El sistema valida las credenciales introducidas.
	5. Si las credenciales son correctas, el sistema carga el panel de administración.
	6. Si las credenciales son incorrectas, el sistema muestra un mensaje de error.

4.3.14. Caso de uso 14: Gestionar mujeres históricas

Cuadro 4.14: Caso de uso 14 - Gestionar mujeres históricas

Nombre del caso de uso	Gestionar mujeres históricas
Actores	Administrador (principal), Sistema (secundario)
Propósito	Permitir al administrador crear, editar o eliminar figuras históricas femeninas dentro del sistema.
Resumen	Este caso de uso describe las acciones que puede realizar el administrador para mantener actualizada la base de datos de mujeres históricas: crear nuevas entradas, modificar datos existentes, o eliminarlas. Cada mujer puede estar asociada a uno o varios lugares geolocalizados.
Tipo	Primario
Referencias	RF17, RF18
Precondiciones	El administrador debe haber iniciado sesión correctamente en el panel de administración.
Postcondiciones	Se actualiza la base de datos de figuras históricas femeninas según las acciones realizadas.
Flujo de eventos principal	
Actor	Sistema
1. El administrador accede a la sección “Mujeres históricas” del panel.	
2. El administrador pulsa sobre “Añadir mujer”, “Editar” o “Eliminar”.	
3. El administrador introduce o modifica datos como nombre, biografía, fechas, y áreas de investigación.	
	4. El sistema valida los campos requeridos.
	5. El sistema guarda los datos nuevos o actualizados en la base de datos.
	6. Si se ha eliminado una figura, se borran también sus asociaciones con puntos.
	7. El sistema muestra un mensaje de confirmación de los cambios.

4.3.15. Caso de uso 15: Gestionar puntos geolocalizados

Cuadro 4.15: Caso de uso 15 - Gestionar puntos geolocalizados

Nombre del caso de uso	Gestionar puntos geolocalizados
Actores	Administrador (principal), Sistema (secundario)
Propósito	Permitir al administrador crear, editar o eliminar lugares geolocalizados y asociarlos a mujeres históricas.
Resumen	El administrador puede mantener actualizada la base de datos de lugares: crear nuevos puntos, modificar su información (nombre, descripción, ubicación, foto), o eliminarlos. Cada lugar puede estar vinculado a una mujer histórica.
Tipo	Primario
Referencias	RF17, RF18
Precondiciones	El administrador debe haber iniciado sesión correctamente en el panel de administración.
Postcondiciones	Se actualiza la base de datos de lugares y sus asociaciones con mujeres históricas.
Flujo de eventos principal	
Actor	Sistema
1. El administrador accede a la sección “Lugares” del panel.	
2. El administrador pulsa sobre “Añadir lugar”, “Editar” o “Eliminar”.	
3. El administrador introduce o modifica datos como nombre, descripción, ubicación, foto y mujer asociada.	
	4. El sistema valida los campos requeridos.
	5. El sistema guarda los datos nuevos o actualizados en la base de datos.
	6. Si se elimina un lugar, se eliminan también sus asociaciones.
	7. El sistema muestra un mensaje de confirmación de los cambios.

4.3.16. Caso de uso 16: Gestionar rutas temáticas

Cuadro 4.16: Caso de uso 16 - Gestionar rutas temáticas

Nombre del caso de uso	Gestionar rutas temáticas
Actores	Administrador (principal), Sistema (secundario)
Propósito	Permitir al administrador crear, editar o eliminar rutas temáticas y asociar puntos a cada ruta.
Resumen	El administrador puede definir rutas temáticas, asignarles nombre y descripción, y asociarles puntos geolocalizados. Puede modificar o eliminar rutas existentes.
Tipo	Primario
Referencias	RF11, RF19
Precondiciones	El administrador debe haber iniciado sesión correctamente en el panel de administración.
Postcondiciones	Se actualiza la base de datos de rutas temáticas y sus asociaciones con puntos.
Flujo de eventos principal	
Actor	Sistema
1. El administrador accede a la sección “Rutas temáticas” del panel.	
2. El administrador pulsa sobre “Añadir ruta”, “Editar” o “Eliminar”.	
3. El administrador introduce o modifica datos como nombre, descripción y puntos asociados.	
	4. El sistema valida los campos requeridos.
	5. El sistema guarda los datos nuevos o actualizados en la base de datos.
	6. Si se elimina una ruta, se eliminan también sus asociaciones.
	7. El sistema muestra un mensaje de confirmación de los cambios.

4.3.17. Caso de uso 17: Consultar lista de usuarios

Cuadro 4.17: Caso de uso 17 - Consultar lista de usuarios

Nombre del caso de uso	Consultar lista de usuarios
Actores	Administrador (principal), Sistema (secundario)
Propósito	Permitir al administrador visualizar la lista de usuarios registrados, su progreso y gestionar sus cuentas.
Resumen	El administrador puede ver todos los usuarios registrados, consultar su progreso (puntos visitados, logros), y eliminar o restablecer cuentas si es necesario.
Tipo	Primario
Referencias	RF26, RF28, RF29
Precondiciones	El administrador debe haber iniciado sesión correctamente en el panel de administración.
Postcondiciones	Se actualiza la base de datos de usuarios si se elimina o restablece alguna cuenta.
Flujo de eventos principal	
Actor	Sistema
1. El administrador accede a la sección “Usuarios” del panel.	
2. El administrador selecciona un usuario para ver su perfil y progreso.	
3. El administrador puede eliminar o restablecer la cuenta del usuario.	
	4. El sistema muestra la información del usuario y su progreso.
	5. Si se elimina o restablece, el sistema actualiza la base de datos y muestra confirmación.

4.3.18. Caso de uso 18: Gestionar materiales multimedia

Cuadro 4.18: Caso de uso 18 - Gestionar materiales multimedia

Nombre del caso de uso	Gestionar materiales multimedia
Actores	Administrador (principal), Sistema (secundario)
Propósito	Permitir al administrador importar, editar o eliminar imágenes, vídeos y modelos 3D asociados a mujeres o lugares.
Resumen	El administrador puede subir, modificar o eliminar archivos multimedia que se mostrarán en la app, asegurando que el contenido esté actualizado y sea relevante.
Tipo	Primario
Referencias	RF18
Precondiciones	El administrador debe haber iniciado sesión correctamente en el panel de administración.
Postcondiciones	Se actualiza la base de datos y el repositorio de archivos multimedia.
Flujo de eventos principal	
Actor	Sistema
1. El administrador accede a la sección de gestión multimedia.	
2. El administrador selecciona “Añadir”, “Editar” o “Eliminar” un archivo multimedia.	
3. El administrador sube o modifica el archivo y lo asocia a una mujer o lugar.	
	4. El sistema valida el formato y tamaño del archivo.
	5. El sistema guarda o elimina el archivo y actualiza la base de datos.
	6. El sistema muestra un mensaje de confirmación de los cambios.

4.3.19. Caso de uso 19: Gestionar áreas de investigación

Cuadro 4.19: Caso de uso 19 - Gestionar áreas de investigación

Nombre del caso de uso	Gestionar áreas de investigación
Actores	Administrador (principal), Sistema (secundario)
Propósito	Permitir al administrador crear, editar o eliminar áreas de investigación asociadas a mujeres históricas.
Resumen	El administrador puede mantener actualizada la lista de áreas de investigación, que luego pueden asociarse a las mujeres históricas para facilitar la búsqueda y filtrado en la app.
Tipo	Primario
Referencias	RF19
Precondiciones	El administrador debe haber iniciado sesión correctamente en el panel de administración.
Postcondiciones	Se actualiza la base de datos de áreas de investigación.
Flujo de eventos principal	
Actor	Sistema
1. El administrador accede a la sección de áreas de investigación.	
2. El administrador pulsa sobre “Añadir”, “Editar” o “Eliminar” área.	
3. El administrador introduce o modifica el nombre del área.	
	4. El sistema valida el campo requerido.
	5. El sistema guarda o elimina el área en la base de datos.
	6. El sistema muestra un mensaje de confirmación de los cambios.

4.4. Diagramas de Actividad

Los diagramas de actividad son una herramienta utilizada en el modelado de procesos y flujos de trabajo dentro de un sistema. Permiten representar gráficamente la secuencia de actividades, decisiones y flujos alternativos que pueden ocurrir durante la ejecución de un proceso. Son especialmente útiles para visualizar el comportamiento dinámico de un sistema y entender cómo interactúan los distintos componentes o usuarios.

En este diagrama se observan los primeros pasos que sigue un usuario al acceder a la aplicación, pasando por las opciones de registro o inicio de sesión, la validación de datos para llegar a la visualización de la aplicación (véase la Figura 4.3).

El diagrama de la Figura 4.4 muestra, de forma paralela, todas las funcionalidades principales a las que puede acceder un usuario tras iniciar sesión en la aplicación. Cada partición corresponde a una sección clave del menú (mapa interactivo, listado de lugares, rutas temáticas, realidad aumentada y perfil personal) o a las notificaciones de proximidad. En cada bloque se detallan las acciones posibles y las decisiones que puede tomar el usuario.

El diagrama de actividad de la Figura 4.5 corresponde al panel de administración. Ilustra los principales procesos y decisiones que puede realizar un administrador tras iniciar sesión en la plataforma web. El flujo se divide en varias particiones, cada una dedicada a una funcionalidad específica: gestión de mujeres históricas, gestión de lugares geolocalizados, gestión de usuarios, gestión de visitas e historial, y gestión de rutas temáticas. En cada sección se muestran las operaciones CRUD (crear, leer, actualizar, eliminar) correspondientes, así como acciones adicionales como el registro de nuevos usuarios, la consulta y eliminación de historiales de visitas y rutas, y el reinicio de visitas en rutas para usuarios concretos.

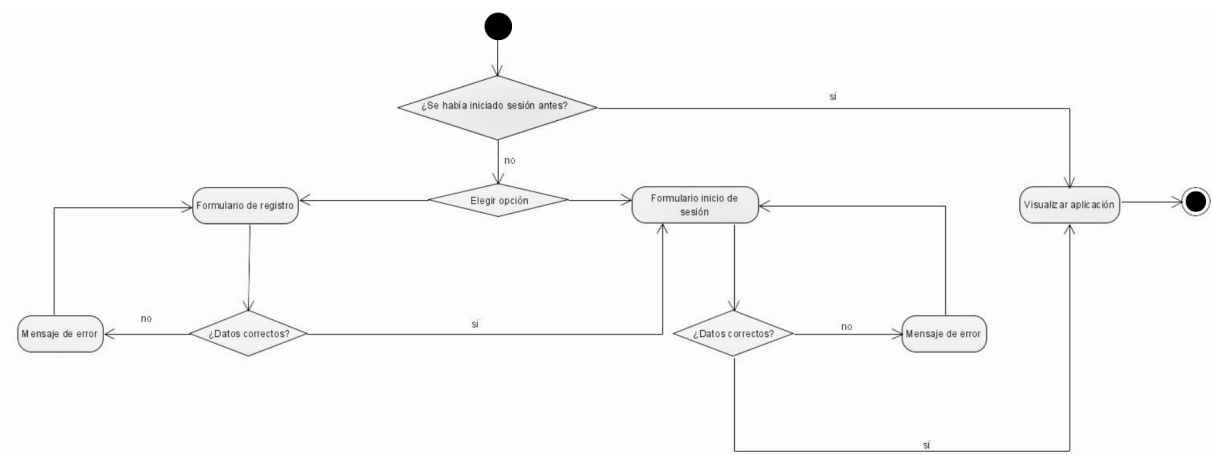


Figura 4.3: Diagrama de actividad de usuario no registrado

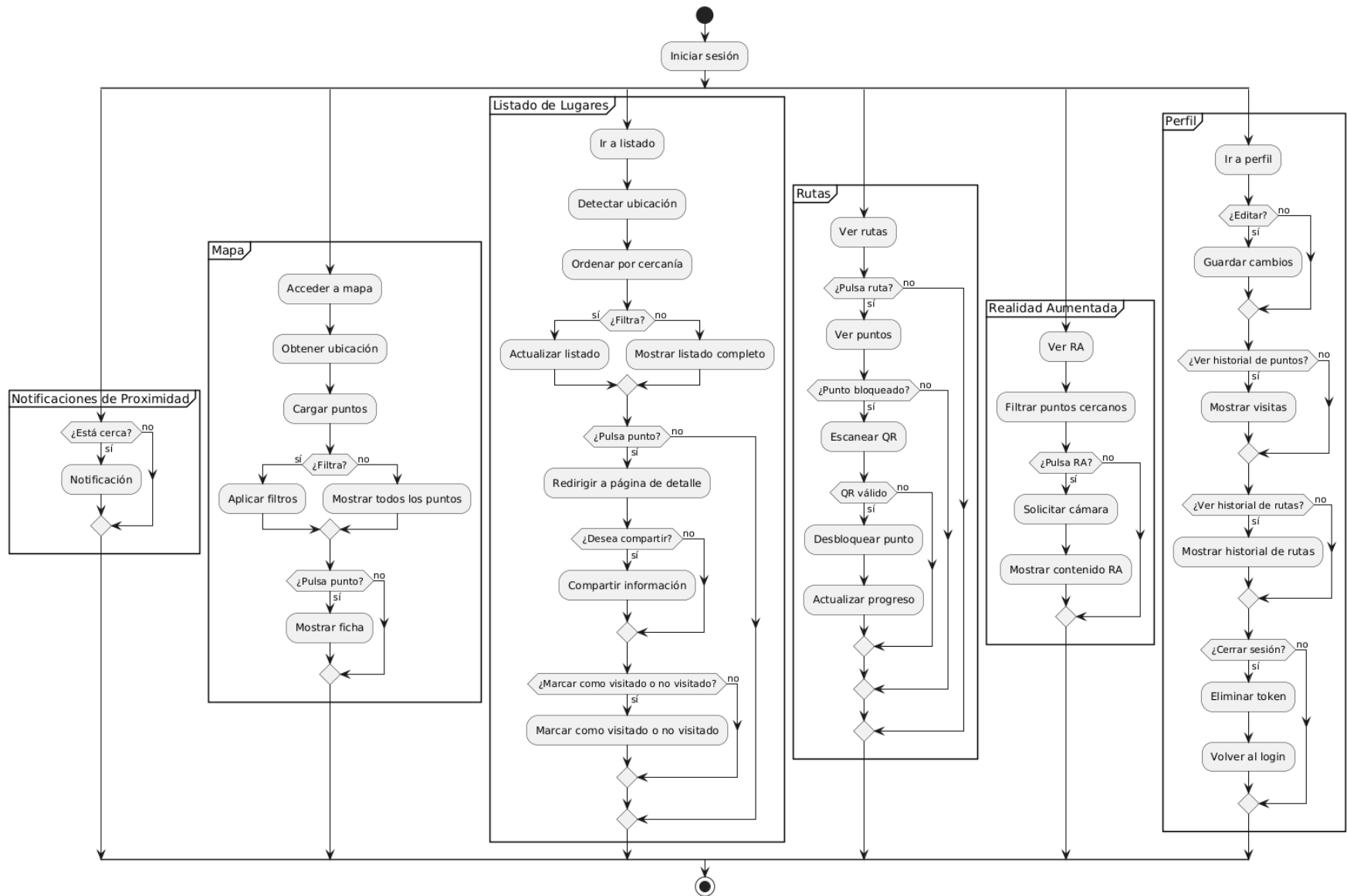


Figura 4.4: Diagrama de actividad de usuario registrado

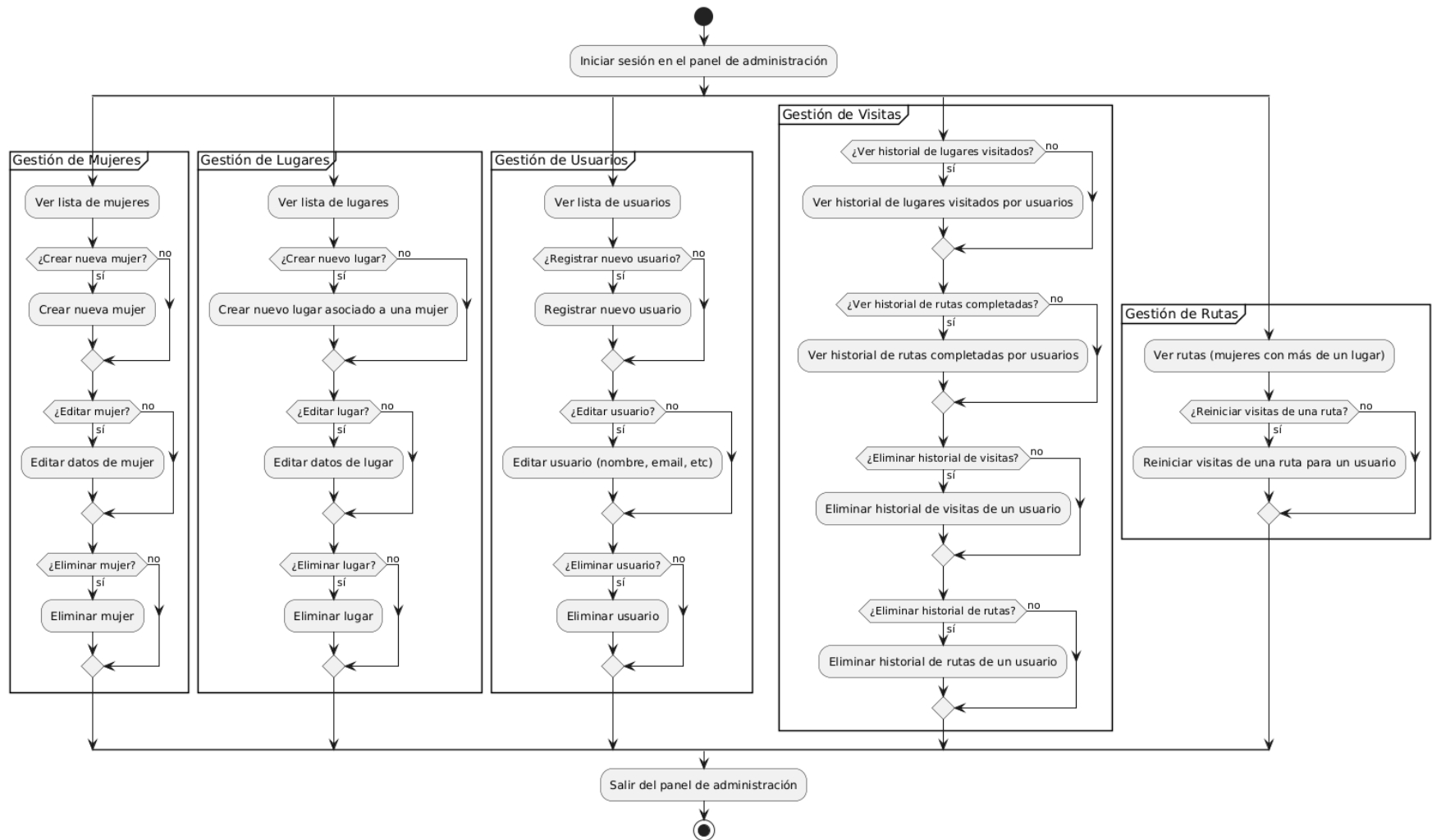


Figura 4.5: Diagrama de actividad del panel de administración

4.5. Diagramas de Estado

Los diagramas de estado son una herramienta fundamental en el modelado de sistemas software, especialmente útil para describir el comportamiento dinámico de un objeto o componente a lo largo de su ciclo de vida. Un diagrama de estado representa los diferentes estados que puede adoptar un elemento del sistema y las transiciones entre ellos, que se producen como respuesta a eventos o acciones. Estos diagramas permiten visualizar de manera clara cómo reacciona el sistema ante distintas situaciones, facilitando la comprensión y el diseño de su lógica interna.

A continuación se presentan **tres diagramas de estado** que describen el comportamiento de los diferentes tipos de usuarios del sistema. El primer diagrama corresponde al **usuario no registrado**, mostrando las transiciones básicas desde el acceso inicial hasta el registro o inicio de sesión. El segundo diagrama representa el **usuario registrado** y, debido a la complejidad y extensión de las funcionalidades disponibles, se ha dividido en **dos partes** para garantizar una visualización óptima y facilitar su comprensión. El tercer diagrama ilustra el comportamiento del **administrador** del sistema, incluyendo todas las operaciones de gestión y mantenimiento disponibles desde el panel administrativo.

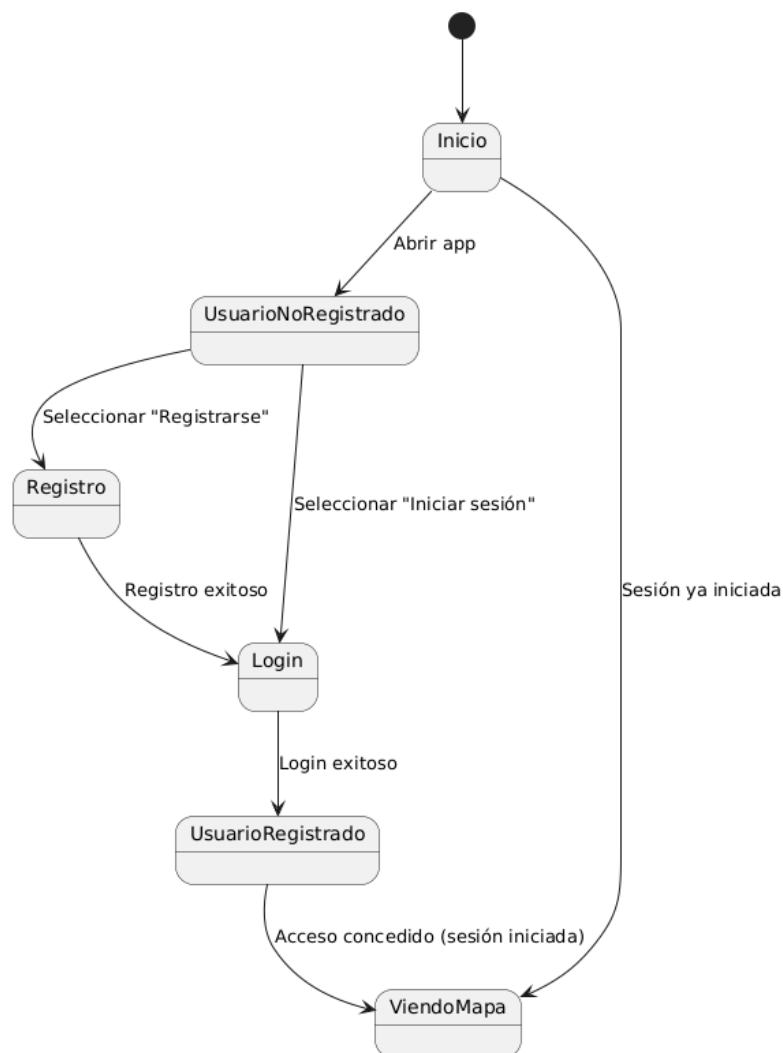


Figura 4.6: Diagrama de estado de usuario no registrado

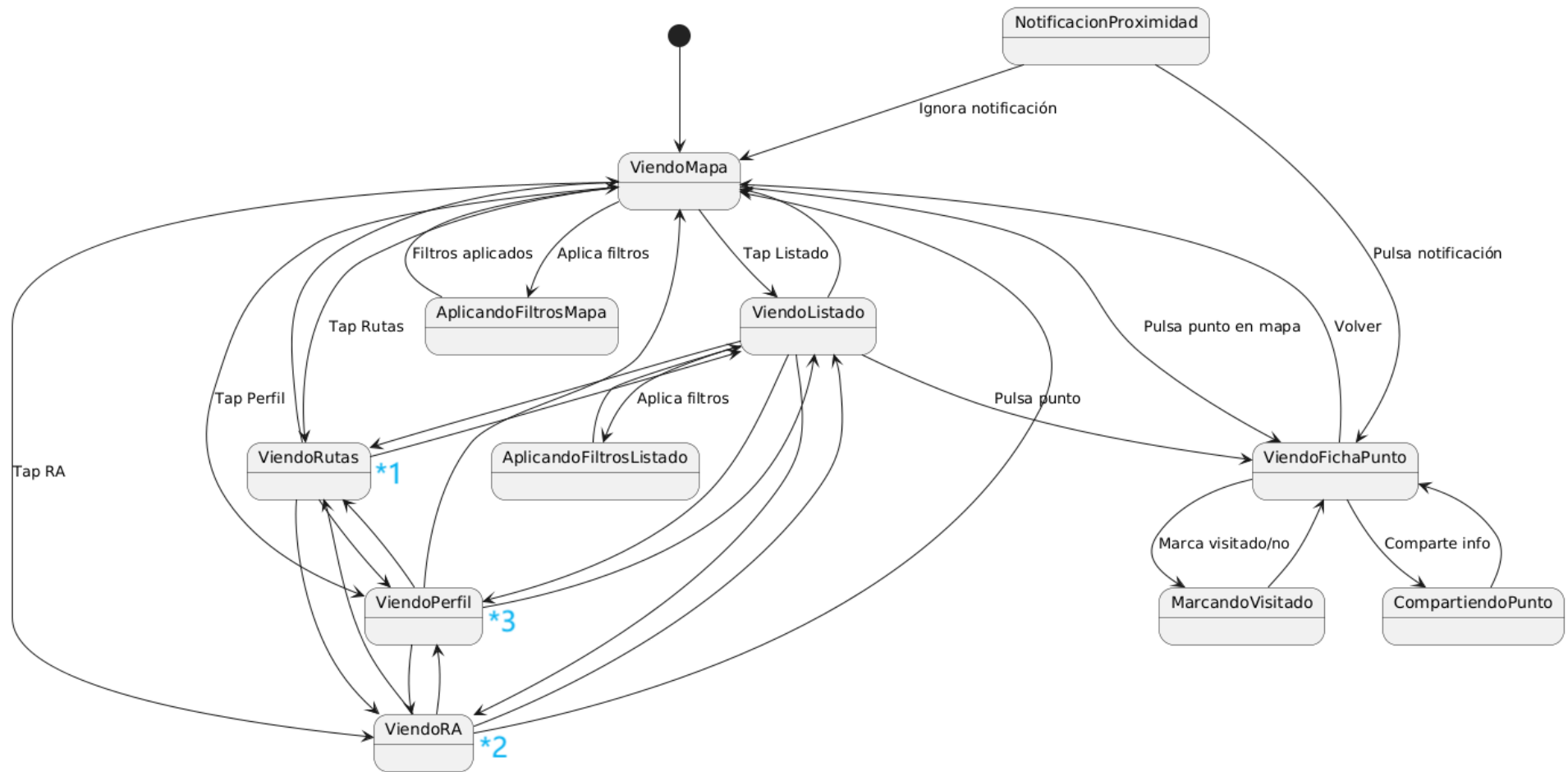


Figura 4.7: Diagrama de estado de usuario registrado 1

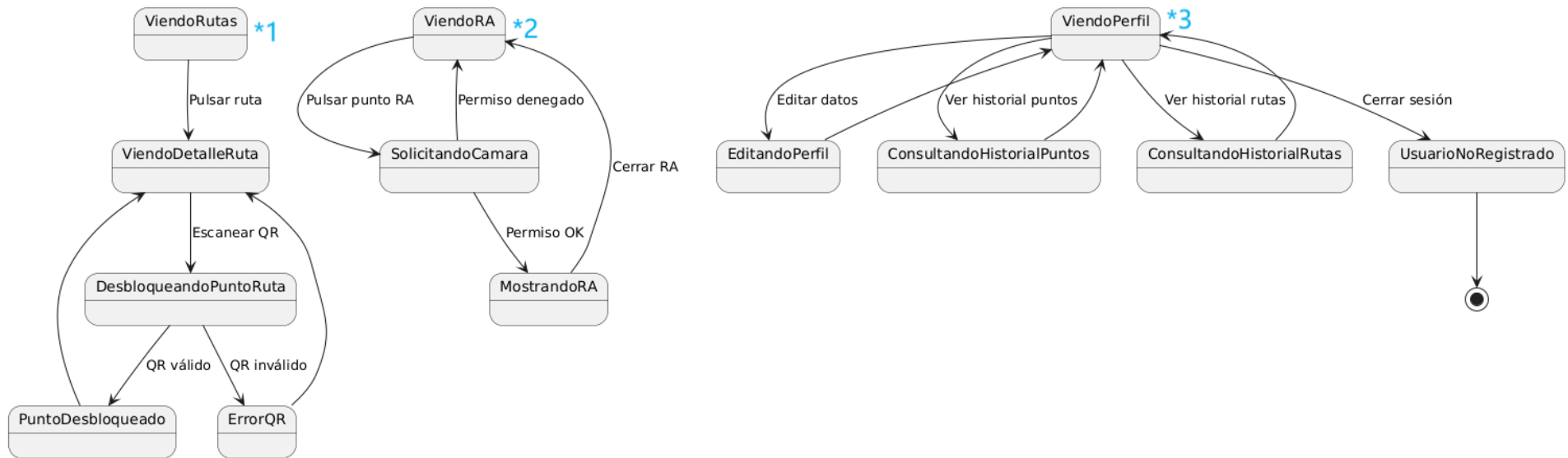


Figura 4.8: Diagrama de estado de usuario registrado 2

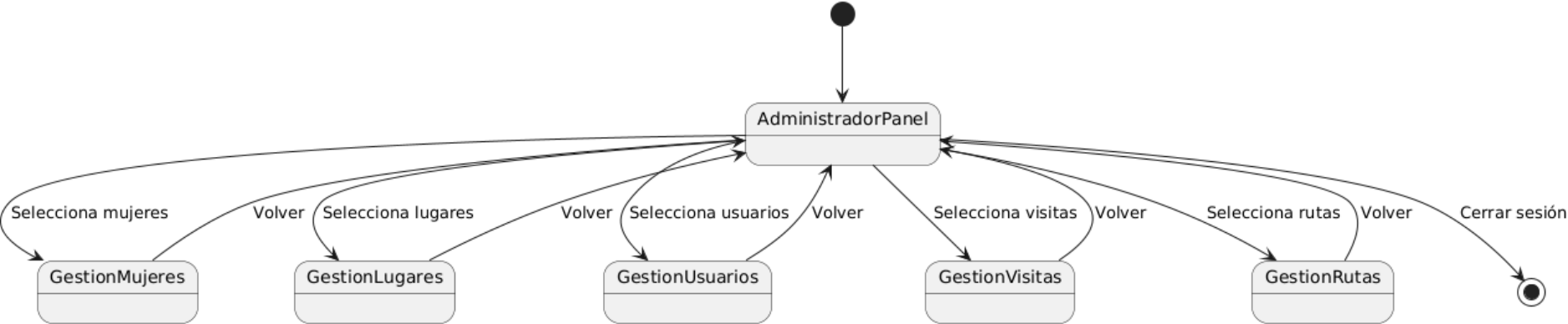


Figura 4.9: Diagrama de estado del panel de administración

Capítulo 5

Diseño

5.1. Diseño de la interfaz

5.1.1. Justificación de la Paleta Cromática

Desde una perspectiva de diseño gráfico, la selección de colores no solo responde a criterios estéticos, sino también a una intención comunicativa y simbólica clara. En *DONA 'm MÓN*, el color **morado** se ha establecido como tono principal por su profundo vínculo con la historia del feminismo. Su uso se remonta al movimiento sufragista de principios del siglo XX, cuando fue adoptado como símbolo de dignidad, fuerza y justicia por organizaciones como la Women's Social and Political Union en Reino Unido [66, 67]. Con el paso del tiempo, el morado se ha convertido en un emblema internacional de la lucha por la igualdad de género [66]. De esta manera, la elección de este color refuerza la identidad visual del proyecto y facilita que los usuarios conecten inmediatamente con los valores y objetivos que representa la aplicación.

Junto al morado, la paleta incluye una gama de colores **análogos**, como el **violeta**, el **rosa**, el **morado claro** y el **magenta**, que comparten una base cromática cercana dentro del círculo cromático (en torno a los rojos y azules) [68]. Esta proximidad favorece una *armonía visual suave*, coherente y atractiva, que permite jugar con distintos niveles jerárquicos y transmitir sensaciones de cohesión, calidez y unidad. El **rosa**, en concreto, se incorpora de forma crítica y resignificada, alejándose de los estereotipos tradicionales y sirviendo como puente entre lo emocional y lo identitario [69]. El **magenta**, más saturado, aporta energía y destaca elementos clave de la interfaz, como botones o llamadas a la acción, favoreciendo la usabilidad y el enfoque visual.

Por otro lado, el color **verde** se utiliza como *complementario* del morado, generando contraste y equilibrio dentro del sistema visual según la teoría del color tradicional [68]. Su inclusión también tiene una base simbólica, ya que formaba parte, junto al morado y al blanco, de la paleta original del movimiento sufragista [67]. Además, el verde sugiere crecimiento, esperanza y transformación, en sintonía con los objetivos educativos y sociales de la aplicación. Finalmente, el uso del **blanco** como color neutro permite crear espacios de respiro visual, mejorar la legibilidad y favorecer una interfaz clara, accesible y ordenada [69].

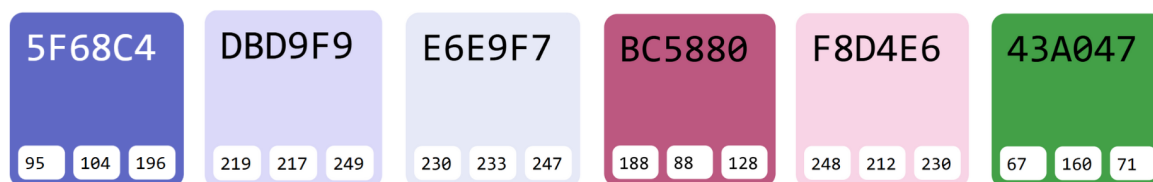


Figura 5.1: Colores de la paleta cromática con código hexadecimal y RGB.

5.1.2. Justificación de la Tipografía

Se han elegido las tipografías **San Francisco** y **Roboto** (Figura 5.2) en el proyecto porque son fuentes modernas, versátiles y especialmente diseñadas para entornos digitales. San Francisco y Roboto son las tipografías nativas de iOS y Android respectivamente, lo que garantiza una integración perfecta con cada sistema operativo y una experiencia visual coherente y profesional para el usuario. Además, Roboto se emplea también en la web de realidad aumentada y mantiene la uniformidad y la legibilidad en todos los dispositivos (IOS y Android).

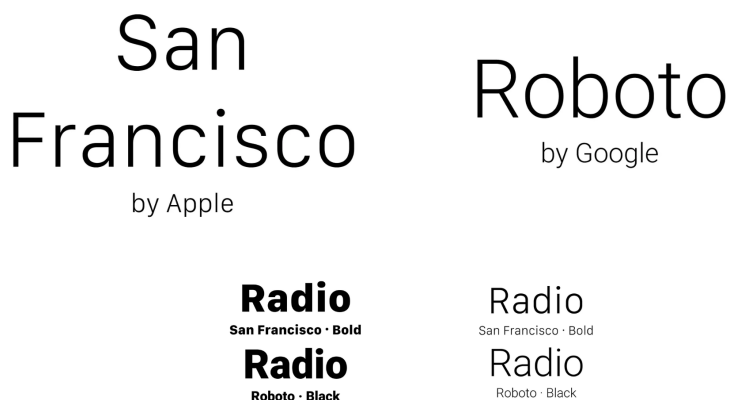
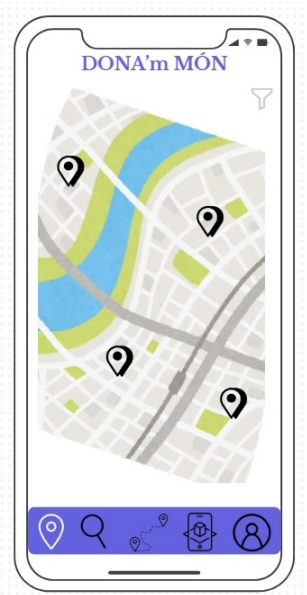


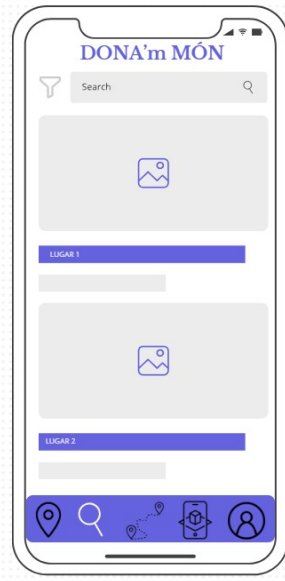
Figura 5.2: Ejemplo de las tipografías San Francisco y Roboto.

5.1.3. Wireframes

Los wireframes son una herramienta fundamental en el proceso de diseño de la interfaz de usuario, ya que permiten visualizar la estructura y disposición de los elementos antes de entrar en detalles estéticos. En el caso de *DONA'm MÓN*, se han creado wireframes para las pantallas principales, siendo estas: el mapa interactivo, las fichas de mujeres y lugares, las rutas temáticas y el perfil de usuario. Estos wireframes se han desarrollado utilizando la herramienta **Canva** [70] y se muestran en la Figura 5.3.



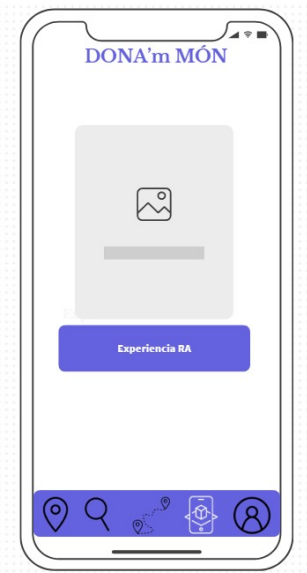
(a) Wireframe 1: Mapa interactivo



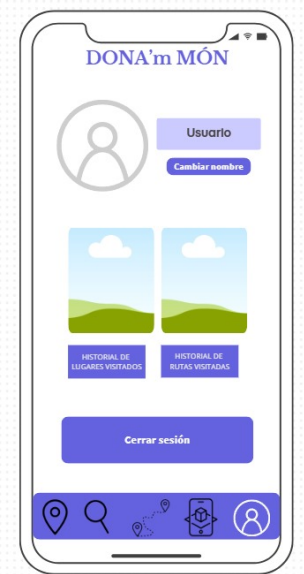
(b) Wireframe 2: Ficha de mujer



(c) Wireframe 3: Rutas temáticas



(d) Wireframe 4: Perfil de usuario



(e) Wireframe 5: Ficha de lugar

Figura 5.3: Wireframes de las pantallas principales de *DONA'm MÓN*

5.2. Diseño de base de datos

La base de datos de la aplicación está diseñada para almacenar información sobre mujeres, sus lugares asociados, rutas temáticas, usuarios y los historiales de visitas. El diseño sigue una estructura relacional y está implementado en **PostgreSQL** mediante modelos definidos en Django.

5.2.1. Tablas principales y justificación de diseño

A continuación se describen las tablas principales que conforman el modelo de datos, junto con la justificación de las decisiones tomadas:

- **AreaInvestigacion**: Permite mantener un catálogo normalizado de áreas de investigación, facilitando la gestión y evitando duplicidades. Se relaciona con las mujeres a través de una relación ManyToMany.
 - **id** (clave primaria, BigAutoField)
 - **nombre**: nombre único del área (CharField, max_length=100, unique=True).
- **Mujer**: Representa a una mujer relevante. El campo `areas_investigacion` se implementa como un `ManyToManyField` hacia la tabla `AreaInvestigacion`, proporcionando normalización de datos y facilitando la gestión desde el panel de administración.
 - **id** (clave primaria, BigAutoField)
 - **nombre** (CharField, max_length=100)
 - **descripcion** (TextField)
 - **foto** (ImageField, upload_to="", null=True, blank=True)
 - **areas_investigacion**: relación ManyToMany con `AreaInvestigacion` (related_name='mujeres', verbose_name='Áreas de investigación')
 - **fechas**: información de fechas relevantes (CharField, max_length=100, null=True, blank=True)
 - **Meta**: ordering=[nombre] para evitar warnings de paginación
- **Lugar**: Almacena información geolocalizada y la relación con una mujer. Se justifica la relación uno-a-muchos para poder asociar varios lugares a una misma mujer.
 - **id** (clave primaria, BigAutoField)
 - **nombre** (CharField, max_length=100)
 - **descripcion** (TextField)
 - **latitud** (FloatField, null=True, blank=True)
 - **longitud** (FloatField, null=True, blank=True)
 - **foto** (ImageField, upload_to="", null=True, blank=True)
 - **ar_url**: enlace a contenido de realidad aumentada (URLField, max_length=300, null=True, blank=True, verbose_name='URL de Realidad Aumentada')
 - **mujer_id** (clave foránea): referencia a la mujer destacada (ForeignKey, related_name='lugares', on_delete=CASCADE)
- **Ruta**: Agrupa lugares y mujeres en recorridos temáticos. Se emplean relaciones ManyToMany para maximizar la flexibilidad y permitir que una ruta incluya múltiples mujeres y lugares, y viceversa.
 - **id** (clave primaria, BigAutoField)

- **nombre** (CharField, max_length=100, unique=True)
- **descripcion** (TextField, blank=True)
- Relación **mujeres**: ManyToManyField con **Mujer** (related_name='rutas', blank=True)
- Relación **lugares**: ManyToManyField con **Lugar** (related_name='rutas', blank=True)
- **UserProfile**: Extiende el modelo de usuario de Django para almacenar información adicional relevante para la aplicación.
 - **id** (clave primaria, BigAutoField)
 - **user_id** (relación uno a uno con User, related_name='profile', on_delete=CASCADE)
 - **birth_date** (DateField, null=True, blank=True)
 - **email** (EmailField, max_length=254, unique=True)
- **VisitedLugar**: Permite registrar el historial de visitas de los usuarios a lugares concretos, facilitando la personalización y la obtención de estadísticas.
 - **id** (clave primaria, BigAutoField)
 - **user_id** (clave foránea, related_name='visited_lugares', on_delete=CASCADE)
 - **lugar_id** (clave foránea, on_delete=CASCADE)
 - **visited_at**: fecha y hora de la visita (DateTimeField, default=timezone.now)
 - **Meta**: unique_together=(úser', 'lugar'), ordering=['-visited_at']
- **VisitedLugarRuta**: Permite registrar visitas de usuarios a lugares dentro de rutas específicas, lo que posibilita analizar el recorrido de los usuarios y mejorar la experiencia de gamificación.
 - **id** (clave primaria, BigAutoField)
 - **user_id** (clave foránea, related_name='visited_lugares_ruta', on_delete=CASCADE)
 - **ruta_id** (clave foránea, related_name='visitas', on_delete=CASCADE)
 - **lugar_id** (clave foránea, on_delete=CASCADE)
 - **visited_at**: fecha y hora de la visita (DateTimeField, default=timezone.now)
 - **Meta**: unique_together=(úser', 'ruta', 'lugar'), ordering=['-visited_at']

5.2.2. Relaciones entre tablas

- Un **Lugar** pertenece a una **Mujer** (relación uno a muchos).
- Una **Mujer** puede tener varios **Lugares** asociados.
- Una **Mujer** puede tener múltiples **AreaInvestigacion** asociadas (relación muchos a muchos).
- Una **AreaInvestigacion** puede estar asociada con múltiples **Mujeres**.
- Una **Ruta** puede incluir varias **Mujeres** y varios **Lugares** (relaciones ManyToMany).

- Un **UserProfile** extiende a un **User** de Django (relación uno a uno).
- **VisitedLugar** y **VisitedLugarRuta** permiten registrar el historial de visitas de los usuarios, tanto de lugares individuales como de lugares dentro de rutas.

5.2.3. Esquema de tablas

El siguiente esquema resume la estructura de las tablas principales de la base de datos:

Tabla	Campos principales
AreaInvestigacion	id (BigAutoField), nombre (CharField, unique)
Mujer	id (BigAutoField), nombre, descripcion, foto (ImageField), areas_investigacion (M2M), fechas, Meta: ordering=[nombre]
Lugar	id (BigAutoField), nombre, descripcion, latitud, longitud, foto (ImageField), ar_url (URLField), mujer_id (FK)
Ruta	id (BigAutoField), nombre (unique), descripcion, mujeres (M2M), lugares (M2M)
UserProfile	id (BigAutoField), user_id (O2O), birth_date, email (unique)
VisitedLugar	id (BigAutoField), user_id (FK), lugar_id (FK), visited_at, Meta: unique_together, ordering
VisitedLugarRuta	id (BigAutoField), user_id (FK), ruta_id (FK), lugar_id (FK), visited_at, Meta: unique_together, ordering

Cuadro 5.1: Esquema resumido de las tablas principales de la base de datos

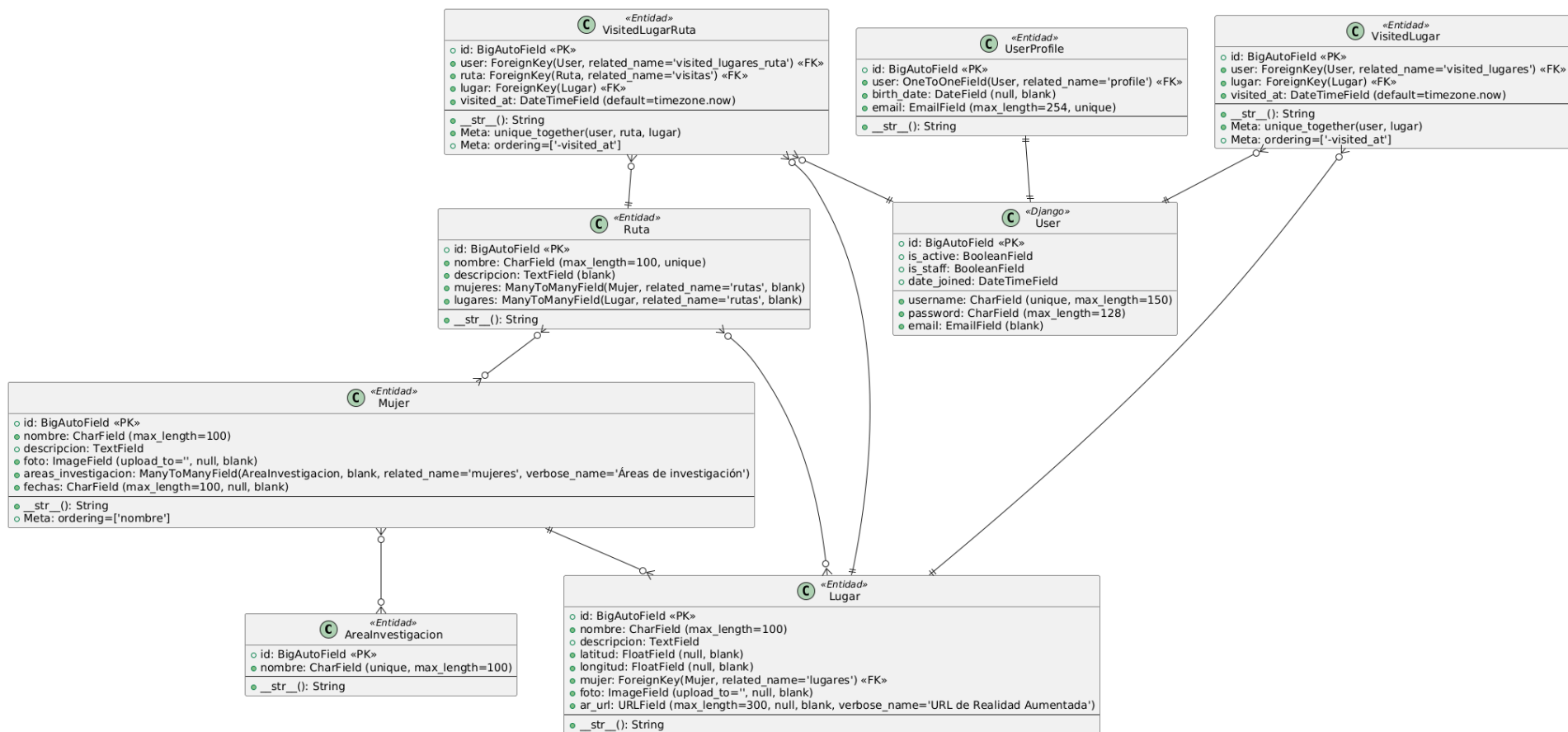


Figura 5.4: Diagrama de clases de la base de datos

5.3. Diseño de la arquitectura del frontend

El frontend de la aplicación está desarrollado en **React Native**, siguiendo una arquitectura basada en componentes funcionales y el uso de contextos para la gestión eficiente del estado global. El hecho de que el diseño sea modular y esté desacoplado hace que sea más sencillo de mantener y escalar, además de permitir añadir nuevas funcionalidades sin complicaciones. Esto también ayuda a que la experiencia de usuario sea más fácil de adaptar en un futuro con nuevas necesidades.

5.3.1. Diagrama de clases del frontend

La Figura 5.5 muestra el diagrama de clases que representa la arquitectura principal del frontend. Los componentes se agrupan en paquetes según su responsabilidad funcional: contextos, navegación, usuario, rutas, lugares y utilidades.

- **Contextos:** El componente `FiltrosContext` centraliza la gestión del estado global de los filtros aplicados en la aplicación, proporcionando métodos para su actualización y reseteo. Este contexto es consumido por componentes como `Filtros`, `Rutas` y `Mapa`, permitiendo la sincronización de los criterios de filtrado en toda la interfaz.
- **Navegación:** El componente `NavigationApp` implementa la navegación principal mediante un *stack navigator*, gestionando el flujo entre pantallas y facilitando la transición entre las distintas funcionalidades de la app.
- **Componentes de Usuario:** Incluyen las pantallas de autenticación (`Login`, `Register`), el perfil de usuario (`PerfilUsuario`) y el acceso al historial de lugares y rutas visitadas (`HistorialLugares`, `HistorialRutas`). Cada uno de estos componentes gestiona su propio estado y lógica de interacción, garantizando la personalización y seguridad de la experiencia de usuario.
- **Componentes de Rutas y Lugares:** `Rutas` y `Mapa` permiten al usuario explorar rutas temáticas y lugares geolocalizados, respectivamente. Los componentes `Detail`, `HistorialLugares` y `HistorialRutas` permiten consultar información detallada de los lugares y el registro de visitas de estos, integrando la lógica de interacción con el backend.
- **Componentes de Utilidad:** Incluyen `Filtros`, `Mujeres` (visualización combinada de mujeres y lugares), la pantalla de bienvenida (`Welcome`), la lógica de realidad aumentada (AR), y la lógica de notificaciones de proximidad (`NotificacionProximidad`), que se ejecuta de forma invisible para el usuario y activa notificaciones según la localización.

Las relaciones entre componentes reflejan tanto el consumo de contexto (*consume*), como la navegación entre pantallas (*navega a*), y el uso de lógica auxiliar (por ejemplo, la integración de notificaciones en el componente principal `Main`).

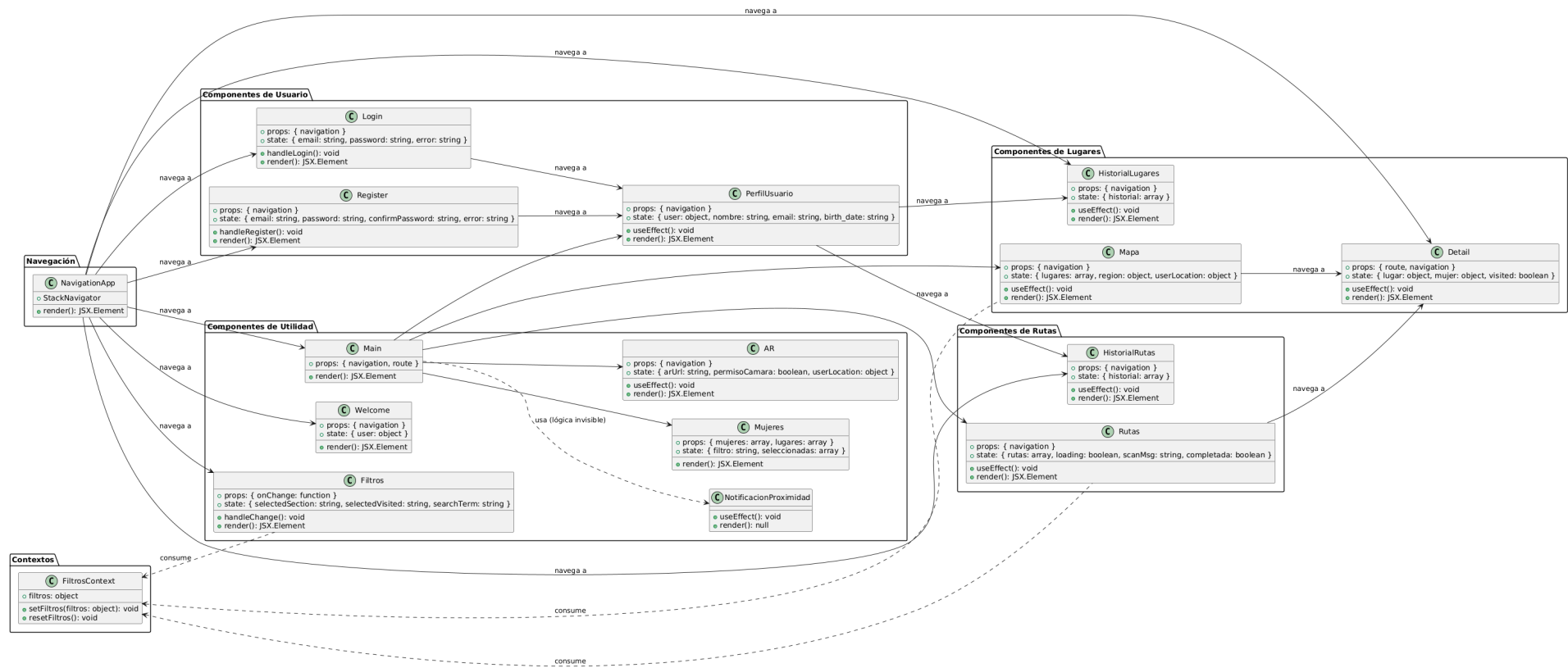


Figura 5.5: Diagrama de clases del frontend React Native: estructura de componentes y relaciones principales.

5.4. Diagramas de secuencia

Los diagramas de secuencia muestran cómo interactúan los distintos elementos del sistema a lo largo del tiempo para llevar a cabo un caso de uso concreto. Representan visualmente el flujo de mensajes entre actores (como el usuario o el administrador) y los componentes internos del sistema (como la aplicación, la base de datos o servicios de autenticación).

En esta sección se presentan los diagramas de secuencia correspondientes a cada uno de los casos de uso definidos en el análisis de requisitos del sistema, descritos en el apartado 4.3 (Especificación de casos de uso).

5.4.1. Diagrama de secuencia del caso de uso 1: Login de usuario

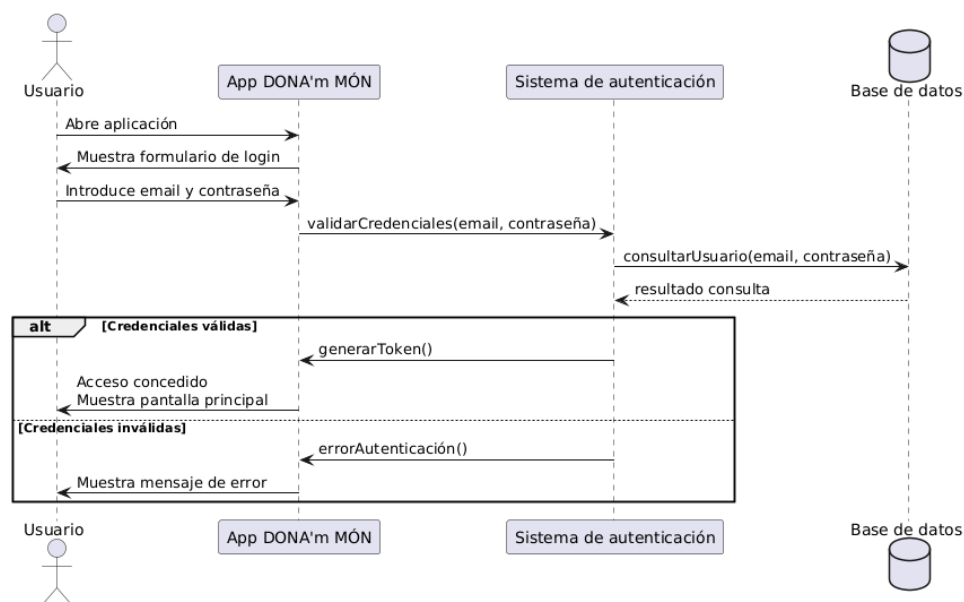


Figura 5.6: Diagrama de secuencia del caso de uso 1: Login de usuario

5.4.2. Diagrama de secuencia del caso de uso 2: Registro de usuario

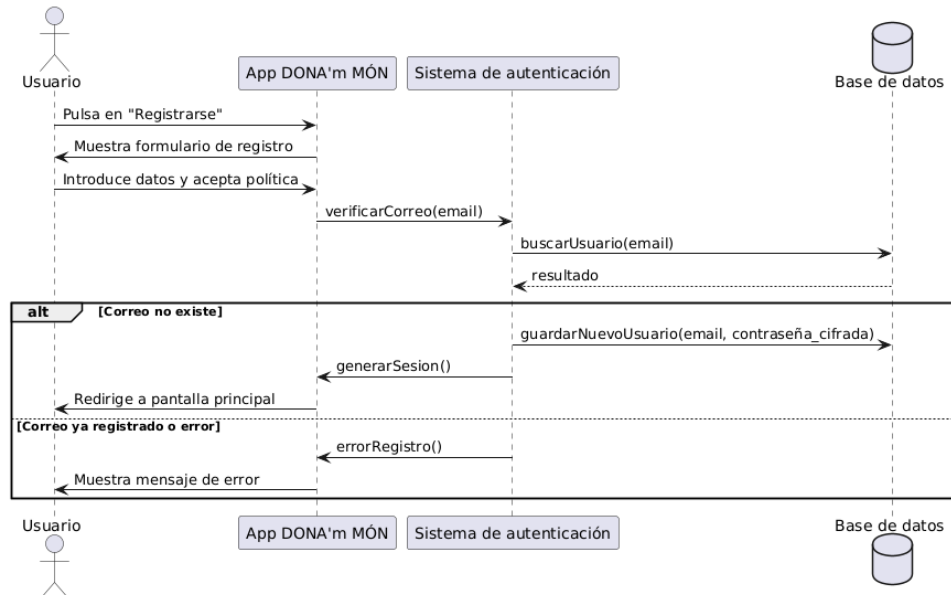


Figura 5.7: Diagrama de secuencia del caso de uso 2: Registro de usuario

5.4.3. Diagrama de secuencia del caso de uso 3: Ver mapa con puntos de interés

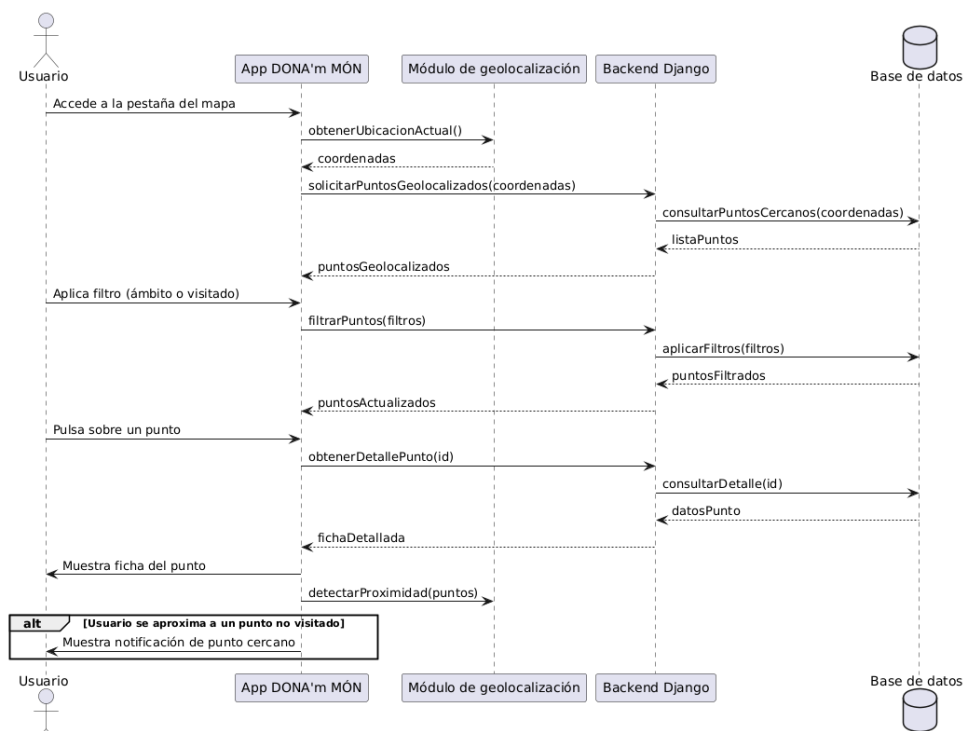


Figura 5.8: Diagrama de secuencia del caso de uso 3: Ver mapa con puntos de interés

5.4.4. Diagrama de secuencia del caso de uso 4: Ver listado de puntos ordenado por cercanía

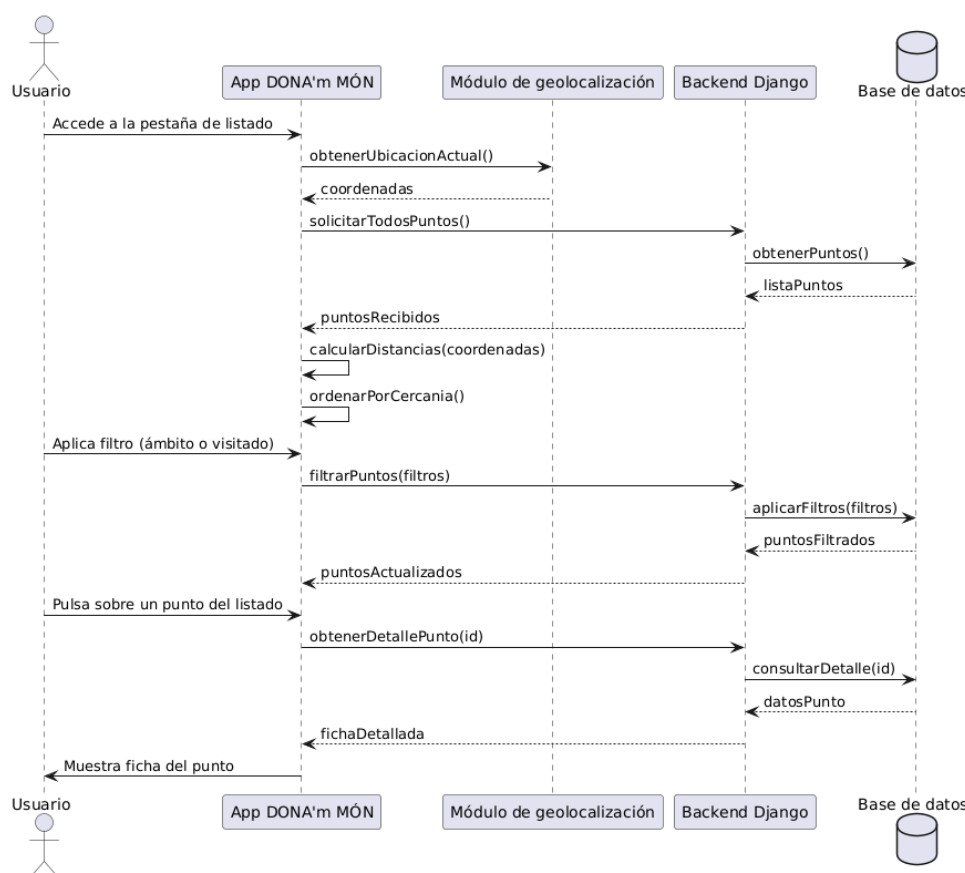


Figura 5.9: Diagrama de secuencia del caso de uso 4: Ver listado de puntos ordenado por cercanía

5.4.5. Diagrama de secuencia del caso de uso 5: Ver ficha del lugar

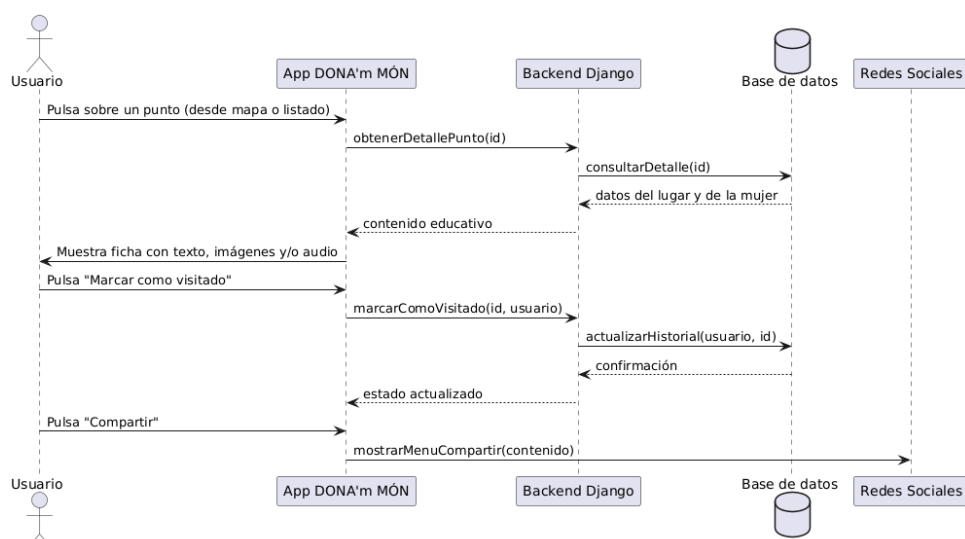


Figura 5.10: Diagrama de secuencia del caso de uso 5: Ver ficha del lugar

5.4.6. Diagrama de secuencia del caso de uso 6: Ver rutas y consultar puntos de una ruta

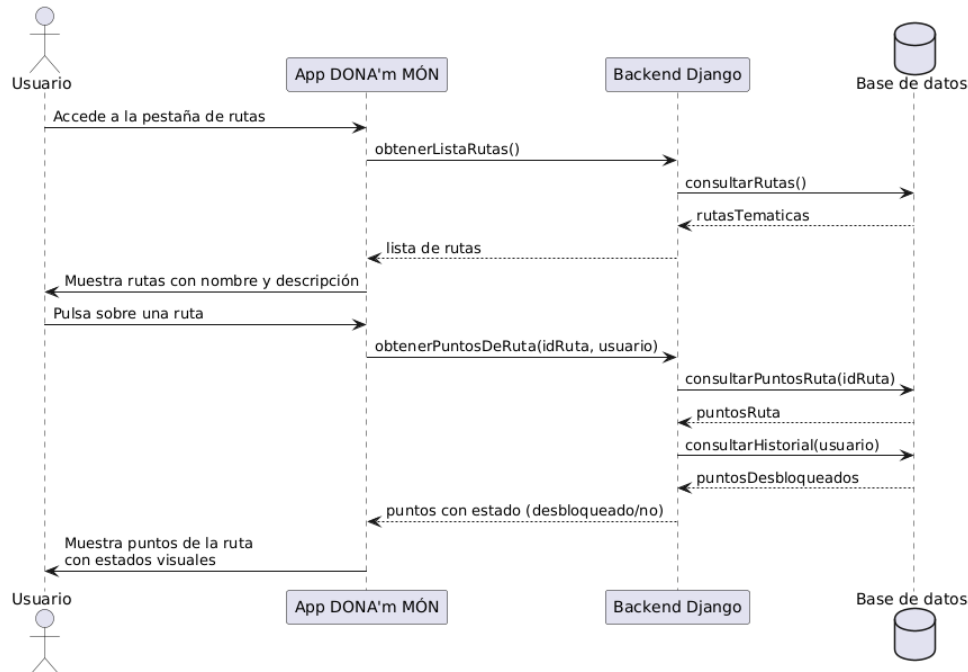


Figura 5.11: Diagrama de secuencia del caso de uso 6: Ver rutas y consultar puntos de una ruta

5.4.7. Diagrama de secuencia del caso de uso 7: Escanear QR de punto de ruta

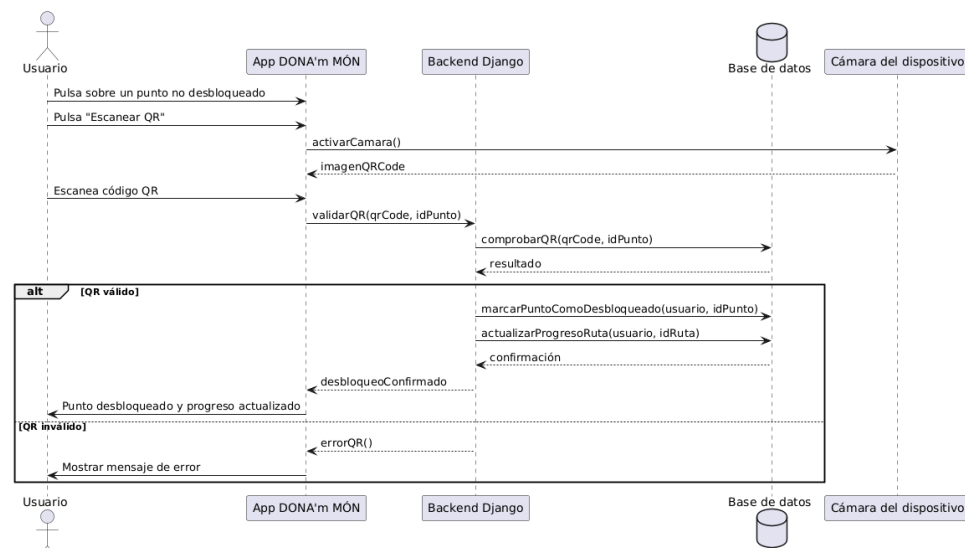


Figura 5.12: Diagrama de secuencia del caso de uso 7: Escanear QR de punto de ruta

5.4.8. Diagrama de secuencia del caso de uso 8: Ver puntos RA disponibles

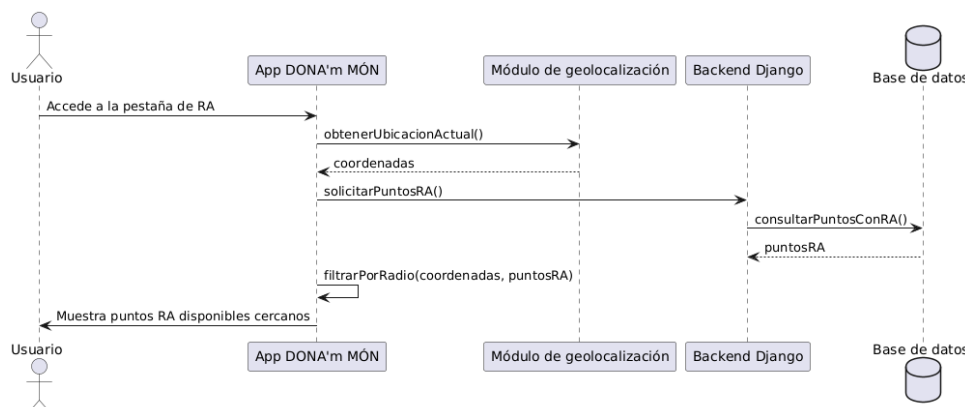


Figura 5.13: Diagrama de secuencia del caso de uso 8: Ver puntos RA disponibles

5.4.9. Diagrama de secuencia del caso de uso 9: Visualizar contenido RA

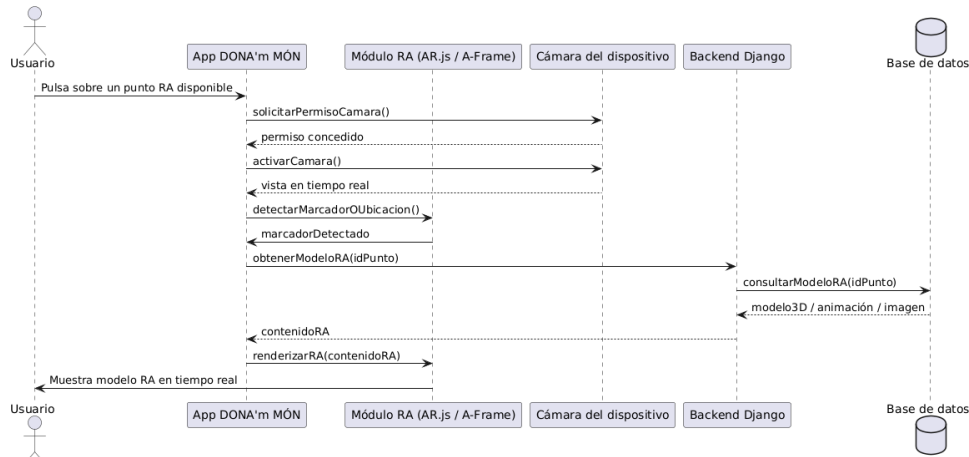


Figura 5.14: Diagrama de secuencia del caso de uso 9: Visualizar contenido RA

5.4.10. Diagrama de secuencia del caso de uso 10: Editar perfil de usuario

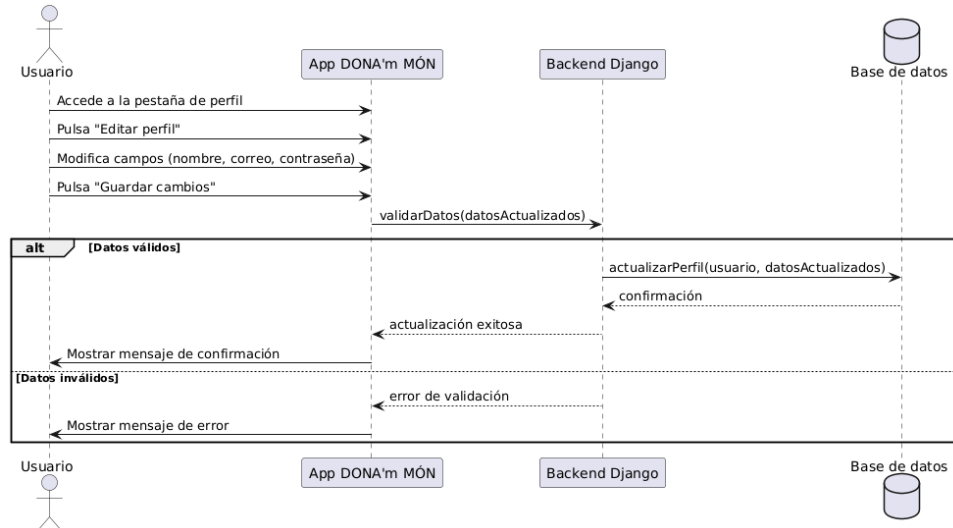


Figura 5.15: Diagrama de secuencia del caso de uso 10: Editar perfil de usuario

5.4.11. Diagrama de secuencia del caso de uso 11: Ver historial de lugares visitados y rutas completadas

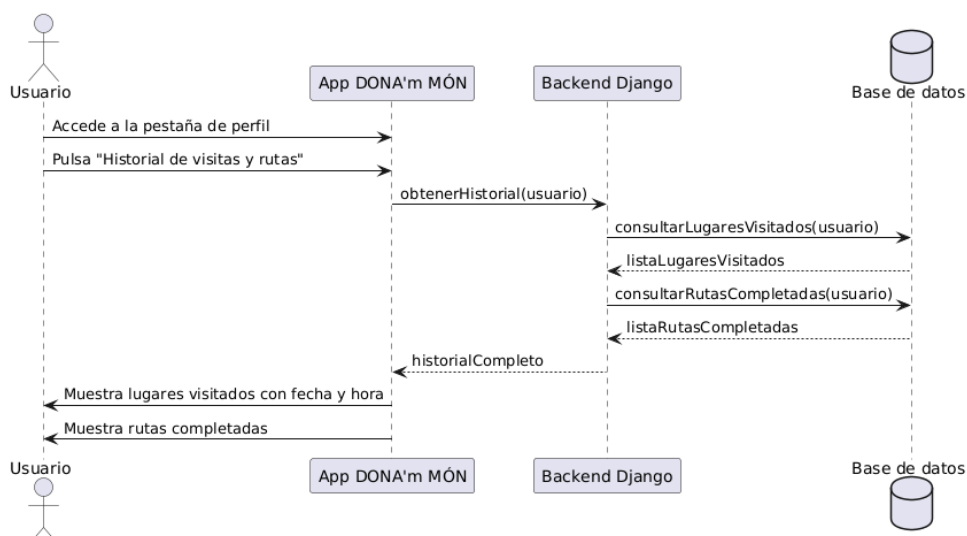


Figura 5.16: Diagrama de secuencia del caso de uso 11: Ver historial de lugares visitados y rutas completadas

5.4.12. Diagrama de secuencia del caso de uso 12: Cerrar sesión

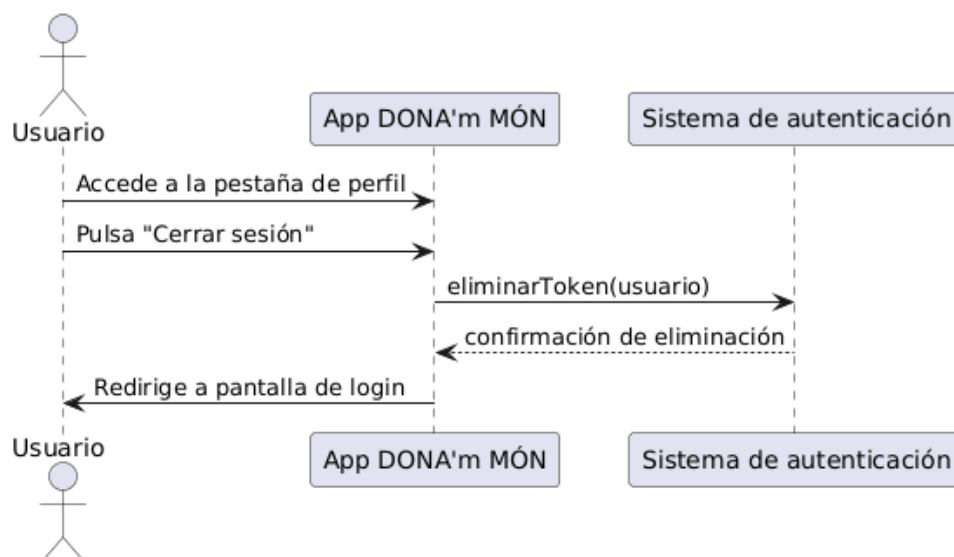


Figura 5.17: Diagrama de secuencia del caso de uso 12: Cerrar sesión

5.4.13. Diagrama de secuencia del caso de uso 13: Iniciar sesión como administrador

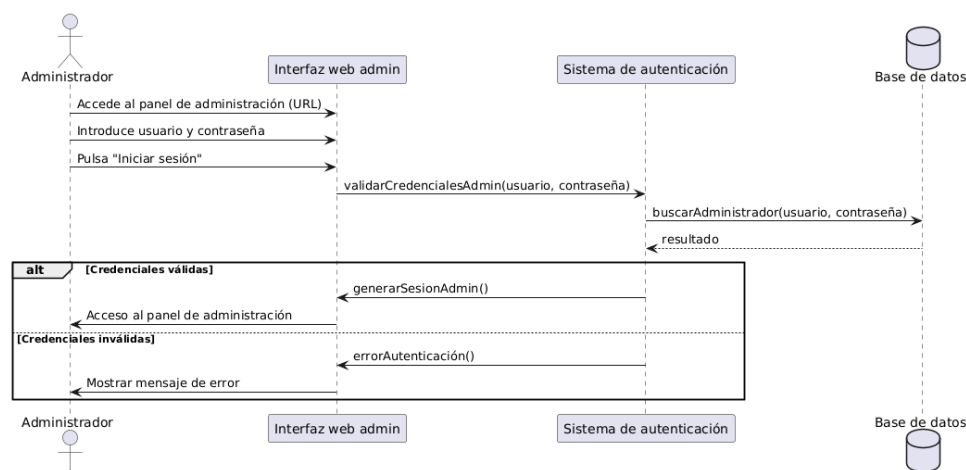


Figura 5.18: Diagrama de secuencia del caso de uso 13: Iniciar sesión como administrador

5.4.14. Diagrama de secuencia del caso de uso 14: Gestionar mujeres históricas

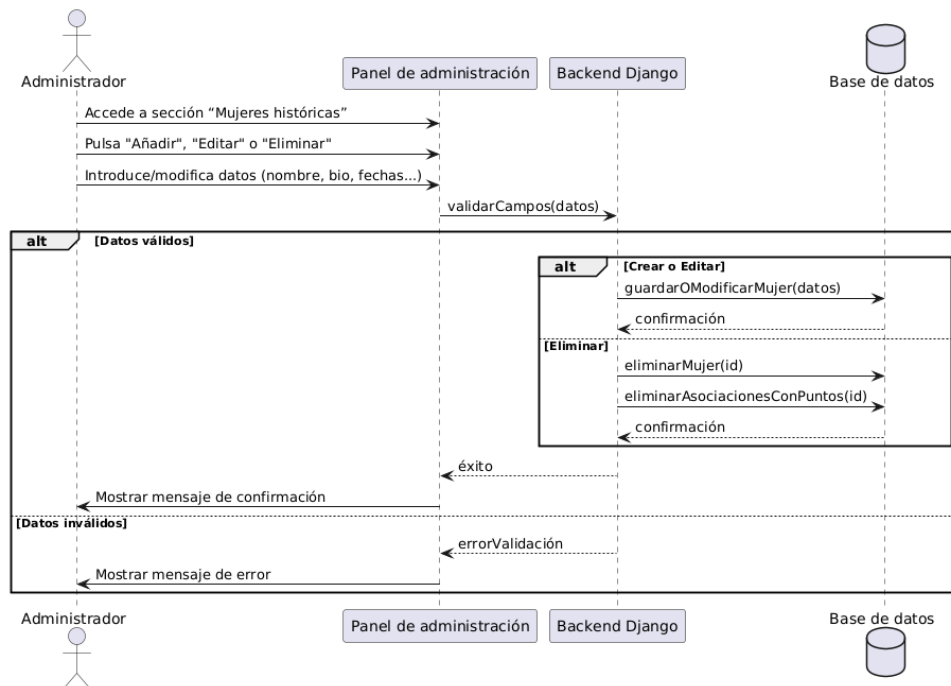


Figura 5.19: Diagrama de secuencia del caso de uso 14: Gestionar mujeres históricas

5.4.15. Diagrama de secuencia del caso de uso 15: Gestionar puntos geolocalizados

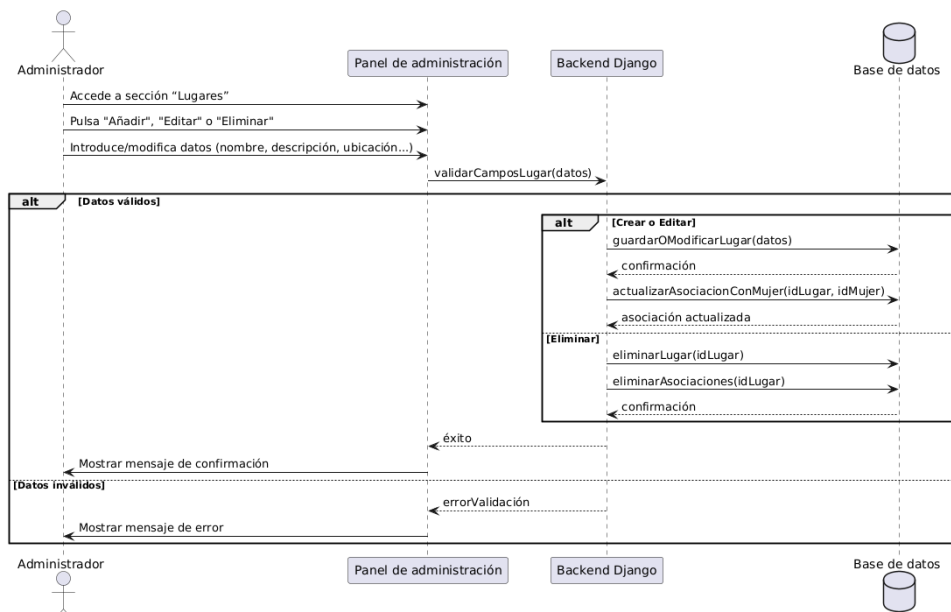


Figura 5.20: Diagrama de secuencia del caso de uso 15: Gestionar puntos geolocalizados

5.4.16. Diagrama de secuencia del caso de uso 16: Gestionar rutas temáticas

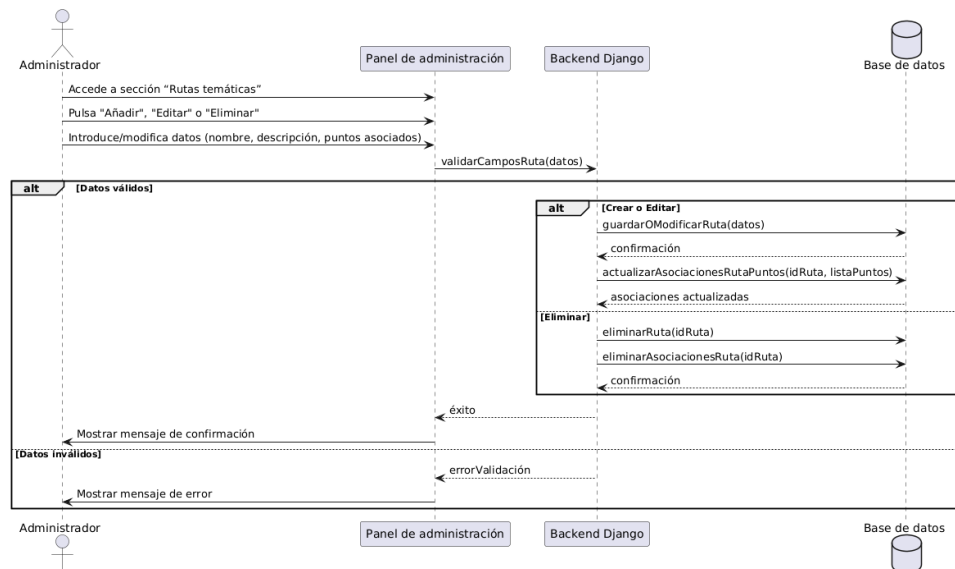


Figura 5.21: Diagrama de secuencia del caso de uso 16: Gestionar rutas temáticas

5.4.17. Diagrama de secuencia del caso de uso 17: Consultar lista de usuarios

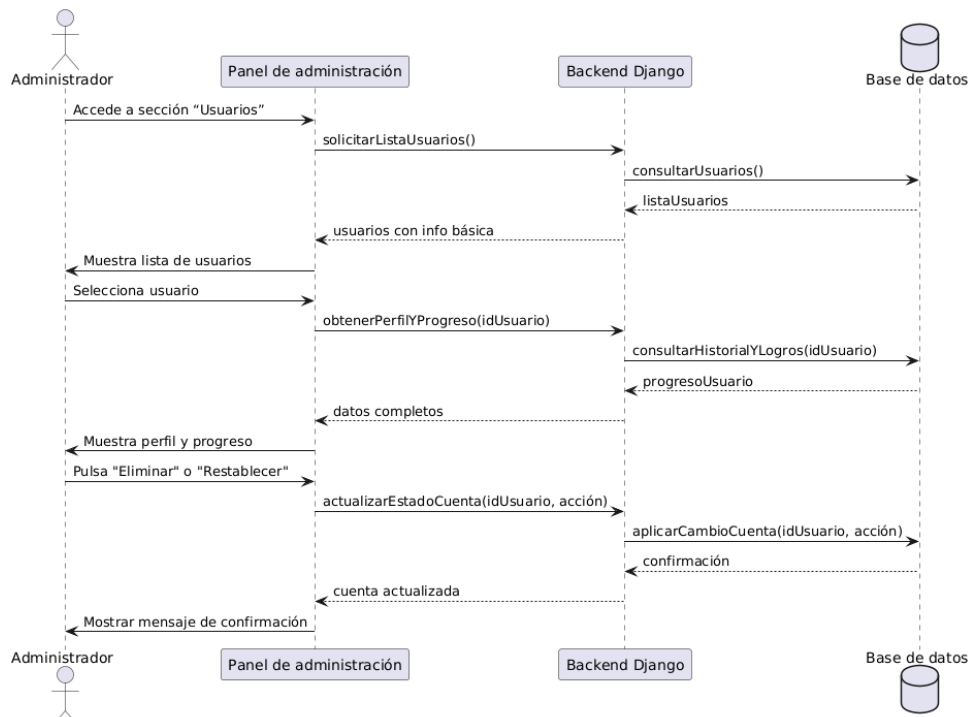


Figura 5.22: Diagrama de secuencia del caso de uso 17: Consultar lista de usuarios

5.4.18. Diagrama de secuencia del caso de uso 18: Gestionar materiales multimedia

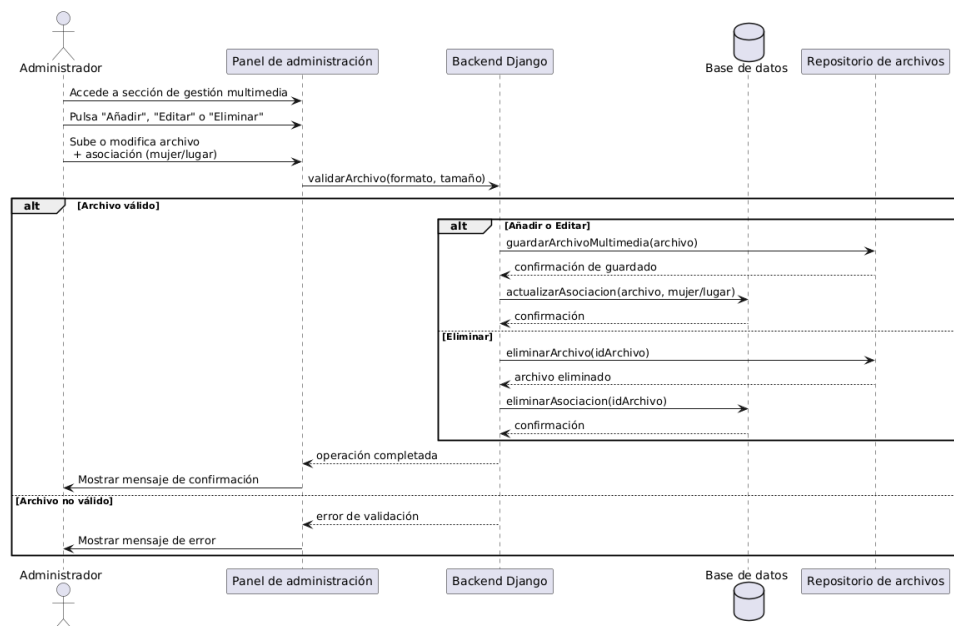


Figura 5.23: Diagrama de secuencia del caso de uso 18: Gestionar materiales multimedia

5.4.19. Diagrama de secuencia del caso de uso 19: Gestionar áreas de investigación

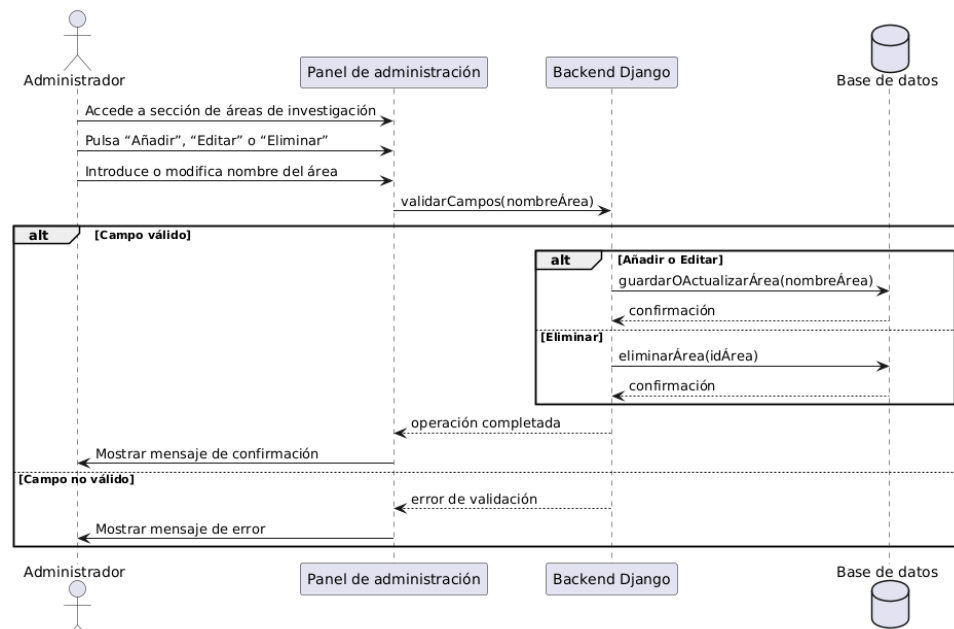


Figura 5.24: Diagrama de secuencia del caso de uso 19: Gestionar áreas de investigación

Capítulo 6

Implementación y pruebas

6.1. Implementación del backend

6.1.1. Herramientas, frameworks y lenguajes utilizados

El backend de la aplicación se ha desarrollado utilizando el framework **Django** (versión 5.2.3) junto con **Django REST Framework** para la construcción de la API. Django facilita la definición de modelos, la gestión de usuarios y la exposición de endpoints RESTful de manera segura y eficiente.

Para la persistencia de datos se ha empleado **PostgreSQL** con extensiones **PostGIS** como sistema gestor de bases de datos relacional. El entorno de desarrollo y despliegue se ha gestionado mediante **Docker** y **Docker Compose**, lo que permite la portabilidad y la replicabilidad del entorno en diferentes máquinas.

Herramientas y tecnologías:

- **Python 3.11**
- **Django 5.2.3**
- **Django REST Framework** (`django-rest-framework`)
- **PostgreSQL** - Sistema gestor de bases de datos
- **Docker** y **Docker Compose** para la gestión de contenedores

Principales librerías:

- `psycopg2-binary` para la conexión con PostgreSQL
- `django-cors-headers` para permitir peticiones desde el frontend
- `Pillow` para el manejo de imágenes en Django

6.1.2. Estructura del proyecto y organización del código

El código fuente del backend se encuentra en la carpeta `donam-mon-backend/`, que contiene la configuración de Docker, los archivos de requisitos y el proyecto Django propiamente dicho. La estructura principal es la siguiente:

Listado 6.1: Estructura principal del backend

```
donam-mon-backend/  
  .env  
  docker-compose.yml  
  Dockerfile  
  requirements.txt  
  
  backend/  
    manage.py  
    db.sqlite3  
    backup_postgres.sql  
  
    config/  
      __init__.py  
      asgi.py  
      settings.py  
      urls.py  
      wsgi.py  
  
    mujeres/  
      __init__.py  
      admin.py  
      apps.py  
      models.py  
      serializers.py  
      tests.py  
      views.py  
      migrations/  
  
    fotos/  
      (archivos de imágenes subidas)  
  
    backend/  
      static/  
        (archivos estáticos de Django y DRF)
```

config/: Contiene la configuración global del proyecto Django (ajustes, rutas principales, etc.).

mujeres/: Es la aplicación principal, donde se definen los modelos, vistas, serializers, administración y lógica de negocio.

fotos/: Directorio donde se almacenan las imágenes subidas.

6.1.3. Modelado de datos

El modelado de datos es uno de los aspectos más relevantes del backend. Se han definido modelos para representar mujeres relevantes, lugares asociados, rutas temáticas, usuarios y el historial de visitas. A continuación se muestra un fragmento representativo del modelo **Mujer**:

Listado 6.2: Modelo Mujer


```

class Mujer(models.Model):
    nombre = models.CharField(max_length=100)
    descripcion = models.TextField()
    foto = models.ImageField(upload_to='', blank=True, null=True)
    areas_investigacion = models.ManyToManyField(
        'AreaInvestigacion',
        blank=True,
        related_name='mujeres',
        verbose_name='Áreas de investigación'
    )
    fechas = models.CharField(max_length=100, blank=True, null=True)

class Meta:
    ordering = ['nombre']

def __str__(self):
    return self.nombre

```

Para las áreas de investigación se utiliza un modelo separado que permite la normalización de datos:

Listado 6.3: Modelo AreaInvestigacion

```

class AreaInvestigacion(models.Model):
    nombre = models.CharField(max_length=100, unique=True)

    def __str__(self):
        return self.nombre

```

Se ha optado por utilizar `ManyToManyField` para el campo `areas_investigacion` en el modelo `Mujer`, estableciendo una relación con la tabla `AreaInvestigacion`.

El modelo `Lugar` representa los lugares geolocalizados asociados a una mujer, e incluye campos para la latitud, longitud, foto y un enlace a contenido de realidad aumentada. En lugar de almacenar los modelos 3D directamente en la base de datos o en el propio backend, se ha optado por subir los archivos 3D y el HTML necesario a **Vercel** y guardar únicamente la URL de acceso en el campo `ar_url`. De esta forma, la base de datos y el backend no se ven saturados con archivos grandes ni con peticiones de contenido estático. Además, al utilizar **Vercel** como plataforma optimizada para servir archivos estáticos y contenido web se mejora tanto la velocidad de carga como la escalabilidad de la aplicación gracias a su **CDN global**. Así, el campo `ar_url` actúa como referencia a la experiencia de realidad aumentada, permitiendo que el frontend acceda directamente al recurso externo de forma eficiente y desacoplada del backend. Esta estrategia también facilita la actualización y gestión de los modelos 3D y sus visores HTML, ya que no es necesario modificar ni desplegar el backend cada vez que se realiza un cambio en estos recursos.

Listado 6.4: Modelo Lugar

```

class Lugar(models.Model):
    nombre = models.CharField(max_length=100)
    descripcion = models.TextField()
    latitud = models.FloatField(null=True, blank=True)
    longitud = models.FloatField(null=True, blank=True)

```

```

mujer = models.ForeignKey(Mujer, related_name='lugares', on_delete=models.
    CASCADE)
foto = models.ImageField(upload_to='', null=True, blank=True)
ar_url = models.URLField(max_length=300, null=True, blank=True, verbose_name
    ='URL de Realidad Aumentada')

def __str__(self):
    return self.nombre

```

El modelo Ruta agrupa lugares y mujeres en recorridos temáticos, utilizando relaciones ManyToManyField para maximizar la flexibilidad:

Listado 6.5: Modelo Ruta

```

class Ruta(models.Model):
    nombre = models.CharField(max_length=100, unique=True)
    descripcion = models.TextField(blank=True)
    mujeres = models.ManyToManyField('Mujer', related_name='rutas', blank=True)
    lugares = models.ManyToManyField('Lugar', related_name='rutas', blank=True)

    def __str__(self):
        return self.nombre

```

Para la gestión de usuarios se utiliza el modelo estándar de Django, extendido mediante un modelo UserProfile que almacena información adicional relevante para la aplicación:

Listado 6.6: Modelo UserProfile

```

class UserProfile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE, related_name='
        profile')
    birth_date = models.DateField(null=True, blank=True)
    email = models.EmailField(max_length=254, unique=True)

    def __str__(self):
        return self.user.username

```

El historial de visitas se gestiona con los modelos VisitedLugar y VisitedLugarRuta, que permiten registrar tanto visitas individuales como visitas dentro de rutas temáticas, tienen restricciones unique_together y ordenamiento temporal:

Listado 6.7: Modelo VisitedLugar

```

class VisitedLugar(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE, related_name='
        visited_lugares')
    lugar = models.ForeignKey(Lugar, on_delete=models.CASCADE)
    visited_at = models.DateTimeField(default=timezone.now)

    class Meta:
        unique_together = ('user', 'lugar')
        ordering = ['-visited_at']

    def __str__(self):

```

```
return f"{self.user.username} visitó {self.lugar.nombre} el {self.
visited_at}"
```

Listado 6.8: Modelo VisitedLugarRuta

```
class VisitedLugarRuta(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE, related_name='
        visited_lugares_ruta')
    ruta = models.ForeignKey('Ruta', on_delete=models.CASCADE, related_name='
        visitas')
    lugar = models.ForeignKey(Lugar, on_delete=models.CASCADE)
    visited_at = models.DateTimeField(default=timezone.now)

    class Meta:
        unique_together = ('user', 'ruta', 'lugar')
        ordering = ['-visited_at']

    def __str__(self):
        return f"{self.user.username} visitó {self.lugar.nombre} en ruta {self.
            ruta.nombre} el {self.visited_at}"
```

6.1.4. Gestión de imágenes

Para la gestión de imágenes, se utiliza el campo `ImageField` en los modelos. En la configuración de Django se define la ruta de almacenamiento y la URL de acceso a los archivos subidos:

Listado 6.9: Configuración de imágenes en settings.py

```
MEDIA_URL = '/fotos/'
MEDIA_ROOT = os.path.join(BASE_DIR, 'fotos')
```

En `config/urls.py` se añade la siguiente configuración para servir archivos en desarrollo:

Listado 6.10: Configuración de media en urls.py

```
from django.conf import settings
from django.conf.urls.static import static

# ...

urlpatterns = [
    # ... rutas ...
] + static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

6.1.5. Paginación de la API

Para mejorar la eficiencia y la experiencia de usuario, la API implementa paginación por defecto:

Listado 6.11: Paginación en settings.py

```
REST_FRAMEWORK = {
```

```
'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
',
'PAGE_SIZE': 10,
}
```

6.1.6. Seguridad y permisos

La autenticación de usuarios se gestiona mediante el sistema estándar de Django, complementado con autenticación basada en tokens para la API. En la configuración de Django REST Framework se especifica el uso de `TokenAuthentication` y los permisos por defecto:

Listado 6.12: Configuración de autenticación y permisos

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework.authentication.TokenAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
}
```

Nota sobre configuración:

Para **desarrollo** se recomienda:

- `DEBUG = True`: Permite debugging y mensajes de error detallados.
- `CORS_ALLOW_ALL_ORIGINS = True`: Facilita pruebas desde cualquier origen.
- `ALLOWED_HOSTS = []`: Acepta cualquier host en modo debug.

Para **producción** es obligatorio:

- `DEBUG = False`
- `SECRET_KEY` desde variable de entorno.
- `CORS_ALLOWED_ORIGINS` restringido a dominios específicos.
- `ALLOWED_HOSTS` configurado con dominios autorizados.

6.1.7. Gestión de variables de entorno y configuración segura

Para evitar exponer credenciales sensibles en el código fuente, se utiliza un archivo `.env` para definir variables de entorno como las credenciales de la base de datos. El archivo `docker-compose.yml` y la configuración de Django acceden a estas variables para una gestión segura y flexible.

Listado 6.13: Variables de entorno para desarrollo

```
POSTGRES_DB=donamon
POSTGRES_USER=ivana
POSTGRES_PASSWORD=superclave
```

Nota: Las credenciales mostradas son apropiadas para el entorno de desarrollo del proyecto. En un despliegue de producción, estas deberían ser reemplazadas por credenciales robustas generadas aleatoriamente.

6.1.8. Serialización y exposición de la API REST

La serialización de los modelos se realiza mediante clases `ModelSerializer`, que permiten transformar los objetos de la base de datos en formatos JSON fácilmente consumibles por el frontend y validar los datos recibidos antes de almacenarlos. En este proyecto, los serializers también gestionan relaciones entre modelos y añaden campos personalizados, como URLs absolutas para las imágenes.

Por ejemplo, el serializador para el modelo `Mujer` incluye la serialización anidada de los lugares asociados y un campo calculado para la URL de la foto:

Listado 6.14: Serializador Mujer

```
class MujerSerializer(serializers.ModelSerializer):
    lugares = LugarSerializer(many=True, read_only=True)
    foto_url = serializers.SerializerMethodField()
    areas_investigacion = AreaInvestigacionSerializer(many=True, read_only=True)

    class Meta:
        model = Mujer
        fields = ('id', 'nombre', 'descripcion', 'foto', 'foto_url', 'lugares', 'areas_investigacion', 'fechas')

    def get_foto_url(self, obj):
        request = self.context.get('request')
        if obj.foto:
            url = obj.foto.url
            url = url.replace('/fotos/fotos/', '/fotos/')
            if request is not None:
                return request.build_absolute_uri(url)
            else:
                return url
        return None
```

El serializador para el modelo `Lugar` también incluye la relación con la mujer asociada (mostrando su nombre), así como un campo personalizado para la URL de la foto:

Listado 6.15: Serializador Lugar

```
class LugarSerializer(serializers.ModelSerializer):
    mujer = serializers.StringRelatedField()
    foto_url = serializers.SerializerMethodField()

    class Meta:
        model = Lugar
        fields = ('id', 'nombre', 'descripcion', 'latitud', 'longitud', 'mujer', 'foto', 'foto_url', 'ar_url')
```

```
def get_foto_url(self, obj):
    request = self.context.get('request')
    if obj.foto:
        url = obj.foto.url
        url = url.replace('/fotos/fotos/', '/fotos/')
        if request is not None:
            return request.build_absolute_uri(url)
        else:
            return url
    return None
```

El serializador de perfil de usuario permite exponer y validar los datos adicionales del usuario:

Listado 6.16: Serializador UserProfile

```
class UserProfileSerializer(serializers.ModelSerializer):
    class Meta:
        model = UserProfile
        fields = ['birth_date', 'email']
```

El serializador de registro de usuario permite crear un usuario y su perfil asociado en una sola petición:

Listado 6.17: Serializador UserRegister

```
class UserRegisterSerializer(serializers.ModelSerializer):
    profile = UserProfileSerializer()
    class Meta:
        model = User
        fields = ['username', 'password', 'profile']
        extra_kwargs = {'password': {'write_only': True}}

    def create(self, validated_data):
        profile_data = validated_data.pop('profile')
        user = User.objects.create_user(username=validated_data['username'],
                                         password=validated_data['password'])
        UserProfile.objects.create(user=user, **profile_data)
        return user
```

El serializador de visitas a lugares incluye la información completa del lugar visitado:

Listado 6.18: Serializador VisitedLugar

```
class VisitedLugarSerializer(serializers.ModelSerializer):
    lugar = serializers.SerializerMethodField()
    class Meta:
        model = VisitedLugar
        fields = ['id', 'lugar', 'visited_at']

    def get_lugar(self, obj):
        return LugarSerializer(obj.lugar, context=self.context).data
```

El serializador de visitas a lugares dentro de rutas:

Listado 6.19: Serializador VisitedLugarRuta

```
class VisitedLugarRutaSerializer(serializers.ModelSerializer):
    lugar = LugarSerializer(read_only=True)
    mujer = serializers.StringRelatedField(read_only=True)

    class Meta:
        model = VisitedLugarRuta
        fields = ('id', 'user', 'mujer', 'lugar', 'visited_at')
```

Y el serializador de rutas, que expone tanto las mujeres como los lugares asociados:

Listado 6.20: Serializador Ruta

```
class RutaSerializer(serializers.ModelSerializer):
    mujeres = serializers.StringRelatedField(many=True)
    lugares = LugarSerializer(many=True, read_only=True)
    class Meta:
        model = Ruta
        fields = ('id', 'nombre', 'descripcion', 'mujeres', 'lugares')
```

6.1.9. Vistas y endpoints de la API

Las vistas de la API se han implementado principalmente mediante `ModelViewSet`, lo que permite disponer de endpoints CRUD (crear, leer, actualizar, borrar) de forma automática y segura. Por ejemplo, la vista para el modelo `Mujer` es:

Listado 6.21: Vista `MujerViewSet` con permisos condicionales

```
class MujerViewSet(viewsets.ModelViewSet):
    queryset = Mujer.objects.all()
    serializer_class = MujerSerializer

    def get_permissions(self):
        if self.action == 'list':
            return [AllowAny()]
        return [IsAuthenticated()]
```

Para la lógica de visitas, se han implementado endpoints que permiten registrar una visita y consultar el historial de visitas de un usuario. Un ejemplo de lógica relevante en la vista podría ser la validación para evitar duplicados en el historial:

Listado 6.22: Vista para gestión de visitas

```
class VisitedLugarView(APIView):
    authentication_classes = [TokenAuthentication]
    permission_classes = [IsAuthenticated]

    def post(self, request):
        lugar_id = request.data.get('lugar_id')
        if not lugar_id:
            return Response({'error': 'lugar_id es requerido'}, status=400)
```

```
lugar = get_object_or_404(Lugar, id=lugar_id)
visited, created = VisitedLugar.objects.get_or_create(user=request.user,
    lugar=lugar)
if not created:
    return Response({'message': 'Ya registrado', 'visited_at': visited.
        visited_at}, status=200)
serializer = VisitedLugarSerializer(visited)
return Response(serializer.data, status=201)
```

6.1.10. Administración y gestión de datos

Django proporciona una interfaz de administración (*Django Admin*) que ha sido personalizada para facilitar la gestión de los modelos principales de la aplicación (véase la figura 6.1). En el archivo `admin.py` de la app `mujeres`, se han registrado los modelos y se han configurado opciones de visualización y filtrado para mejorar la experiencia de los administradores:

Listado 6.23: Configuración personalizada de Django Admin

```
from django.contrib import admin
from .models import Mujer, Lugar, VisitedLugarRuta, Ruta, AreaInvestigacion

class LugarAdmin(admin.ModelAdmin):
    list_display = ('nombre', 'descripcion', 'latitud', 'longitud', 'ar_url')
    search_fields = ('nombre',)

class MujerAdmin(admin.ModelAdmin):
    list_display = ('nombre', 'descripcion')
    search_fields = ('nombre',)
    filter_horizontal = ('areas_investigacion',)

class RutaAdmin(admin.ModelAdmin):
    list_display = ('nombre', 'descripcion')
    search_fields = ('nombre',)
    filter_horizontal = ('mujeres', 'lugares')

admin.site.register(Mujer, MujerAdmin)
admin.site.register(Lugar, LugarAdmin)
admin.site.register(VisitedLugarRuta)
admin.site.register(Ruta, RutaAdmin)
admin.site.register(AreaInvestigacion)
```

La interfaz de listado muestra los campos configurados en `list_display` y proporciona funcionalidades de búsqueda basadas en los campos especificados en `search_fields`. La paginación automática mejora el rendimiento cuando se gestiona un gran volumen de registros.

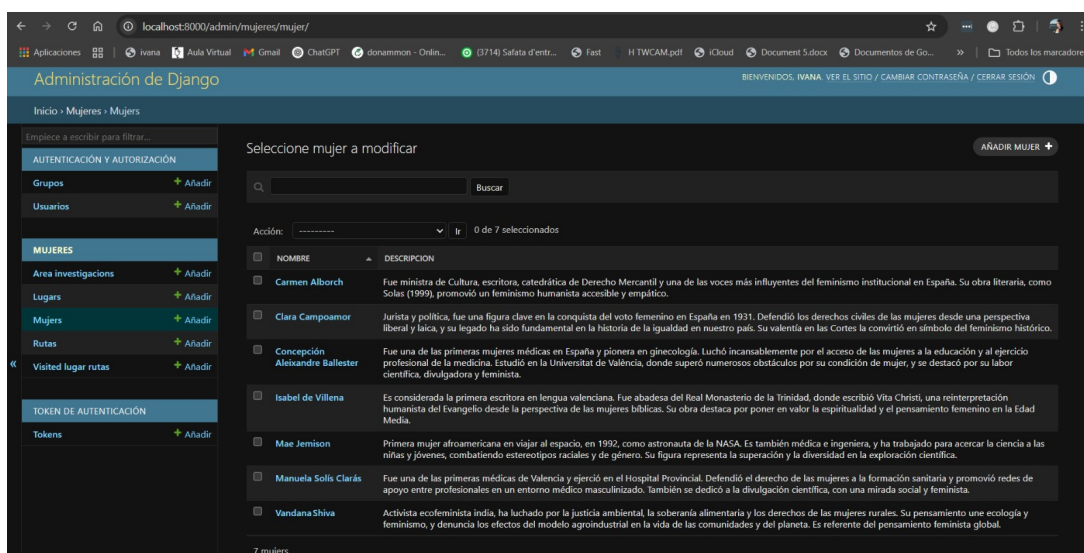


Figura 6.1: Vista de listado del modelo Mujer en Django Admin

El formulario de edición individual presenta una interfaz optimizada para la gestión de relaciones complejas. La configuración `filter_horizontal` en el campo `areas_investigacion` proporciona un widget de selección múltiple que facilita la asignación de áreas de investigación a cada mujer, permitiendo búsquedas y selecciones eficientes en relaciones `ManyToManyField` como se muestra en la figura 6.2:

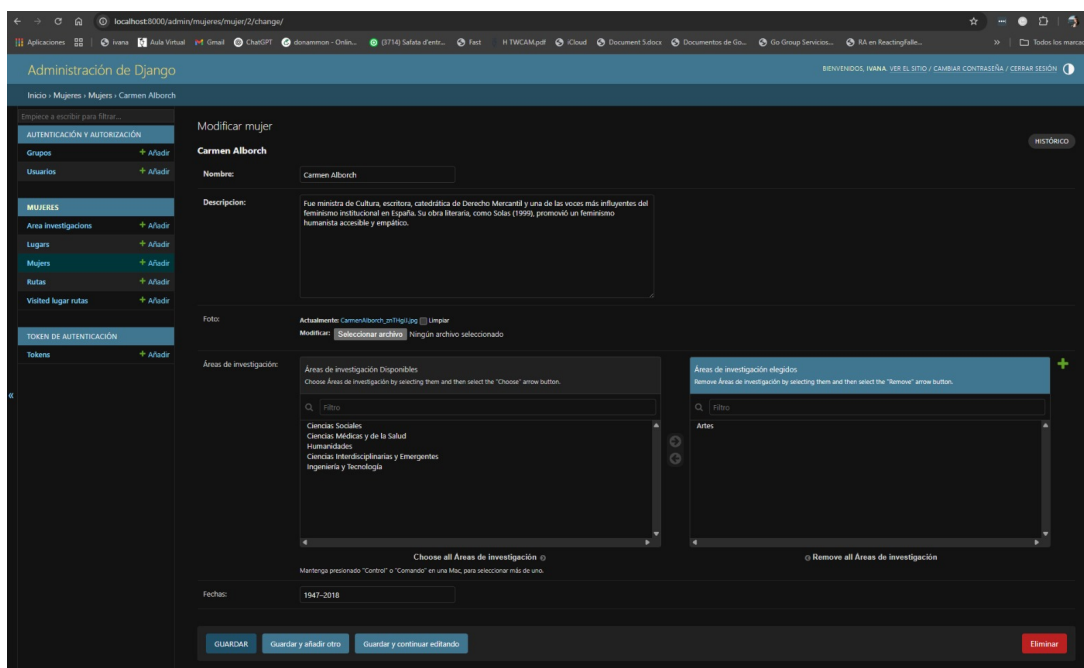


Figura 6.2: Formulario de edición para el modelo Mujer

6.1.11. Cambio de contraseña del usuario administrador de Django

Si se olvida la contraseña del usuario administrador o es necesario restablecerla, Django permite cambiarla fácilmente mediante un comando de gestión. Dado que el backend se ejecuta en un contenedor Docker, el proceso recomendado es el siguiente:¹

¹Nota: Si se ejecuta Django fuera de Docker, se usa el mismo comando pero directamente en la terminal.

Listado 6.24: Cambio de contraseña del usuario administrador en Docker

```
# Acceder al contenedor donde corre Django (normalmente llamado 'web')
docker compose exec web python manage.py changepassword ivana
```

El sistema solicitará la nueva contraseña de forma interactiva. El servicio se llama `web`, tal y como se indica en el archivo `docker-compose.yml`.

6.1.12. Gestión de migraciones y consistencia de la base de datos

Django utiliza un sistema de migraciones para versionar y aplicar cambios en el esquema de la base de datos. Cada vez que se modifica un modelo, se genera una nueva migración que puede ser aplicada en cualquier entorno, garantizando la coherencia entre desarrollo, pruebas y producción.

En este proyecto, el campo `areas_investigacion` del modelo `Mujer` evolucionó desde `ArrayField` hacia `ManyToManyField` para mejorar la gestión administrativa del sistema.

Listado 6.25: Migración de `ArrayField` a `ManyToManyField`

```
# 0001_initial.py (Estado inicial con ArrayField)
migrations.CreateModel(
    name='AreaInvestigacion',
    fields=[
        ('id', models.BigAutoField(auto_created=True, primary_key=True)),
        ('nombre', models.CharField(max_length=100, unique=True)),
    ],
),
migrations.CreateModel(
    name='Mujer',
    fields=[
        ('areas_investigacion', django.contrib.postgres.fields.ArrayField(
            base_field=models.CharField(max_length=100), blank=True,
            default=list, verbose_name='Áreas de investigación')),
        # ... otros campos
    ],
),

# 0005_remove_mujer_areas_investigacion_and_more.py
# Migración a ManyToManyField
migrations.RemoveField(model_name='mujer', name='areas_investigacion'),
migrations.AddField(
    model_name='mujer',
    name='areas_investigacion',
    field=models.ManyToManyField(blank=True, related_name='mujeres',
                                to='mujeres.areainvestigacion'),
),
```

La decisión de migrar de `ArrayField` a `ManyToManyField` se basó principalmente en la mejora de la experiencia administrativa, ya que el widget `filter_horizontal` de Django Admin facilita significativamente la gestión de áreas de investigación mediante una interfaz de selección múltiple intuitiva. Adicionalmente, esta aproximación proporciona mayor flexibilidad para futuras extensiones del sistema y aprovecha las funcionalidades nativas

de Django para la gestión de relaciones, mientras que el sistema de serialización mantiene la compatibilidad con el frontend móvil convirtiendo automáticamente las relaciones a strings.

Nota: La migración de datos se realizó preservando la información existente, y en entornos de producción es fundamental planificar estas migraciones cuidadosamente para evitar pérdida de datos.

6.1.13. Integración con el frontend y gestión de CORS

Para permitir la comunicación entre el frontend (desarrollado en React Native) y el backend, se ha configurado el middleware `django-cors-headers`, que habilita las peticiones CORS (Cross-Origin Resource Sharing). Esto es fundamental para que la aplicación móvil pueda consumir la API REST sin restricciones de origen.

En el archivo `settings.py` se ha añadido la configuración necesaria:

Listado 6.26: Configuración de CORS en Django

```
INSTALLED_APPS = [
    # ...otras apps...
    'corsheaders',
    'rest_framework',
    'mujeres',
]

MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware',
    # ...otros middlewares...
]

CORS_ALLOW_ALL_ORIGINS = True # Para desarrollo; en producción se recomienda
                               restringir los orígenes permitidos
```

6.1.14. Integración y despliegue con Docker

El backend se ejecuta en un contenedor Docker. El archivo `docker-compose.yml` define los servicios necesarios, incluyendo la base de datos PostgreSQL y el servidor de la aplicación Django.

Listado 6.27: `docker-compose.yml`

```
version: '3.9'

services:
  db:
    image: postgis/postgis:14-3.3
    restart: always
    env_file: .env
    ports:
      - "5432:5432"
```

```

volumes:
  - postgres_data:/var/lib/postgresql/data

web:
  build: .
  command: sh -c "python manage.py migrate && python manage.py runserver
    0.0.0.0:8000"
  volumes:
    - ./backend:/app
  ports:
    - "8000:8000"
  depends_on:
    - db
  env_file: .env

volumes:
  postgres_data:

```

Esto permite levantar todo el entorno de desarrollo y pruebas con un solo comando, garantizando la coherencia entre entornos y facilitando la colaboración.

6.1.15. Pruebas automáticas

Para garantizar la calidad y robustez del backend, se han implementado pruebas automáticas utilizando el framework de testing de Django. Un ejemplo de test para el modelo *Mujer* es:

Listado 6.28: Test unitario para el modelo *Mujer*

```

from django.test import TestCase
from .models import Mujer, AreaInvestigacion

class MujerModelTest(TestCase):
    def test_creacion_mujer(self):
        # Crear áreas de investigación primero
        area_matematicas = AreaInvestigacion.objects.create(nombre='Matemáticas')
        area_computacion = AreaInvestigacion.objects.create(nombre='Computación')

        mujer = Mujer.objects.create(
            nombre='Ada Lovelace',
            descripcion='Pionera de la programación',
            foto='fotos/ada.jpg',
            fechas='1815-1852'
        )
        # Agregar áreas después de crear el objeto (ManyToMany)
        mujer.areas_investigacion.add(area_matematicas, area_computacion)

        self.assertEqual(mujer.nombre, 'Ada Lovelace')
        self.assertIn(area_computacion, mujer.areas_investigacion.all())

```

Listado 6.29: Test de API para el endpoint de mujeres

```

from rest_framework.test import APITestCase
from django.urls import reverse
from .models import Mujer, AreaInvestigacion
from django.contrib.auth.models import User
from rest_framework.authtoken.models import Token

class MujerAPITest(APITestCase):
    def setUp(self):
        self.user = User.objects.create_user(username='testuser', password='
            testpass')
        self.token = Token.objects.create(user=self.user)

    def test_listado_mujeres(self):
        # Crear área y mujer correctamente
        area = AreaInvestigacion.objects.create(nombre='Matemáticas')
        mujer = Mujer.objects.create(nombre='Ada', descripcion='Pionera')
        mujer.areas_investigacion.add(area)

        url = reverse('mujer-list')
        response = self.client.get(url)
        self.assertEqual(response.status_code, 200)
        self.assertEqual(response.data['count'], 1)

```

6.1.16. Internacionalización y localización

El backend está preparado para soportar varios idiomas, facilitando la internacionalización de la aplicación. En `settings.py` se configuran los parámetros de idioma y zona horaria:

Listado 6.30: Internacionalización en `settings.py`

```

LANGUAGE_CODE = 'es-es'
TIME_ZONE = 'Europe/Madrid'
USE_I18N = True
USE_TZ = True

```

6.1.17. Gestión de archivos estáticos

Para servir archivos estáticos (CSS, JS, imágenes no subidas por usuarios), se ha configurado la ruta correspondiente en `settings.py`:

Listado 6.31: Archivos estáticos en `settings.py`

```

STATIC_URL = '/static/'
STATIC_ROOT = os.path.join(BASE_DIR, 'static')

```

Durante el despliegue, se puede ejecutar el comando `python manage.py collectstatic` para recopilar todos los archivos estáticos en la carpeta definida.

```
PS C:\Users\isiva\Documents\tfg\donam-mon-backend\backend> python manage.py collectstatic
163 static files copied to 'C:\Users\isiva\Documents\tfg\donam-mon-backend\backend\backend\static'.
```

Figura 6.3: Recopilación de archivos estáticos con `collectstatic` en Django

6.1.18. Gestión de emails y notificaciones

Para permitir el envío de notificaciones por correo electrónico (por ejemplo, confirmación de registro o recuperación de contraseña), se ha configurado el backend para utilizar un servidor SMTP. En `settings.py` se definen los parámetros necesarios:

Listado 6.32: Configuración de email en `settings.py`

```
EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'
EMAIL_HOST = 'smtp.gmail.com'
EMAIL_PORT = 587
EMAIL_USE_TLS = True
EMAIL_HOST_USER = 'tu_email@gmail.com'
EMAIL_HOST_PASSWORD = 'tu_contraseña'
```

El envío de emails se realiza mediante la función `send_mail` de Django:

Listado 6.33: Ejemplo de envío de email

```
from django.core.mail import send_mail

send_mail(
    'Bienvenido a la app',
    'Gracias por registrarte.',
    'tu_email@gmail.com',
    [usuario.email],
    fail_silently=False,
)
```

6.1.19. Internacionalización y localización

El backend está preparado para soportar varios idiomas, facilitando la internacionalización de la aplicación. En `settings.py` se configuran los parámetros de idioma y zona horaria:

Listado 6.34: Internacionalización en `settings.py`

```
LANGUAGE_CODE = 'es-es'
TIME_ZONE = 'Europe/Madrid'
USE_I18N = True
USE_TZ = True
```

6.2. Implementación del frontend

6.2.1. Herramientas, frameworks y lenguajes utilizados

El frontend de la aplicación se ha desarrollado utilizando el framework **React Native** para asegurar la compatibilidad multiplataforma (Android, iOS y web). Se ha utilizado **Expo** para simplificar el desarrollo, pruebas y despliegue, así como para acceder a funcionalidades nativas como la cámara, la geolocalización y las notificaciones.

Herramientas y tecnologías:

- **React Native** (v0.79.2)
- **Expo** (v53.0.9)

Frameworks y librerías:

- **React Navigation** para la gestión de la navegación entre pantallas
- **@react-native-async-storage/async-storage** para almacenamiento local seguro
- **axios** para peticiones HTTP
- **expo-location** y **expo-notifications** para geolocalización y notificaciones push
- **react-native-maps** para mostrar mapas y marcadores
- **react-native-vector-icons** para iconografía
- **expo-camera** para escaneo de QR en rutas
- **@react-native-picker/picker** para selectores personalizados

6.2.2. Estructura del proyecto y organización del código

El código fuente del frontend se encuentra en la carpeta **donam-mon-frontend/my-app/**, que contiene los componentes, utilidades, estilos y recursos gráficos. La estructura principal es la siguiente:

Listado 6.35: Estructura principal del frontend

```
my-app/  
  App.js  
  apiConfig.js  
  index.js  
  assets/  
    # Imágenes y recursos gráficos  
    ...  
  Donammon/  
    Main.js  
    Mapa.js  
    Mujeres.js  
    PerfilUsuario.js  
    Detail.js  
    RutaDetail.js  
    HistorialLugares.js  
    HistorialRutas.js
```

```

    NotificacionProximidad.js
    AR.js
    Filtros.js
    FiltrosContext.js
    Login.js
    Register.js
    Welcome.js
    NavigationApp.js
    Rutas.js
    ...
  styles/
    styles.js
    styles-detail.js
    styles-filtros.js
    styles-perfil.js
    ...

```

App.js: Punto de entrada principal, configura el proveedor de filtros y la navegación.

Donammon/: Carpeta principal de componentes de pantalla y utilidades.

assets/: Imágenes y recursos gráficos usados en la interfaz.

6.2.3. Navegación y estructura de pantallas

La navegación principal se gestiona mediante un **Bottom Tab Navigator** definido en **Main.js**, que organiza la aplicación en pestañas: Mapa, Mujeres, Rutas, Realidad Aumentada (RA) y Perfil.

Listado 6.36: Definición de pestañas en Main.js

```

<Tab.Navigator
  screenOptions={{
    tabBarActiveTintColor: '#5f68c4',
    tabBarInactiveTintColor: '#000000',
    tabBarStyle: { backgroundColor: '#edebff' },
  }}
>
  <Tab.Screen
    name="Mapa"
    component={Mapa}
    initialParams={route.params}
    options={{
      headerShown: false,
      tabBarIcon: ({ color, size }) => (
        <Image source={require('../assets/mapa.png')} />
      ),
    }}
  />
  <Tab.Screen name="Mujeres" component={MujeresCombinadas} ... />
  <Tab.Screen name="Rutas" component={Rutas} ... />
  <Tab.Screen name="RA" component={AR} ... />
  <Tab.Screen name="Perfil" component={PerfilUsuario} ... />
</Tab.Navigator>

```


Cada pantalla principal se implementa como un componente independiente, facilitando la escalabilidad y el mantenimiento.

Además, la navegación global de la app se gestiona mediante un **Stack Navigator** definido en `NavigationApp.js`, que permite acceder a pantallas como el login, registro, detalle de lugar, historial de visitas o filtros, además de la pantalla principal con pestañas. El componente `App.js` monta el contexto global de filtros y notificaciones, y envuelve toda la navegación de la aplicación.

6.2.4. Gestión de usuarios y autenticación

La autenticación de usuarios se realiza mediante peticiones a la API del backend. El login y el registro almacenan el token de autenticación en `AsyncStorage` para su uso en futuras peticiones.

Listado 6.37: Ejemplo de login en `Login.js`

```
const validateLogin = async () => {
  try {
    const response = await fetch('http://192.168.1.44:8000/api/api-token-auth/', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ username, password }),
    });
    if (response.ok) {
      const data = await response.json();
      await AsyncStorage.setItem('token', data.token);
      navigation.navigate('Main');
    } else {
      Alert.alert('Error', 'Usuario o contraseña incorrectos');
    }
  } catch (error) {
    Alert.alert('Error', 'No se pudo conectar con el servidor');
  }
};
```

El perfil de usuario permite cambiar el nombre de usuario y cerrar sesión, eliminando el token almacenado.

6.2.5. Visualización de mujeres y lugares

El archivo `Mujeres.js` permite mostrar una lista de lugares destacados asociados a mujeres relevantes, permitiendo filtrar por área, visitados y búsqueda por nombre.

Listado 6.38: Filtrado y renderizado de lugares

```
const filteredLugares = allLugares
  .filter(lugar => selectedSection === 'Todas' || lugar.area ===
    selectedSection)
  .filter(lugar => selectedVisited === 'Todas' || (selectedVisited === '
    Visitadas' ? lugar.visited : !lugar.visited))
  .filter(lugar => !searchTerm || lugar.nombre.toLowerCase().includes(
    searchTerm.toLowerCase()));
```

Cada tarjeta de lugar muestra la imagen, nombre, área, distancia y estado de visita, y permite acceder al detalle.

6.2.6. Detalle de lugar y gestión de visitas

El archivo `Detail.js` permite mostrar la información completa de un lugar, incluyendo la mujer asociada, descripción, dirección, fechas, imagen y estado de visita. Permite marcar el lugar como visitado o no visitado, sincronizando el estado con el backend mediante una petición HTTP autenticada. Además, la pantalla ofrece funcionalidades avanzadas para mejorar la experiencia del usuario:

- **Compartir información del lugar:** El usuario puede compartir el lugar y su imagen a través de las opciones nativas del dispositivo. Se genera un mensaje con los datos relevantes y, si existe, se adjunta la imagen descargada temporalmente.
- **Visualización de imágenes ampliadas:** Al pulsar sobre la imagen de la mujer o del lugar, se muestra la imagen en grande en un modal.
- **Visualización de información enriquecida:** Se muestran áreas de investigación como chips, fechas, ámbito, descripción, dirección, un mapa con la localización exacta y la distancia al usuario.

A continuación se muestra un ejemplo de la función para marcar un lugar como visitado o no visitado:

Listado 6.39: Marcar como visitado

```
const toggleVisited = async () => {
  const updatedLugar = { ...lugar, visited: !lugar.visited };
  navigation.setParams({ lugar: updatedLugar });
  const token = await AsyncStorage.getItem('token');
  if (!token) return;
  if (!lugar.visited) {
    // Marcar como visitado
    try {
      const response = await fetch('http://192.168.1.44:8000/api/visit-
        lugar/', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json', 'Authorization': '
          Token ${token}' },
        body: JSON.stringify({ lugar_id: lugar.id }),
      });
      const data = await response.json().catch(() => ({}));
      if (!response.ok) {
        throw new Error(`Error ${response.status}: ${JSON.stringify(data)}
          `);
      }
    } catch (error) {
      console.error('Error registrando visita:', error);
    }
  } else {
    // Marcar como no visitado (eliminar del historial)
    try {
      await fetch('http://192.168.1.44:8000/api/visited-lugares/', {
```

```

        method: 'PUT',
        headers: { 'Content-Type': 'application/json', 'Authorization': 'Token ${token}' },
        body: JSON.stringify({ lugar_id: lugar.id }),
    });
} catch (error) {
    console.error('Error eliminando visita:', error);
}
}
};

```

Y la función para compartir la información del lugar:

Listado 6.40: Función para compartir la información del lugar

```

const compartirLugar = async () => {
    try {
        let message = "Echa un vistazo a este lugar de la aplicación Dona'm Món!\n\n";
        message += `Nombre del lugar: ${lugar.nombre}\n`;
        if (lugar.mujer_nombre) message += `Mujer destacada: ${lugar.mujer_nombre}\n`;
        if (lugar.mujer_descripcion) message += `Sobre ella: ${lugar.mujer_descripcion}\n`;
        if (lugar.direccion) message += `Dirección: ${lugar.direccion}\n`;
        if (lugar.descripcion) message += `Descripción: ${lugar.descripcion}\n`;
        let options = { message };
        if (lugar.foto) {
            // Descargar la imagen a un archivo temporal
            const fileUri = FileSystem.cacheDirectory + 'lugar.jpg';
            const downloadRes = await FileSystem.downloadAsync(lugar.foto, fileUri);
            if (downloadRes.status === 200) {
                options = { ...options, url: downloadRes.uri };
            }
        }
        await Share.share(options);
    } catch (error) {
        console.error('Error al compartir el lugar:', error);
    }
};

```

6.2.7. Mapas y geolocalización

El archivo `Mapa.js` utiliza `react-native-maps` para mostrar todos los lugares en un mapa, con iconos personalizados y colores según el estado de visita. Permite filtrar lugares y ver detalles al pulsar sobre un marcador.

La aplicación implementa cálculos de distancia precisos mediante la fórmula de Haversine[58], que permite determinar la distancia entre la ubicación del usuario y cada lugar en la superficie terrestre:

Listado 6.41: Cálculo de distancia con fórmula de Haversine

```
const getDistance = (lat1, lon1, lat2, lon2) => {
  if (!lat1 || !lon1 || !lat2 || !lon2) return null;
  const toRad = x => x * Math.PI / 180;
  const R = 6371; // Radio de la Tierra en kilómetros
  const dLat = toRad(lat2 - lat1);
  const dLon = toRad(lon2 - lon1);
  const a = Math.sin(dLat / 2) * Math.sin(dLat / 2) +
    Math.cos(toRad(lat1)) * Math.cos(toRad(lat2)) *
    Math.sin(dLon / 2) * Math.sin(dLon / 2);
  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
  return R * c;
};
```

Esta implementación considera la curvatura de la Tierra para proporcionar distancias precisas en kilómetros, que se muestran tanto en los marcadores del mapa como en las tarjetas de lugares para ayudar al usuario a identificar los sitios más cercanos.

La implementación de los marcadores en el mapa utiliza un código de colores diferenciado: los lugares visitados se muestran en verde (#43a047) mientras que los no visitados aparecen en azul (#5f68c4). Cada marcador incluye un Callout que muestra información detallada del lugar, la mujer asociada y la distancia calculada:

Listado 6.42: Marcadores en el mapa con información de distancia

```
<MapView>
  {filteredLugares.map((lugar) => {
    let iconColor = '#5f68c4';
    if (lugar.visited) iconColor = '#43a047';
    return (
      <Marker
        key={lugar.id}
        coordinate={{
          latitude: lugar.latitud,
          longitude: lugar.longitud
        }}
      >
        <View style={styles.iconContainer}>
          <Icon name="gender-female" size={30} color={iconColor} />
        </View>
        <Callout tooltip>
          <ScrollView style={styles.callout}>
            <Text style={styles.calloutTitle}>{lugar.mujer_nombre}</Text>
            <Text style={{ fontWeight: 'bold' }}>{lugar.nombre}</Text>
            {lugar.distance !== undefined && (
              <Text style={{ color: '#bc5880' }}>
                Distancia: {lugar.distance?.toFixed(2)} km
              </Text>
            )}
          </ScrollView>
        </Callout>
      </Marker>
    );
  })
};
```

```
    }}}
  </MapView>
```

La ubicación del usuario se obtiene mediante `expo-location` con permisos de geolocalización, y las distancias se actualizan dinámicamente cada vez que el usuario cambia de ubicación o se refrescan los datos.

6.2.8. Gestión de rutas temáticas

La pantalla `Rutas.js` y `RutaDetail.js` permiten visualizar rutas temáticas, ver los lugares que las componen y marcar cada lugar como visitado mediante escaneo de QR. El progreso de la ruta se muestra visualmente y se puede reiniciar la ruta si se desea.

Listado 6.43: Escaneo de QR y registro de visita

```
const handleBarcodeScanned = async ({ data }) => {
  const lugar = ruta.lugares.find(l => l.nombre.toLowerCase() === data.
    toLowerCase());
  if (lugar) {
    await fetch('http://192.168.1.44:8000/api/visit-lugar-ruta/', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json', 'Authorization': '
        Token ${token}' },
      body: JSON.stringify({ ruta_id: ruta.id, lugar_id: lugar.id }),
    });
    setScanMsg('Lugar de ruta marcado como visitado!');
    await fetchVisitedRuta();
  } else {
    setScanMsg('QR no corresponde a ningun lugar de la ruta.');
```

6.2.9. Historial de lugares y rutas visitadas

Las pantallas `HistorialLugares.js` y `HistorialRutas.js` muestran el historial de lugares y rutas visitadas por el usuario, permitiendo borrar el historial si se desea.

Listado 6.44: Borrado de historial de rutas

```
const handleClearHistory = async () => {
  const token = await AsyncStorage.getItem('token');
  for (const ruta of rutasCompletadas) {
    await fetch('http://192.168.1.44:8000/api/visited-lugares-ruta/?ruta_id=$
      {ruta.id}', {
        method: 'DELETE',
        headers: { 'Authorization': 'Token ${token}' },
      });
  }
  setRutasCompletadas([]);
  Alert.alert('Éxito', 'Historial de rutas completadas borrado.');
```

6.2.10. Sistema de filtros avanzado y gestión de estado

La aplicación implementa un sistema de filtros avanzado mediante el contexto `FiltrosContext.js`, que permite mantener el estado de los filtros entre diferentes pantallas y componentes.

Listado 6.45: Contexto global de filtros

```
export const FiltrosProvider = ({ children }) => {
  const [selectedSection, setSelectedSection] = useState('Todas');
  const [selectedVisited, setSelectedVisited] = useState('Todas');
  const [searchTerm, setSearchTerm] = useState('');
  const [sections, setSections] = useState(['Todas']);

  return (
    <FiltrosContext.Provider value={{
      selectedSection, setSelectedSection,
      selectedVisited, setSelectedVisited,
      searchTerm, setSearchTerm,
      sections, setSections
    }}>
      {children}
    </FiltrosContext.Provider>
  );
};
```

Este contexto permite que los filtros aplicados en la pantalla de `MujeresCombinadas` se mantengan al navegar al `Mapa`, proporcionando una experiencia de usuario coherente.

6.2.11. Sincronización de estados de visita

El sistema implementa una gestión inteligente de los estados de visita que se sincroniza automáticamente entre el almacenamiento local y el backend:

Listado 6.46: Sincronización de visitas

```
const fetchVisitedAndSet = async (lugares) => {
  try {
    const token = await AsyncStorage.getItem('token');
    if (!token) {
      setMujeres(lugares.map(l => ({ ...l, visited: false })));
      return;
    }
    const response = await fetch('http://192.168.1.44:8000/api/visited-
      lugares/', {
      method: 'GET',
      headers: { 'Authorization': 'Token ${token}' },
    });
    let visitedIds = [];
    if (response.ok) {
      const data = await response.json();
      visitedIds = data.map(v => v.lugar.id);
    }
    setMujeres(lugares.map(l => ({ ...l, visited: visitedIds.includes(l.id)
    })));
  }
};
```

```
    } catch (error) {  
      setMujeres(lugares.map(l => ({ ...l, visited: false })));  
    }  
  };  
};
```

6.2.12. Notificaciones y proximidad

La app utiliza `expo-notifications` y `expo-location` para enviar notificaciones cuando el usuario se acerca a un lugar destacado. El componente `NotificacionProximidad.js` gestiona esta lógica, comprobando la distancia a los lugares y mostrando alertas contextuales.

Listado 6.47: Notificación de proximidad

```
if (dist !== null && dist <= 3 && !notifiedIds.current.has(lugar.id)) {  
  await Notifications.scheduleNotificationAsync({  
    content: {  
      title: 'Estas cerca de un lugar destacado!',  
      body: 'Te encuentras a ${dist.toFixed(2)} km de ${lugar.nombre} (${lugar.mujer_nombre})',  
    },  
    trigger: null,  
  });  
  notifiedIds.current.add(lugar.id);  
}
```

6.2.13. Realidad aumentada

La pantalla `AR.js` permite acceder a experiencias de realidad aumentada asociadas a ciertos lugares, abriendo el visor web correspondiente si el usuario está cerca del lugar.

Listado 6.48: Acceso a AR

```
if (distance <= 3) {  
  openURL(lugar.ar_url);  
}
```

6.2.14. Gestión de estilos y diseño

Los estilos se gestionan en archivos como `styles.js` y `styles-perfil.js`, utilizando `StyleSheet` de React Native para mantener la coherencia visual y facilitar la personalización.

Listado 6.49: Ejemplo de estilos en `styles.js`

```
const styles = StyleSheet.create({  
  container: {  
    flex: 1,  
    backgroundColor: '#f9efe8',  
    alignItems: 'center',  
    justifyContent: 'center',  
  },  
  filterButton: {
```

```
        backgroundColor: '#7196e4',
        paddingVertical: 0,
        paddingHorizontal: 30,
        borderRadius: 10,
        marginTop: 23,
        alignSelf: 'center',
        width: width + 20,
        height: 39,
        alignItems: 'center',
    },
    // otros estilos...
});
```

6.2.15. Gestión de almacenamiento local

Se utiliza `AsyncStorage` para guardar el token de usuario, preferencias y otros datos persistentes, asegurando que la sesión se mantenga entre reinicios de la app.

Listado 6.50: Uso de `AsyncStorage`

```
await AsyncStorage.setItem('token', data.token);
const token = await AsyncStorage.getItem('token');
```

6.2.16. Integración con la API y manejo de errores

Todas las peticiones a la API del backend se realizan mediante `fetch` o `axios`, gestionando los errores y mostrando mensajes claros al usuario en caso de fallo.

Listado 6.51: Manejo de errores en peticiones

```
try {
    const response = await fetch('http://192.168.1.44:8000/api/visited-lugares/'
        , { /* ... */ });
    if (!response.ok) throw new Error('Error al cargar lugares visitados');
    const data = await response.json();
    setVisitedCount(data.length);
} catch (error) {
    Alert.alert('Error', 'No se pudo cargar el historial');
}
```

6.2.17. Pruebas y depuración

Durante el desarrollo, se han utilizado logs en consola y mensajes de alerta para depurar la lógica de la app y asegurar el correcto funcionamiento de todas las funcionalidades.

Listado 6.52: Ejemplo de logs y alertas

```
console.log('Detalle lugar:', lugar);
Alert.alert('Éxito', 'Nombre de usuario actualizado');
```


6.3. Implementación de Realidad Aumentada

6.3.1. Herramientas, frameworks y tecnologías utilizadas

Para la implementación de las experiencias de realidad aumentada se ha optado por una solución web basada en **A-Frame** con **AR.js**, que permite crear experiencias inmersivas accesibles desde cualquier navegador web moderno sin necesidad de aplicaciones nativas específicas.

Las principales tecnologías utilizadas son:

- **A-Frame 1.7.0** - Framework web para realidad virtual y aumentada
- **AR.js** - Librería JavaScript que añade capacidades de realidad aumentada a A-Frame
- **WebGL** - Para renderizado 3D en tiempo real
- **Troika Text** - Renderizado avanzado de texto 3D en WebGL para A-Frame
- **Vercel** - Plataforma de hosting para despliegue de experiencias AR
- **GLB** - Formato de modelos 3D optimizados para web
- **MP3** - Audio inmersivo y música ambiental

6.3.2. Arquitectura y estructura del proyecto AR

Cada lugar destacado en la aplicación tiene asociada una experiencia de realidad aumentada independiente, desplegada como una aplicación web estática en Vercel. Esta arquitectura descentralizada proporciona escalabilidad, rendimiento optimizado y facilita el mantenimiento, ya que cada experiencia puede actualizarse de forma independiente.

La estructura típica de una experiencia AR es:

Listado 6.53: Estructura de una experiencia AR

```
experiencia-ar/  
  index.html # Aplicación principal AR  
  mundo.glb # Modelo 3D del logo (tierra interactiva)  
  lupa.glb # Modelo 3D secundario (símbolo de Venus)  
  mujer.glb # Modelo 3D principal (personaje histórico)  
  carmen_audio.mp3 # Audio narrativo específico  
  musica_de_fondo_suave.mp3 # Música ambiental  
  Roboto-Regular.ttf # Fuente tipográfica utilizada
```

Cada experiencia AR se aloja en una URL única, por ejemplo: <https://ivam.vercel.app/>. Esta URL se asocia a un lugar específico en la base de datos del backend, como hemos mencionado anteriormente.

6.3.3. Diseño de la experiencia interactiva

Interfaz inicial y elementos de navegación

La experiencia AR comienza mostrando un logo interactivo compuesto por dos elementos 3D: un modelo de la Tierra que rota continuamente y un símbolo de Venus estático

(representando una lupa). Ambos elementos están agrupados en un contenedor jerárquico que permite la interacción conjunta:

Listado 6.54: Contenedor principal de la interfaz AR

```
<!-- Contenedor padre para mundo + lupa -->
<a-entity id="logoContainer" look-at="target: camera"
  position="0 1.75 -5.5" rotation="0 90 0"
  class="clickable" visible="true"
  animation__click="property: scale; to: 0 0 0; startEvents: click; dur:
    500"
  animation__hide="property: visible; to: false; startEvents: click;
    delay: 500">

  <!-- Modelo de mundo -->
  <a-entity id="logoModel" gltf-model="mundo.glb"
    position="0 0 0" scale="0.6 0.6 0.6" logo-click-handler
    animation__spin="property: rotation; to: 0 360 0; dur: 10000; easing:
      linear; loop: true">
  </a-entity>

  <!-- Modelo de lupa -->
  <a-entity id="lupaModel" gltf-model="lupa.glb"
    position="-0.69968 0.30818 0.37761" rotation="0 270 0"
    scale="0.3 0.3 0.3" logo-click-handler>
  </a-entity>
</a-entity>
```

Al hacer click en cualquiera de los dos elementos, se inicia una secuencia de animación que hace desaparecer el logo mediante una reducción de escala, dando paso al personaje histórico principal.

Renderizado del personaje principal

El modelo 3D del personaje histórico utiliza un conjunto de animaciones simultáneas para crear una experiencia inmersiva y natural:

Listado 6.55: Configuración del modelo principal

```
<!-- Personaje histórico -->
<a-entity id="mujerModel" gltf-model="mujer.glb" mujer-click-handler
  look-at="target: camera" position="0 1.75 -8"
  scale="0 0 0" visible="false" material="transparent: true; opacity: 0"
  animation__visible="property: visible; to: true; startEvents:
    makeVisible"
  animation__show="property: scale; to: 1 1 1; startEvents: showMujer;
    dur: 1500"
  animation__move="property: position; to: 0 1.75 -5.5; startEvents:
    showMujer; dur: 1500"
  animation__opacity="property: components.material.material.opacity; to:
    1; startEvents: showMujer; dur: 1500"
  animation__spin="property: rotation; to: 0 360 0; dur: 17000; easing:
    linear; loop: true">
```

```
</a-entity>
```

La secuencia de aparición incluye:

- **Aparición gradual:** Transición de invisible a visible mediante escalado y ajuste de opacidad (1.5 segundos)
- **Movimiento espacial:** Acercamiento desde la posición inicial hacia el usuario
- **Rotación continua:** Giro lento que permite observar el modelo desde todos los ángulos (17 segundos por vuelta)
- **Look-at dinámico:** Orientación automática hacia la cámara del usuario

Sistema de transiciones y coordinación temporal

La transición entre la interfaz inicial y el contenido principal se gestiona mediante un sistema de eventos personalizado que coordina múltiples animaciones de forma secuencial:

Listado 6.56: Gestor de transiciones AR

```
AFRAME.registerComponent('logo-click-handler', {
  init: function () {
    this.el.addEventListener('click', function () {
      // Iniciar animación de desaparición del logo
      document.querySelector('#logoContainer').emit('click');

      setTimeout(function () {
        // Hacer visible el modelo de la mujer
        const mujerModel = document.querySelector('#mujerModel');
        mujerModel.emit('makeVisible');

        setTimeout(() => {
          // Activar animaciones principales y contenido informativo
          mujerModel.emit('showMujer');
          document.querySelector('#textoCarmen').emit('showMujer');
          document.querySelector('#bioCarmen').emit('showMujer');

          // Gestión sincronizada del audio
          const fondo = document.getElementById('musicaFondo');
          const voz = document.getElementById('audioCarmenAsset');

          if (voz && fondo) {
            fondo.volume = 0.01; // Reducir música de fondo
            voz.volume = 1.0; // Audio principal al máximo
            voz.play().catch(e => console.warn("Error al reproducir audio:", e));
            voz.onended = () => { fondo.volume = 0.02; }; // Restaurar volumen
          }
        }, 100);
      }, 600);
    });
  }
});
```

Este sistema implementa una **temporización precisa** que controla cada fase de la experiencia, asegurando transiciones fluidas entre estados.

Arquitectura de audio multicapa

En el código del apartado anterior, también se muestra la **gestión de audio inteligente** que reduce automáticamente el volumen de la música ambiental durante las narraciones, restaurándolo progresivamente al finalizar. Para la música ambiental se ha escogido la “Canción sin miedo” de Vivir Quintana [71] y las narraciones son audios propios sobre las mujeres y sus lugares asociados, que se reproducen al mostrarse el modelo de la mujer.

El sistema presenta las siguientes características:

- **Música ambiental:** Reproducción continua a 2 % del volumen máximo
- **Audio narrativo:** Contenido educativo específico sobre el personaje histórico
- **Ducking automático:** Reducción del volumen ambiental al 1 % durante narraciones
- **Recuperación inteligente:** Restauración automática del volumen original al finalizar

Listado 6.57: Configuración de assets de audio

```
<a-assets>
  <audio id="audioCarmenAsset" src="carmen_audio.mp3" preload="auto"></audio>
  <audio id="musicaFondo" src="musica_de_fondo_suave.mp3" preload="auto" loop></
    audio>
</a-assets>
```

Compatibilidad con políticas de navegador

Para cumplir con las restricciones de autoplay de los navegadores modernos, se implementa un sistema de activación progresiva que gestiona la reproducción automática:

Listado 6.58: Gestión de autoplay de audio

```
AFRAME.registerComponent('background-music', {
  init: function () {
    const musicaFondo = document.getElementById('musicaFondo');
    if (!musicaFondo) return;

    musicaFondo.volume = 0.02;

    const tryPlay = () => {
      musicaFondo.play()
        .then(() => {
          document.body.removeEventListener('click', tryPlay);
        })
        .catch(() => {
          document.body.addEventListener('click', tryPlay);
        });
    };
  }
});
```

```

    });
  };

  this.el.addEventListener('loaded', () => {
    setTimeout(tryPlay, 500);
  });
}
});

```

Este mecanismo intenta reproducir el audio automáticamente al cargar la escena, pero establece un respaldo que activa la reproducción con la primera interacción del usuario si el navegador bloquea el autoplay inicial.

6.3.4. Gestión de la tipografía en textos 3D

Durante la implementación de los elementos textuales en la escena 3D, se identificó una limitación en el componente nativo `<a-text>` de A-Frame, el cual no soporta caracteres extendidos como acentos o la letra “ñ” debido a que utiliza fuentes basadas en texturas MSDF con un conjunto restringido de caracteres. Esta restricción impedía mostrar correctamente textos en castellano, incluso cuando la codificación del documento se establecía en UTF-8 [72].

Para solventar este problema, se incorporó el componente `aframe-troika-text`, que permite renderizar tipografía en tiempo real a partir de fuentes en formatos estándar (TTF o WOFF) y ofrece soporte completo para caracteres Unicode. Esto se consigue añadiendo la librería mediante la siguiente instrucción en el documento HTML:

Listado 6.59: Inclusión del componente `aframe-troika-text`.

```

<script src="https://unpkg.com/aframe-troika-text/dist/aframe-troika-text.min.js"
"></script>

```

Posteriormente, se definió un panel informativo en la escena con un elemento `<a-entity>` que emplea el componente `troika-text`, configurando propiedades como el texto, la fuente personalizada (en este caso, *Roboto* en formato TTF), el color, el tamaño y el ancho máximo. El siguiente fragmento de código muestra la implementación:

Listado 6.60: Uso de `aframe-troika-text` para mostrar texto con acentos.

```

<!-- Panel informativo -->
<a-entity id="infoPanel"
  geometry="primitive: plane; height: 1; width: 2"
  material="color: #f8d4e6; opacity: 0.8"
  position="0 -0.036 -3.99313"
  visible="false">
  <a-entity troika-text="value: Su compromiso con la igualdad y la cultura la
    llevó a dirigir el Instituto Valenciano de Arte Moderno (IVAM) entre 1988
    y 1993, etapa en la que el museo se consolidó como referente nacional e
    internacional del arte contemporáneo. Desde allí impulsó una programación
    abierta, crítica y sensible al papel de las mujeres en la creación artí
    stica.;
    font: ./fonts/Roboto-Regular.ttf;
    color: #bc5880;
    fontSize: 0.09;

```

```
    maxWidth: 1.8;  
    align: left"  
    position="0 0.060 0.136"></a-entity>  
</a-entity>
```

El fragmento anterior, corresponde al panel interactivo que proporciona información detallada sobre la mujer (siendo el mismo contenido que se escucha en los audios correspondientes). Este panel se activa pulsando sobre el modelo 3D de la mujer, implementando un sistema de toggle que permite mostrar u ocultar la información extendida según las necesidades del usuario.

6.3.5. Formato y optimización de modelos 3D

Todos los modelos empleados en la experiencia de realidad aumentada se han utilizado en formato `.glb`. Este formato, perteneciente al estándar glTF 2.0, es uno de los soportados por A-Frame mediante el componente `gltf-model` [73]. A-Frame también permite el uso de otros formatos como `.obj` (a través del componente `obj-model`) y, mediante loaders de *three.js*, formatos adicionales como `.ply`, `.fbx` o JSON. Sin embargo, la documentación oficial recomienda el uso de glTF/GLB por sus ventajas: encapsula geometría, texturas, materiales y animaciones en un único archivo binario, optimizado para transmisión web, reduciendo el número de peticiones HTTP y facilitando la carga en dispositivos móviles.

Obtención de modelos

Los modelos utilizados se han conseguido de dos formas:

1. Descargando desde repositorios especializados como Cults3D² y Thingiverse³, que ofrecen modelos en diferentes formatos posteriormente convertidos a `.glb`.
2. Generándolos mediante inteligencia artificial usando la herramienta Meshy⁴. Este proceso consiste en subir una imagen, obtener cuatro modelos generados automáticamente y seleccionar el más adecuado. Posteriormente, Meshy crea una textura para el modelo elegido (como se observa en la figuras 6.4, 6.5 y 6.6).

Es importante destacar que el uso de modelos generados por IA no se plantea como una solución final para la aplicación, sino como una aproximación visual preliminar de cómo podrían lucir los personajes históricos.

²https://cults3d.com/en/3d-model/art/vandana-shiva?srsltid=AfmBOoo2OFFWztX_LpR_AyTC21vPPs9T8Xd-n4IRsIl4syxbvKVhqbF3

³<https://www.thingiverse.com/thing:5899901>

⁴<https://www.meshy.ai>

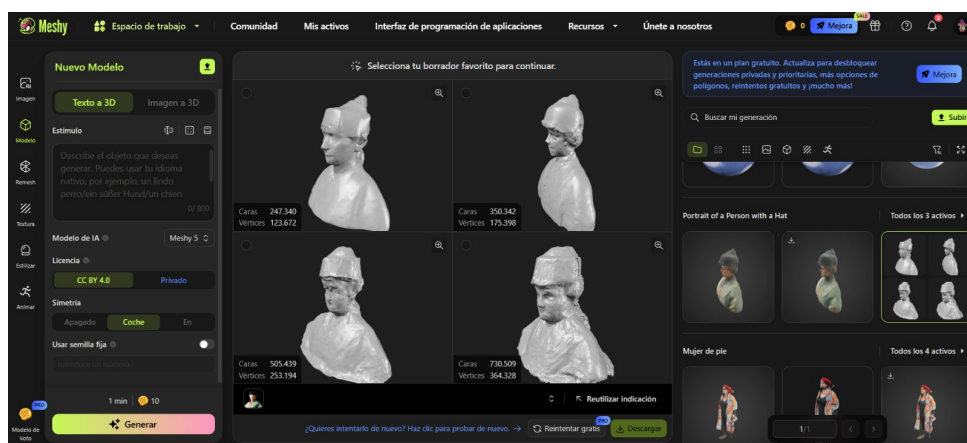


Figura 6.4: Cuatro modelos generados automáticamente por Meshy a partir de una imagen de entrada.

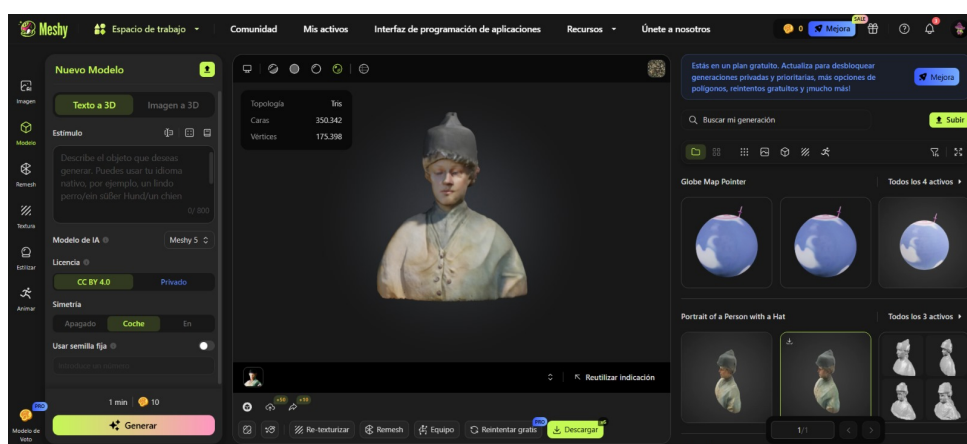


Figura 6.5: Modelo seleccionado de los cuatro generados, ya con la textura aplicada por Meshy.



Figura 6.6: Imagen original de Manuela Solís que se utiliza como entrada en Meshy para la generación automática de modelos 3D.

Optimización y limitaciones técnicas

Durante el desarrollo se presentaron varias restricciones de tamaño. La herramienta Meshy solo admite modelos de hasta 50 MB, lo que obligó a realizar procesos de optimización y reducción de malla (re-vectorización) antes de subirlos para generar las texturas.

Asimismo, la plataforma de despliegue Vercel establece un límite máximo recomendado de 10 MB por archivo para proyectos conectados por repositorios GitHub [74], como es nuestro caso. Esto exigió la aplicación de técnicas adicionales de compresión y simplificación para asegurar la compatibilidad.

Finalmente, en algunos casos los modelos descargados desde repositorios carecían de texturas, por lo que se recurrió a Meshy para generarlas. Esto implicó un flujo de trabajo iterativo que combinaba conversión a `.glb`, reducción de tamaño y texturizado mediante IA.

6.3.6. Implementación del sistema de tracking en AR

Como se ha mencionado anteriormente, la experiencia de realidad aumentada se implementó usando **A-Frame** junto con la librería **AR.js**. Esta combinación permite integrar modelos 3D en entornos web mediante tecnologías abiertas como WebGL y WebRTC, sin requerir aplicaciones nativas. No obstante, presenta una limitación fundamental: **AR.js no incorpora seguimiento posicional basado en el entorno (SLAM)**, sino que se apoya en **marcadores visuales** o en coordenadas GPS para determinar la posición del contenido en la escena. Como consecuencia, no es posible anclar los objetos 3D a superficies reales ni proporcionar una experiencia completamente inmersiva con movimiento libre.

Una solución a esta carencia es el uso de la **API WebXR**, que ofrece soporte nativo para AR con capacidades de seguimiento posicional en navegadores modernos. Sin embargo, esta alternativa presenta un problema de compatibilidad relevante: actualmente, **Safari en iOS no soporta la API WebXR**, lo que imposibilita su uso en dispositivos Apple [75, 76]. Este aspecto limita su adopción en escenarios donde se busca garantizar la disponibilidad multiplataforma.

Ante esta restricción, se diseñó un mecanismo alternativo que simula el comportamiento del anclaje espacial, como un *tracking visual simulado*. Este enfoque consiste en mantener el modelo 3D en una posición fija dentro de la escena virtual, aplicando transformaciones relativas a la cámara. De este modo, al rotar el dispositivo, el objeto parece permanecer en un punto del espacio, generando la **sensación de anclaje** aunque en realidad no se esté realizando un seguimiento del entorno. Si bien este método no sustituye al SLAM real, permite mejorar la percepción de estabilidad en la experiencia y reduce la disonancia visual en comparación con un objeto flotante sin referencia.

6.3.7. Despliegue y distribución

Hosting en Vercel

Cada experiencia AR se despliega independientemente en Vercel, proporcionando:

- **CDN global:** Acceso rápido desde cualquier ubicación
- **HTTPS automático:** Requisito para acceso a cámara y sensores
- **Escalabilidad automática:** Se adapta al número de usuarios concurrentes
- **Versionado:** Permite actualizaciones sin interrupciones

Ventajas de la arquitectura distribuida

Escalabilidad:

- Cada experiencia puede escalarse independientemente
- Recursos distribuidos evitan cuellos de botella

Mantenimiento:

- Actualizaciones de contenido sin afectar el backend principal
- Desarrollo paralelo de múltiples experiencias

Rendimiento:

- Carga únicamente los recursos necesarios para cada experiencia
- Optimización específica para cada contexto histórico

Flexibilidad:

- Permite diferentes tipos de experiencias AR por lugar
- Facilita la experimentación con nuevas tecnologías

6.3.8. Consideraciones técnicas y limitaciones

Compatibilidad de dispositivos

Requisitos mínimos:

- Navegador web moderno con soporte WebGL
- Acceso a cámara (para AR tracking)
- Procesador capaz de renderizado 3D en tiempo real

Dispositivos soportados:

- Smartphones Android (Chrome, Firefox)
- iPhones (Safari, Chrome)
- Tablets con navegadores modernos
- Ordenadores con webcam

Capítulo 7

Pruebas

7.1. Pruebas funcionales

Las pruebas funcionales tienen como propósito verificar que todas las funcionalidades implementadas en el sistema *DONA'm MÓN* operan conforme a los requisitos establecidos durante la fase de análisis. Se han ejecutado pruebas exhaustivas que abarcan la totalidad de los casos de uso definidos, tanto para el backend API REST como para la aplicación móvil y las funcionalidades de realidad aumentada.

La siguiente tabla presenta los casos de prueba realizados, especificando para cada uno la funcionalidad evaluada, el procedimiento seguido, el comportamiento esperado y el resultado obtenido durante la ejecución de las pruebas.

Funcionalidad	Descripción del Procedimiento	Resultado Esperado	Resultado Obtenido
F1. Visualizar un mapa interactivo con los puntos de interés distribuidos por Valencia.	Abrir la app y acceder a la pestaña de mapa para visualizar los puntos geolocalizados.	El mapa muestra correctamente todos los puntos de interés en Valencia.	✓ El mapa se carga y aparecen todos los puntos de interés distribuidos por la ciudad, permitiendo su exploración visual.
F2. Detectar en tiempo real la ubicación actual del usuario.	Activar la localización y comprobar que la app detecta la posición actual.	La ubicación del usuario se detecta y actualiza en tiempo real.	✓ El icono de usuario se posiciona correctamente en el mapa y se actualiza al moverse.
F3. Calcular y mostrar la distancia a cada punto desde la ubicación actual del usuario.	Navegar por el mapa y observar las distancias mostradas junto a cada punto.	Se muestran las distancias precisas desde la ubicación del usuario a cada punto.	✓ Junto a cada punto aparece la distancia en kilómetros, que cambia dinámicamente al desplazarse.

F4. Notificar al usuario cuando se acerque a un punto no visitado.	Acercarse físicamente a un punto de interés no visitado.	El usuario recibe una notificación automática de proximidad.	✓ Al acercarse a un punto, la app muestra una notificación emergente indicando que hay contenido disponible.
F5. Mostrar puntos cercanos en función de la ubicación.	Consultar la lista de puntos y comprobar el filtrado por cercanía.	Solo se muestran los puntos cercanos a la ubicación actual.	✓ La lista de puntos se actualiza y solo aparecen los que están dentro del radio de proximidad configurado.
F6. Acceder a la ficha de una mujer histórica al llegar a un punto.	Seleccionar un punto en el mapa o acercarse físicamente.	Se muestra la ficha informativa de la mujer histórica asociada.	✓ Al seleccionar un punto, se abre una pantalla con información detallada, imágenes y biografía de la mujer.
F7. Mostrar contenido multimedia asociado: texto, imágenes, audios o vídeos.	Acceder a la ficha de una mujer y visualizar los contenidos multimedia.	Todo el contenido multimedia se muestra correctamente.	✓ En la ficha se pueden ver imágenes, escuchar audios y ver vídeos relacionados con la figura histórica.
F8. Consultar un listado general de todas las mujeres incluidas en la app.	Acceder a la sección de listado de mujeres.	Se muestra la lista completa de mujeres disponibles.	✓ Se despliega una lista con todas las mujeres, permitiendo buscar y seleccionar cualquiera para ver su información.
F9. Visualizar imágenes ampliadas en modal al hacer tap sobre ellas.	Pulsar sobre una imagen en la ficha de detalle.	La imagen se amplía en un modal a pantalla completa.	✓ Al pulsar una imagen, esta se muestra ampliada ocupando toda la pantalla, con opción de cerrar el modal.
F10. Compartir información de lugares vía aplicaciones nativas del dispositivo.	Pulsar el botón de compartir en la ficha de un lugar.	Se abre el menú de apps nativas para compartir la información.	✓ Al pulsar compartir, se despliega el menú del sistema para enviar la información por WhatsApp, correo, etc.
F11. Registrar un nuevo usuario mediante nombre de usuario y contraseña.	Completar el formulario de registro y enviar los datos.	El usuario se registra correctamente y accede a la app.	✓ Tras rellenar el formulario, el usuario es dado de alta y accede automáticamente a la pantalla principal.
F12. Iniciar sesión en la aplicación.	Introducir credenciales válidas en la pantalla de login.	El usuario accede correctamente a su cuenta.	✓ Al introducir usuario y contraseña válidos, se accede a la app sin errores.

F13. Mantener la sesión activa entre usos.	Cerrar y volver a abrir la app tras iniciar sesión.	La sesión permanece activa sin necesidad de volver a iniciar sesión.	✓ Al reabrir la app, el usuario sigue autenticado y accede directamente a la pantalla principal.
F14. Permitir al usuario editar su nombre de usuario.	Acceder al perfil y modificar el nombre de usuario.	El cambio se guarda correctamente y se refleja en la app.	✓ Tras editar el nombre, el nuevo valor aparece en el perfil y en las secciones donde se muestra el usuario.
F15. Cerrar sesión manualmente.	Pulsar el botón de cerrar sesión en el perfil.	El usuario es desconectado y vuelve a la pantalla de login.	✓ Al pulsar cerrar sesión, la app vuelve a la pantalla de inicio de sesión y borra los datos de sesión.
F16. Filtrar lugares por área de investigación y estado de visita.	Aplicar filtros en la lista de lugares por área y estado.	Los resultados se actualizan correctamente según los filtros seleccionados.	✓ Al seleccionar filtros, la lista de lugares se actualiza mostrando solo los que cumplen los criterios.
F17. Buscar mujeres y lugares por nombre en tiempo real.	Escribir texto en el campo de búsqueda.	Los resultados se filtran en tiempo real según el texto introducido.	✓ Al escribir, la lista se actualiza instantáneamente mostrando solo coincidencias.
F18. Sincronizar estado de filtros entre diferentes pantallas de la aplicación.	Cambiar filtros en una pantalla y navegar a otra.	Los filtros aplicados se mantienen en todas las pantallas.	✓ Al cambiar de pantalla, los filtros seleccionados siguen activos y afectan a todas las vistas.
F19. Consultar el historial de puntos visitados, con fecha y hora.	Acceder al historial de visitas en el perfil.	Se muestra la lista cronológica de lugares visitados con fecha y hora.	✓ El historial muestra cada lugar visitado junto con la fecha y hora exacta de la visita.
F20. Consultar el historial de rutas completadas, con fecha y hora.	Acceder al historial de rutas en el perfil.	Se muestra la lista de rutas completadas con fecha y hora.	✓ Se visualiza un listado de rutas finalizadas, cada una con su fecha de finalización.

F21. Iniciar sesión como administrador desde el backend web.	Acceder al panel de administración e iniciar sesión con credenciales de admin.	El administrador accede correctamente al panel.	✓ Tras introducir las credenciales, se accede al panel de gestión con todas las opciones habilitadas.
F22. Añadir nuevas figuras históricas con nombre, descripción y material multimedia.	Crear una nueva figura histórica desde el panel de administración.	La figura se añade correctamente y aparece en la app.	✓ Al guardar una nueva figura, esta aparece en la app móvil y puede ser consultada por los usuarios.
F23. Asociar cada mujer con un punto geolocalizado.	Editar o crear una figura y asignarle una ubicación.	La asociación se guarda y se refleja en el mapa.	✓ Al asignar una ubicación, el punto aparece en el mapa vinculado a la figura correspondiente.
F24. Cargar imágenes, audios, vídeos y modelos 3D desde el panel.	Subir archivos multimedia al crear o editar figuras.	Los archivos se cargan y se muestran correctamente en la app.	✓ Los archivos subidos desde el panel se visualizan en la ficha de la figura en la app móvil.
F25. Editar o eliminar puntos o figuras ya creadas.	Modificar o borrar una figura o punto desde el panel.	Los cambios se aplican correctamente en la app.	✓ Al editar o eliminar, los cambios se reflejan en la app tras sincronizar los datos.
F26. Ver la lista de usuarios registrados.	Acceder a la sección de usuarios en el panel de administración.	Se muestra la lista completa de usuarios registrados.	✓ El panel muestra todos los usuarios registrados, con opción de ver detalles.
F27. Eliminar usuarios si fuese necesario.	Seleccionar un usuario y eliminarlo desde el panel.	El usuario es eliminado correctamente del sistema.	✓ Tras eliminar, el usuario desaparece de la lista y no puede acceder a la app.
F28. Adaptar la interfaz a distintos tamaños de pantalla.	Probar la app en dispositivos con diferentes resoluciones.	La interfaz se adapta correctamente a todos los tamaños de pantalla.	✓ La app se visualiza correctamente en móviles y tablets, ajustando los elementos a cada resolución.
F29. Navegar por la app de forma intuitiva y con accesibilidad mobile-first.	Utilizar la app en modo móvil y comprobar la navegación.	La navegación es fluida, intuitiva y accesible.	✓ Los menús y botones son accesibles y la navegación entre pantallas es sencilla y rápida.
F30. Garantizar el uso fluido y sin errores tanto en Android como en iOS.	Ejecutar la app en ambos sistemas operativos.	La app funciona correctamente en Android e iOS.	✓ La app se ejecuta sin errores ni diferencias relevantes en ambos sistemas.

F31. Ver una lista de rutas temáticas con sus nombres y descripciones.	Acceder a la sección de rutas en la app.	Se muestra la lista de rutas temáticas con sus descripciones.	✓ Se visualizan todas las rutas disponibles, cada una con su nombre y descripción.
F32. Consultar los puntos que forman parte de cada ruta.	Seleccionar una ruta y ver los puntos asociados.	Se muestran correctamente los puntos de cada ruta.	✓ Al seleccionar una ruta, se visualizan todos los puntos que la componen en la pantalla.
F33. Escanear códigos QR para desbloquear puntos de una ruta.	Escanear el QR de un punto físico de una ruta temática.	El punto se marca como visitado solo con el QR válido.	✓ Tras escanear el código QR, el punto correspondiente se desbloquea y se actualiza el progreso de la ruta.
F34. Reiniciar el progreso de rutas ya completadas para repetir la experiencia.	Utilizar la opción de reinicio de rutas en la app.	El progreso de la ruta se reinicia correctamente.	✓ Al pulsar la opción de reinicio, todos los puntos de la ruta vuelven a estar pendientes y se puede repetir la experiencia desde cero.
F35. Ver una lista de los puntos con contenido RA disponible.	Acceder a la sección de RA en la app.	Se muestra la lista de puntos con RA disponible.	✓ La sección de RA muestra únicamente los puntos que disponen de contenido de realidad aumentada.
F36. Mostrar únicamente los puntos con RA que estén a una distancia determinada del usuario.	Comprobar la lista de RA desde diferentes ubicaciones.	Solo se muestran los puntos con RA accesibles por proximidad.	✓ La lista de RA se actualiza dinámicamente y solo aparecen los puntos accesibles según la ubicación actual del usuario.
F37. Al seleccionar uno de estos puntos, abrir automáticamente la cámara y cargar el modelo RA asociado.	Seleccionar un punto RA y permitir acceso a la cámara.	Se abre la cámara y se carga el modelo RA correspondiente.	✓ Al seleccionar un punto con RA, la app solicita acceso a la cámara y muestra el modelo 3D o animación correspondiente en la pantalla.

Todas las funcionalidades han sido probadas y se cumplen correctamente según los requisitos y especificaciones definidos.

7.2. Pruebas de Usabilidad

Las pruebas de usabilidad se realizaron para evaluar la experiencia de uso de la aplicación, su facilidad de navegación y la percepción general de los usuarios. Para ello, se empleó el método *System Usability Scale* (SUS) [77], una herramienta estándar que permite obtener una medida cuantitativa de la usabilidad.

7.2.1. Metodología

El cuestionario SUS consta de 10 afirmaciones relacionadas con aspectos clave como la facilidad, consistencia y confianza del usuario. Cada afirmación se puntúa en una escala de 1 (Totalmente en desacuerdo) a 5 (Totalmente de acuerdo).

En esta evaluación participaron **17 usuarios**, tal y como se puede observar en las imágenes Figura 7.1, quienes utilizaron la aplicación de forma integral durante una sesión completa que incluyó:

- **Registro e inicio de sesión** con credenciales personales.
- **Navegación entre pestañas** principales (Mapa, Mujeres, Rutas, RA, Perfil).
- **Exploración de lugares** con filtros y búsquedas .
- **Marcado de lugares como visitados** y gestión del historial personal.
- **Completado de rutas temáticas** mediante escaneo de códigos QR.
- **Experiencia de Realidad Aumentada** con modelos 3D interactivos.
- **Recepción de notificaciones** automáticas por proximidad.
- **Uso del mapa interactivo** con marcadores e información contextual.

Tras completar esta experiencia integral con todas las funcionalidades de la aplicación, los usuarios respondieron al cuestionario SUS para evaluar la usabilidad general del sistema.



Figura 7.1: Usuarios probando la aplicación DONA'm MÓN en el IVAM

7.2.2. Cálculo del SUS

El cálculo de la puntuación SUS se realiza del siguiente modo:

- Para preguntas impares (1, 3, 5, 7, 9): (valor) − 1.
- Para preguntas pares (2, 4, 6, 8, 10): 5 − (valor).
- Se suman los resultados anteriores y se multiplica por 2,5.

Ejemplo para U1:

$$\text{Ajuste} = ((4 - 1) + (5 - 1) + (5 - 1) + (4 - 1) + (4 - 1)) +$$

$$((5 - 1) + (5 - 2) + (5 - 1) + (5 - 1) + (5 - 1))$$

$$\text{Suma total} = 34, \text{ SUS} = 34 \times 2,5 = 85$$

7.2.3. Resultados del cuestionario

En la Tabla 7.2 se muestran las puntuaciones otorgadas por cada usuario.

Usuario	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10
<i>Escala: 1 = Totalmente en desacuerdo, 5 = Totalmente de acuerdo</i>										
U1	4	1	4	1	5	2	5	1	4	1
U2	4	3	4	2	4	3	4	2	5	2
U3	5	1	5	1	5	1	5	1	5	1
U4	4	1	5	1	5	1	4	1	5	1
U5	4	1	5	1	5	1	5	1	5	5
U6	5	2	4	2	5	1	4	2	5	2
U7	5	2	5	4	4	2	5	2	5	2
U8	4	1	4	4	5	1	4	3	5	2
U9	4	1	5	1	5	2	4	1	5	1
U10	4	1	5	1	4	2	5	1	5	1
U11	5	2	5	1	5	2	5	2	5	2
U12	4	2	5	2	5	2	4	2	4	2
U13	5	1	4	1	5	1	5	2	5	1
U14	4	3	4	2	5	2	4	2	4	2
U15	5	4	5	2	5	1	5	1	5	1
U16	4	1	5	1	4	2	5	2	5	1
U17	5	1	5	2	5	1	5	1	5	1

Cuadro 7.2: Respuestas del cuestionario SUS por usuario

7.2.4. Visualización de resultados por pregunta

Además de la tabla general, se presentan a continuación las preguntas del cuestionario SUS junto con las gráficas que ilustran la distribución de respuestas para cada una. Estas visualizaciones permiten comprender de manera más clara la percepción de los usuarios en relación con los distintos aspectos de la usabilidad evaluados.

Pregunta 1: Considero que me gustaría utilizar frecuentemente esta aplicación.

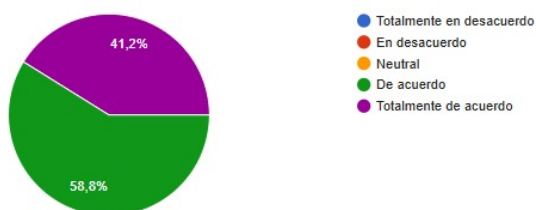


Figura 7.2: Distribución de respuestas a la Pregunta 1

Pregunta 2: He encontrado la aplicación innecesariamente compleja en algunos momentos.

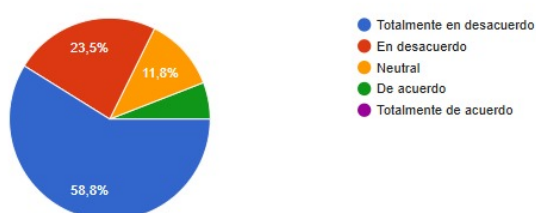


Figura 7.3: Distribución de respuestas a la Pregunta 2

Pregunta 3: Me ha parecido fácil de usar.

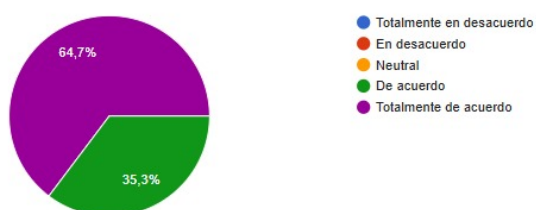


Figura 7.4: Distribución de respuestas a la Pregunta 3

Pregunta 4: Creo que necesitaría ayuda de una persona con conocimientos técnicos para poder usarla.

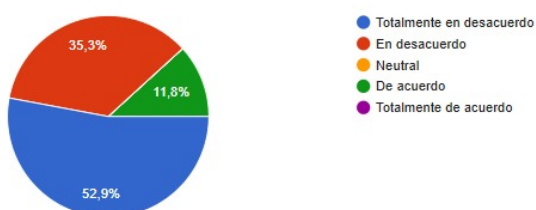


Figura 7.5: Distribución de respuestas a la Pregunta 4

Pregunta 5: Las funcionalidades están bien integradas y tienen coherencia entre sí.

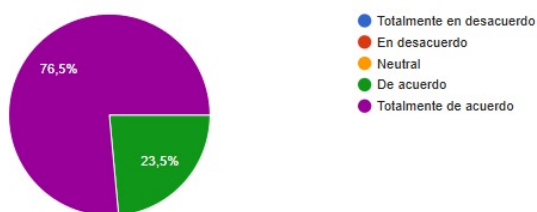


Figura 7.6: Distribución de respuestas a la Pregunta 5

Pregunta 6: Me parece que hay demasiada inconsistencia en la aplicación.

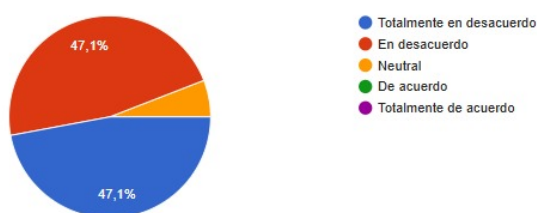


Figura 7.7: Distribución de respuestas a la Pregunta 6

Pregunta 7: Creo que la mayoría de la gente aprendería a usar esta aplicación con rapidez.

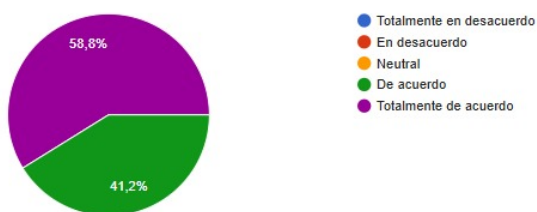


Figura 7.8: Distribución de respuestas a la Pregunta 7

Pregunta 8: Me ha parecido confusa o difícil de manejar en algunos aspectos.

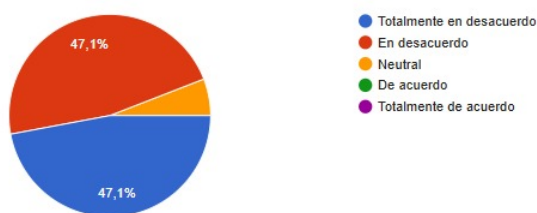


Figura 7.9: Distribución de respuestas a la Pregunta 8

Pregunta 9: Me he sentido segura usando la aplicación.

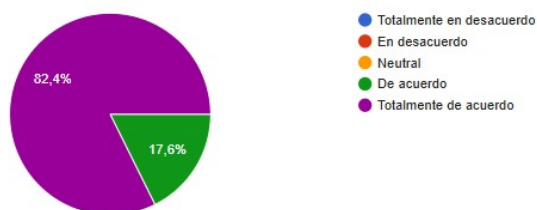


Figura 7.10: Distribución de respuestas a la Pregunta 9

Pregunta 10: Necesité aprender muchas cosas antes de poder usarla correctamente.

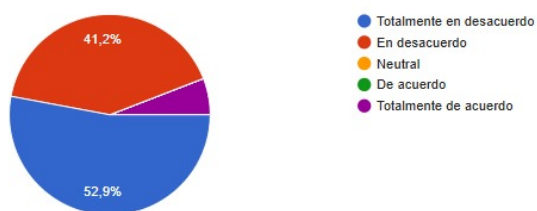


Figura 7.11: Distribución de respuestas a la Pregunta 10

7.2.5. Puntuaciones SUS

En la Tabla 7.3 se muestran las puntuaciones calculadas aplicando la fórmula mencionada anteriormente, junto con información básica sobre los participantes (edad, ocupación y familiaridad con Realidad Aumentada). Cabe destacar que todos los usuarios tienen experiencia utilizando aplicaciones móviles.

Las edades de los usuarios oscilan principalmente entre los 22 y los 27 años, que es el público objetivo principal de la aplicación. No obstante, también se ha realizado la prueba con usuarios en torno a los 50 años para recoger opiniones y percepciones de otros rangos de edad.

Usuario	Edad	Descripción	Familiaridad RA	Puntuación SUS
U1	22	Estudiante de Bellas Artes	Baja	85
U2	21	Periodista	Baja	77,5
U3	19	Estudiante de Óptica y Optometría	Baja	100
U4	24	Ingeniero en Telecomunicaciones	Media	95
U5	22	Especialista en marketing	Baja	92,5
U6	22	Fisioterapeuta	Baja	90
U7	55	Limpiadora	Baja	80
U8	50	Instalador del gas	Baja	82,5
U9	24	Estudiante de Desarrollo Web	Alta	90
U10	22	Estudiante de Desarrollo Web	Alta	90
U11	24	Restauradora de Arte	Baja	87,5
U12	27	Ingeniero Multimedia	Alta	82,5
U13	23	Ingeniera en Telecomunicaciones	Media	95
U14	23	Estudiante de Ingeniería Multimedia	Alta	77,5
U15	23	Dietista	Baja	87,5
U16	24	Estudiante de Desarrollo Web	Alta	90
U17	22	Estudiante de Desarrollo Web	Alta	95
Media	–	–	–	87,65

Cuadro 7.3: Perfil de los participantes y puntuación SUS final

Los resultados obtenidos reflejan una puntuación media de **87,65** en la escala SUS, lo que sitúa la aplicación en la categoría *Excelente*. Como se observa en la Figura 7.12, la mayoría de usuarios obtiene puntuaciones superiores a 80, con varios casos alcanzando valores cercanos o iguales a 100.

La distribución de las puntuaciones SUS por rangos puede verse en la Figura 7.13, donde se aprecia que la mayor parte de los participantes se concentra en los intervalos de 90-99, lo que refuerza la excelente percepción de usabilidad de la aplicación.

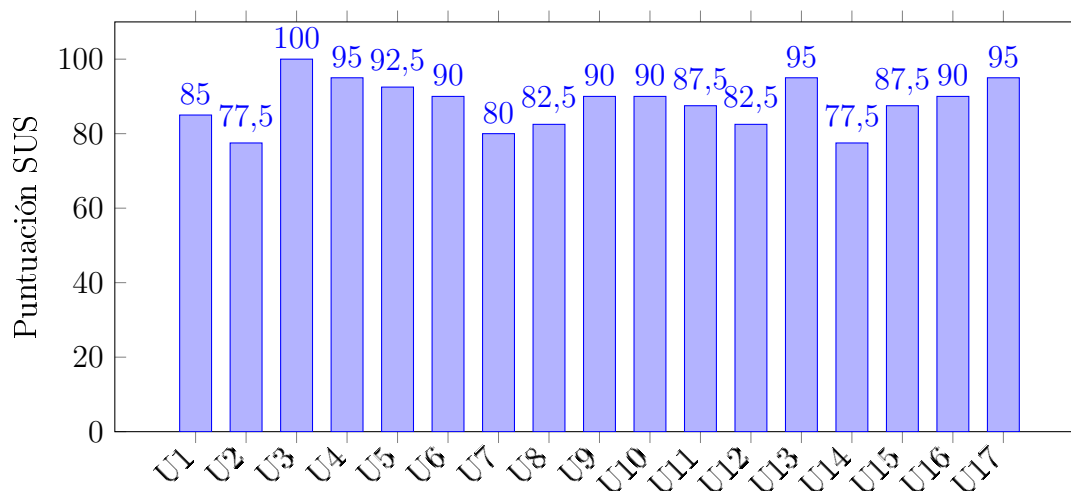


Figura 7.12: Puntuaciones SUS por usuario.

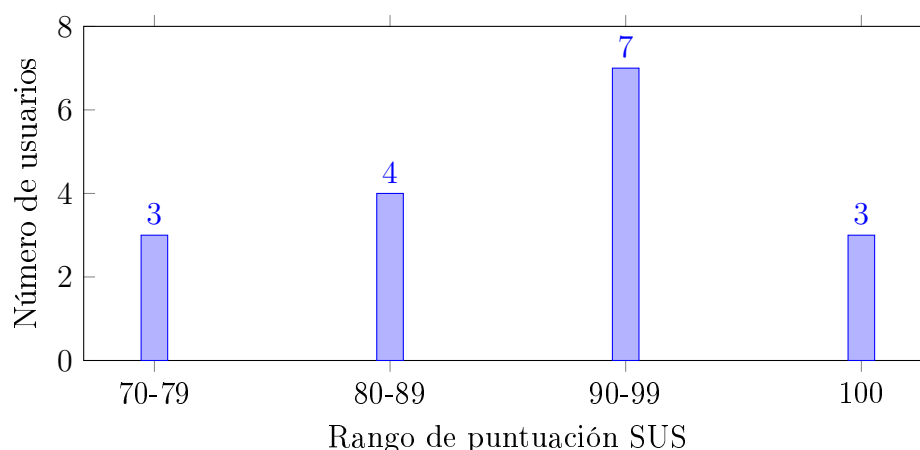


Figura 7.13: Distribución de puntuaciones SUS.

7.2.6. Comentarios adicionales de los usuarios

Además del cuestionario SUS, se realizaron preguntas abiertas para obtener una visión más cualitativa de la experiencia. A continuación, se destacan las más relevantes:

¿Qué parte de la aplicación te ha resultado más útil o interesante?

- *“Las rutas de mujeres destacadas”.*
- *“La parte de la realidad aumentada”.*
- *“El mapa interactivo, es muy innovador y curioso”.*
- *“Como aspecto más interesante e innovador destacaría la opción para hacer rutas en base a las distintas ubicaciones. También es interesante poder ver la fecha en la que visitaste cada lugar”.*
- *“Cuando sale en 3D la mujer”.*

¿Has encontrado algún problema mientras usabas la app?

- La mayoría respondió “No” o *“Ninguno en general, el funcionamiento es completamente satisfactorio”.*
- Solo un usuario opino que: *“La Realidad Aumentada es un poco difícil de usar”.*

¿Te ha parecido intuitiva la navegación entre pantallas?

- Comentarios más repetidos: *“Sí, es muy fácil de usar, además está todo muy bien relacionado”*, *“Muy sencilla e intuitiva”.*

¿Has disfrutado la experiencia de realidad aumentada? ¿Por qué?

- *“Es mi parte favorita de la app, es muy entretenida y original”.*
- *“Es fascinante, hace que la app sea mucho más inmersiva”.*
- *“Aporta un elemento diferente en comparación con otras aplicaciones”.*
- Solo una respuesta negativa: *“No mucho, nunca me ha gustado la RA en aplicaciones”.*

¿Consideras que la aplicación te ha permitido descubrir información nueva o interesante sobre las mujeres de Valencia?

- *“Sí, es necesario tener herramientas que nos permitan conocer más sobre las acciones que las mujeres han hecho durante la historia”.*
- *“Por supuesto, además de añadir conocimientos de valor acerca de nuestra cultura”.*

¿Recomendarías esta aplicación a otras personas?

- Todas las respuestas fueron afirmativas: *“Por supuesto!!!!”, “Si están interesadas en aprender sobre el tema, sin duda”.*

¿Alguna sugerencia para mejorar la aplicación?

- La mayoría respondió *“No”* o *“Está todo correcto”.*
- Comentarios destacados: *“Tal vez un enlace para ampliar aún más la información de cada protagonista”, “Que los usuarios puedan recomendar nuevos puntos de interés”.*

Podemos concluir, basándonos en los comentarios de los usuarios, que han percibido la navegación como intuitiva y sencilla. Han destacado la coherencia entre las distintas secciones y la facilidad para acceder a la información. Las funcionalidades principales, en particular el mapa interactivo, las rutas temáticas y la experiencia de Realidad Aumentada, se consolidan como los aspectos más valorados, ya que aportan dinamismo y un carácter innovador frente a otras aplicaciones similares. Además, se han propuesto mejoras que se han añadido a la lista de trabajo a futuro.

7.3. Hardware empleado en las Pruebas de rendimiento

Las pruebas de rendimiento se llevaron a cabo en un único equipo portátil para el backend y en un dispositivo móvil para el frontend y la experiencia de realidad aumentada. En la tabla 7.4 se detallan las características técnicas de ambos dispositivos, ya que estas especificaciones pueden influir en los resultados obtenidos.

Cuadro 7.4: Especificaciones del hardware utilizado en las pruebas

Componente	Portátil [78]	Móvil [79]
Modelo	Asus ROG Strix G15 G513IC-HN004	iPhone 11
Procesador	AMD Ryzen 7 4800H (8 núcleos, 2.9-4.2 GHz)	A13 Bionic (6 núcleos)
Memoria RAM	16 GB DDR4 a 3200 MHz	4 GB
Almacenamiento	512 GB SSD M.2 NVMe PCIe	64 GB
GPU	NVIDIA GeForce RTX 3050, 4 GB GDDR6	GPU Apple de 4 núcleos
Pantalla	15.6" Full HD, 144 Hz	6.1" Liquid Retina HD (1792 x 828)
Sistema operativo	Windows 11 Home	iOS 18.5
Conectividad	Wi-Fi 6, Bluetooth 5.1	Wi-Fi 802.11ax, Bluetooth 5.0

7.4. Pruebas de Rendimiento del Backend

7.4.1. Metodología

Para comprobar el rendimiento del backend, se han realizado pruebas con **Apache JMeter** [80], simulando condiciones de uso normal y situaciones extremas. Se han analizado métricas como el tiempo de respuesta (desde que se envía la petición hasta que llega la respuesta), percentiles para entender la distribución, la desviación estándar para medir estabilidad, el throughput (peticiones procesadas por segundo) y el porcentaje de errores.

Configuración de Pruebas

Se definieron dos escenarios:

- **Carga Normal:** 10 usuarios concurrentes, 10 iteraciones por usuario (100 peticiones por endpoint) y usuarios añadidos progresivamente durante 5 segundos (ramp-up de 5 s).
- **Carga de Estrés:** 50 usuarios concurrentes, 20 iteraciones por usuario (1000 peticiones por endpoint) y usuarios añadidos progresivamente durante 10 segundos (ramp-up de 10 s).

Los endpoints evaluados corresponden a las principales operaciones: autenticación (*Login*), lectura de datos (*GET Mujeres*, *GET Lugares*, *GET Rutas*, *GET Visited Lugares*) y escritura (*POST Visit Lugar*, *POST Visit Lugar Ruta*). Todos, salvo *Login*, requieren autenticación JWT.

7.4.2. Resultados y Análisis

Carga Normal

La Tabla 7.5 muestra los resultados con 10 usuarios concurrentes. Se incluyen el tiempo medio (**Avg**), valores percentiles (P90, P95), máximo, desviación estándar y throughput.

Cuadro 7.5: Resultados bajo carga normal (10 usuarios, 100 peticiones por endpoint)

Endpoint	Avg	Min	Median	P90	P95	Max	Std Dev	Throughput	Error %
Login	458	358	460	500	519	529	36.0	8.22	0
GET Mujeres	37	20	34	54	66	80	13.7	8.46	0
GET Lugares	42	20	38	65	72	126	19.1	8.46	0
GET Rutas	54	27	48	81	100	153	22.8	8.44	0
POST Visit Lugar	67	53	64	79	86	99	9.9	8.42	0
POST Visit Lugar Ruta	70	59	69	88	89	119	11.1	8.42	0
GET Visited Lugares	71	58	69	89	89	120	11.9	8.42	0

Bajo carga normal, el sistema presenta un comportamiento óptimo tanto en velocidad como en estabilidad. Como se aprecia en la Tabla 7.5, la mayoría de endpoints procesan las peticiones en menos de 100 ms, garantizando una experiencia de usuario fluida y sin retardos perceptibles.

Las operaciones de lectura destacan por su rapidez: *GET Mujeres* responde en apenas 37 ms, y *GET Lugares* en 42 ms, valores excelentes considerando que implican consultas y serialización de datos JSON. Incluso *GET Rutas*, con 54 ms, mantiene tiempos muy

bajos. Las operaciones de escritura (*POST Visit Lugar* y *POST Visit Lugar Ruta*) son ligeramente más costosas, situándose en torno a 67-70 ms, debido a la validación de datos y la inserción en base de datos, pero siguen siendo muy rápidas.

El único endpoint que supera los 400 ms es *Login* (458 ms), lo cual era esperado porque incluye la generación y firma criptográfica del token JWT, una operación CPU-bound.

La baja desviación estándar (≤ 36 ms en todos los casos) evidencia que la latencia es estable y predecible, sin picos relevantes. Además, no se registraron errores (0) y el throughput ronda las 8.4 req/s, lo que indica que el backend no solo es rápido, sino también consistente.

Carga de Estrés

La Tabla 7.6 presenta los resultados con 50 usuarios y 1000 peticiones por endpoint.

Cuadro 7.6: Resultados bajo carga de estrés (50 usuarios, 1000 peticiones por endpoint)

Endpoint	Avg	Min	Median	P90	P95	Max	Std Dev	Throughput	Error %
Login	1154	371	1150	1490	1659	2155	279.6	8.81	0
GET Mujeres	674	24	696	902	968	1429	207.6	8.82	0
GET Lugares	783	23	806	1014	1096	1779	226.8	8.78	0
GET Rutas	892	26	901	1147	1227	1788	225.5	8.75	0
POST Visit Lugar	613	60	634	794	841	1477	176.3	8.75	0
POST Visit Lugar Ruta	571	60	590	760	810	1419	183.3	8.75	0
GET Visited Lugares	554	69	570	741	809	1084	177.1	8.74	0

Bajo carga de estrés, la latencia aumenta significativamente, especialmente en operaciones de lectura (*GET*), que llegan a 892 ms en *GET Rutas* y 783 ms en *GET Lugares*. Las operaciones de escritura escalan mejor, aunque también se incrementan a valores cercanos a 600 ms. El endpoint *Login* alcanza 1154 ms, pero mantiene una degradación proporcionalmente menor ($\approx 2.5x$).

A pesar del aumento, la ausencia de errores y un throughput constante de 8.7 req/s demuestran que el sistema sigue siendo estable y no pierde peticiones, sacrificando únicamente velocidad. Sin embargo, estos tiempos podrían impactar la experiencia en escenarios de alta concurrencia prolongada, lo que sugiere que optimizaciones a nivel de consultas y cacheado serían recomendables para mejorar la escalabilidad.

Interpretación AVG: La Figura 7.14 muestra cómo los tiempos promedio bajo estrés aumentan significativamente en comparación con la carga normal. Operaciones de lectura (*GET*) son las más afectadas, alcanzando entre 674 ms y 892 ms, mientras que las de escritura (*POST*) escalan mejor.

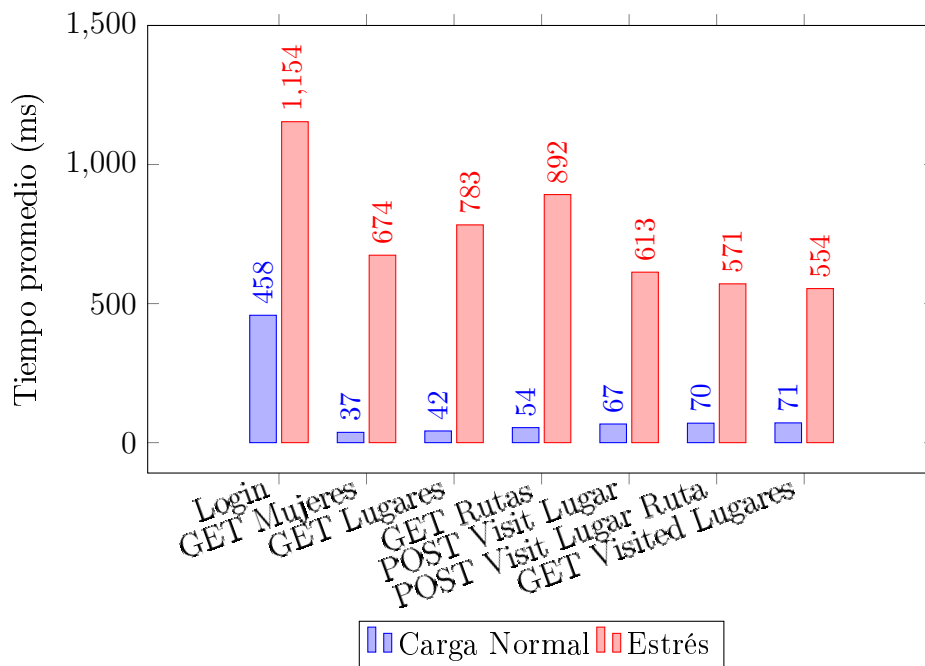


Figura 7.14: Comparación de tiempos promedio por endpoint

Interpretación P95: El percentil 95 refleja el tiempo que tarda el 95% de las peticiones en completarse. Como se ve en la Figura 7.15, bajo carga normal la mayoría de endpoints se mantienen por debajo de 100 ms, excepto *Login*. Bajo estrés, el P95 alcanza 1659 ms en *Login* y 1227 ms en *GET Rutas*, lo que indica que las colas de espera y la saturación afectan significativamente la experiencia.

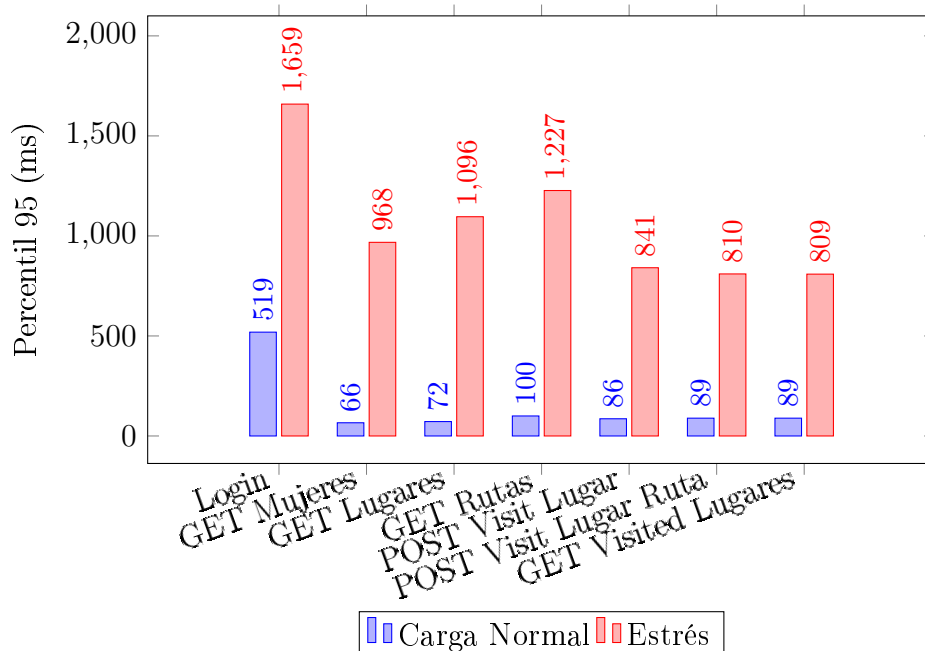


Figura 7.15: Comparación del percentil 95 por endpoint

Interpretación Max: El tiempo máximo representa los casos más extremos. En la Figura 7.16 se aprecia que bajo estrés los picos superan los 2 segundos en *Login* y alcanzan casi 1.8 segundos en operaciones de lectura. Aunque son valores excepcionales, son importantes porque pueden impactar la percepción de fluidez.

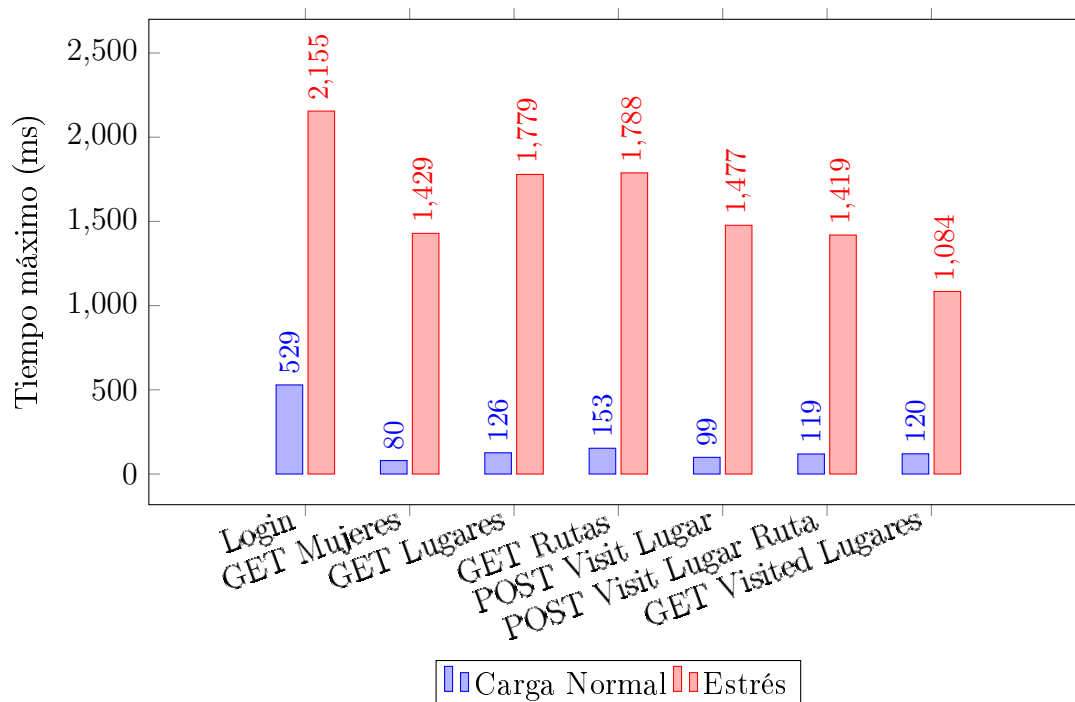


Figura 7.16: Comparación de tiempos máximos por endpoint

Conclusión Global

El sistema muestra un diseño estable y escalable. Bajo carga normal, los tiempos son óptimos (<100 ms en casi todos los endpoints) y sin errores. Bajo estrés, la degradación es significativa (P95 hasta 1659 ms), pero el throughput se mantiene (8.7 req/s) y la tasa de error sigue en 0 %, lo que confirma que el backend soporta alta concurrencia sin fallos, aunque la latencia pueda afectar la UX en escenarios extremos.

7.4.3. Pruebas de Rendimiento del Frontend

Para evaluar el rendimiento del frontend de la aplicación, se ha utilizado la herramienta *Performance Monitor* integrada en **Expo Go**. Esta permite monitorizar métricas clave como el uso de memoria, la tasa de fotogramas por segundo (*frames per second, FPS*) y la inicialización de módulos nativos, entre otros.

En la Figura 7.17 se muestra la interfaz de la aplicación junto con el panel de información proporcionado. Entre los parámetros monitorizados destacan:

- **Uso de RAM:** Durante la ejecución normal, el consumo promedio de memoria se situó en torno a **430 MB**.
- **Velocidad de actualización (FPS):**
 - En el hilo de interfaz de usuario (*UI thread*), los FPS se mantuvieron de forma constante en **60**, salvo en operaciones más exigentes como aplicar filtros o compartir contenido, donde se observó una caída puntual a **58 FPS** durante aproximadamente un segundo.
 - En el hilo de JavaScript (*JS thread*), la frecuencia también se mantuvo estable en **60 FPS** en condiciones normales, pero se redujo a **51 FPS** en acciones que requieren procesamiento adicional (por ejemplo, filtrado y compartición).

Durante la navegación entre pantallas, el promedio del hilo de JavaScript se situó en torno a **55 FPS**.

- **Inicialización y carga de scripts:** Los valores relacionados con la descarga y ejecución del *bundle* (e.g., *ScriptDownload*, *ScriptExecution*, *RAMBundleLoad*) se mantuvieron en **0 ms**, reflejando que la aplicación ya se encontraba completamente cargada en el momento de la prueba.

En general, los resultados demuestran que se mantiene una experiencia fluida incluso en operaciones que implican renderizado adicional, con un consumo de memoria dentro de lo esperado.

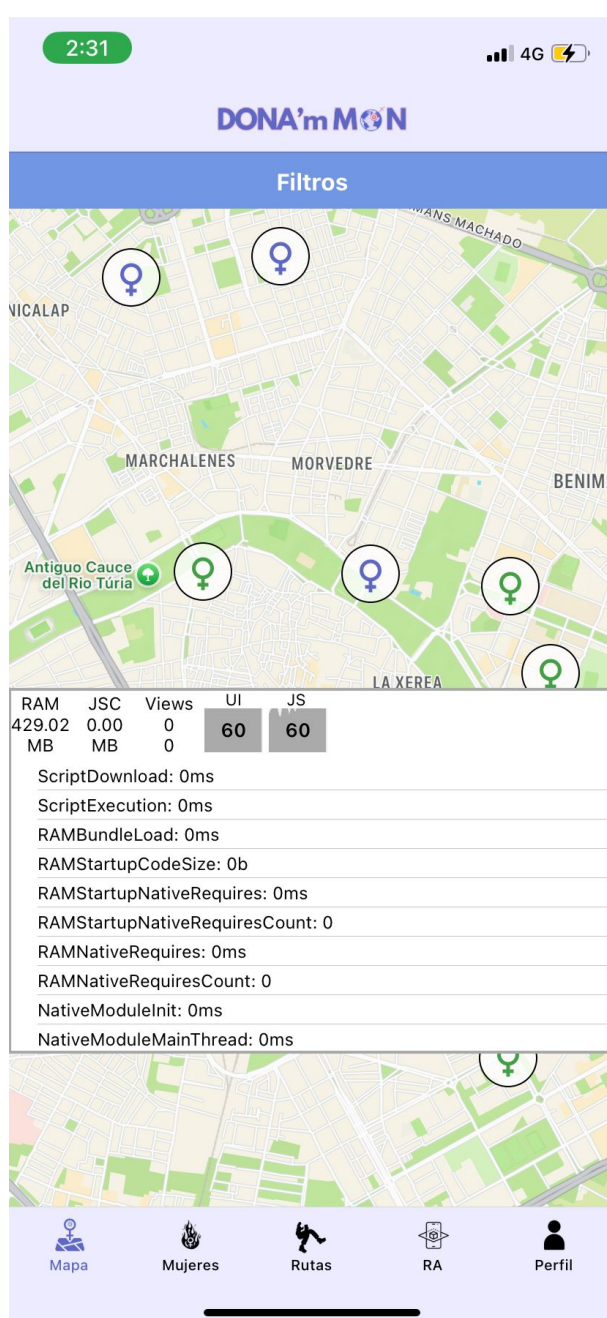


Figura 7.17: Aplicación DONA'm MÓN ejecutándose en Expo Go con el panel *Performance Monitor*.

7.5. Pruebas de Rendimiento de la Realidad Aumentada

Para evaluar el rendimiento de la experiencia de realidad aumentada implementada con A-Frame y AR.js, se han realizado pruebas sobre la versión desplegada en *Vercel* (<https://ivam.vercel.app/>) representando el entorno real de uso. El objetivo fue medir el comportamiento en términos de tiempos de carga y estabilidad del renderizado (FPS) durante la interacción del usuario.

7.5.1. Metodología

Se adaptó el código mediante un componente propio en A-Frame que registraba datos de rendimiento en consola, visibles en el dispositivo móvil gracias a la librería *Eruda* [81]. Estos registros podían consultarse directamente en el dispositivo móvil mediante la consola, como se muestra en la Figura 7.18.

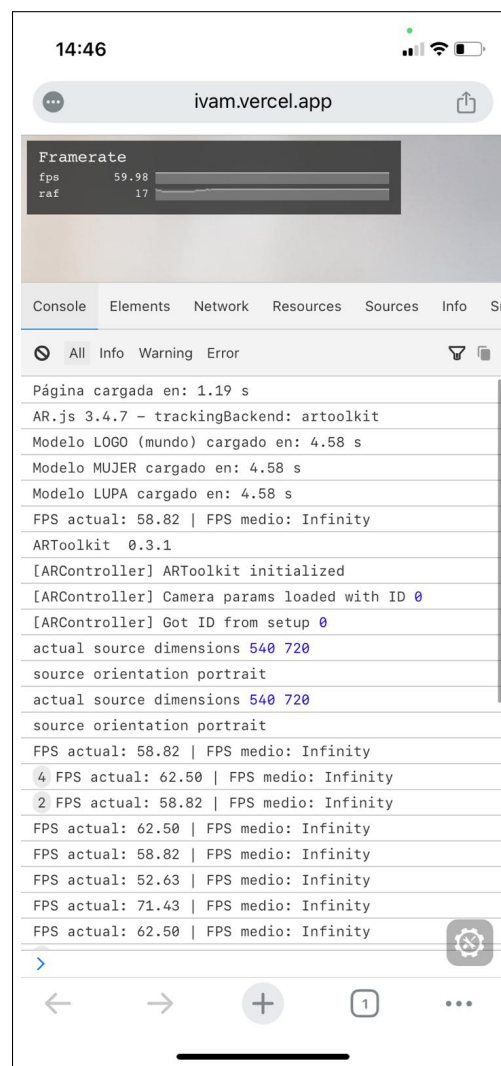


Figura 7.18: Captura de los registros obtenidos en el dispositivo durante la prueba

Los parámetros evaluados fueron:

- **Tiempo de carga inicial:** desde la carga de la página hasta la activación de la cámara AR.
- **Carga de modelos 3D:** tiempo exacto en que cada modelo (mundo, lupa, mujer) se encontraba disponible en la escena.
- **Evolución del FPS:** tasa de frames por segundo antes y después de interacciones críticas.

Para registrar los valores se añadió el siguiente código JavaScript:

Listado 7.1: Fragmento de código para medición de FPS y eventos clave

```
const startTime = performance.now();
AFRAME.registerComponent('fps-logger', {
  tick: function (time, delta) {
    const fps = 1000 / delta;
    if (Math.floor(time) % 1000 === 0) console.log('FPS:', fps.toFixed(2));
  }
});
document.querySelector('#mundoModel').addEventListener('model-loaded', () =>
  console.log('Modelo MUNDO cargado en:', ((performance.now() - startTime)/1000)
    .toFixed(2), 's')
);
document.querySelector('#lupaModel').addEventListener('model-loaded', () =>
  console.log('Modelo LUPA cargado en:', ((performance.now() - startTime)/1000)
    .toFixed(2), 's')
);
document.querySelector('#mujerModel').addEventListener('model-loaded', () =>
  console.log('Modelo MUJER cargado en:', ((performance.now() - startTime)/1000)
    .toFixed(2), 's')
);
```

7.5.2. Resultados

En la Tabla 7.7 se resumen los valores más representativos. El tiempo indicado para cada evento corresponde al momento en que se produce, medido desde la carga inicial de la página.

Cuadro 7.7: Tiempos de carga y eventos críticos

Evento	Tiempo (s)	FPS aproximado
Carga completa de la página	1.19	58–62
Modelo MUNDO cargado	4.58	58–62
Modelo LUPA cargado	4.58	58–62
Modelo MUJER cargado	4.58	55–60
Click en el logo (animación)	~6	30–15 (mínimo 13)
Click en la mujer (panel informativo)	~8.5	25–30 (impacto leve)

La Figura 7.19 representa la evolución del FPS durante la interacción: inicialmente estable, caída abrupta tras el clic en el logo, leve descenso al mostrar el panel informativo y recuperación completa después de unos segundos.

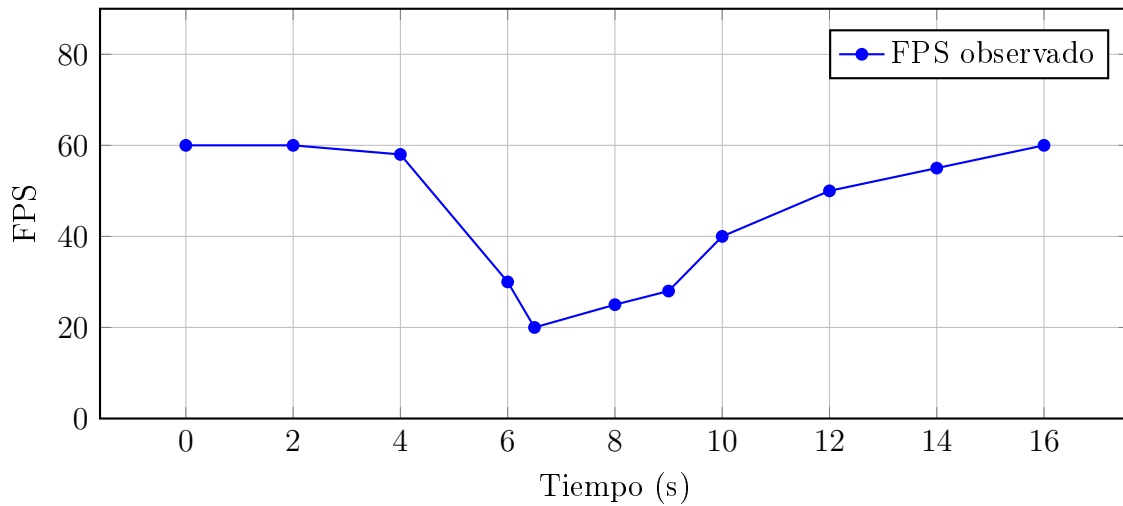


Figura 7.19: Evolución del FPS durante la interacción con elementos AR

7.5.3. Análisis

Los resultados indican que la experiencia mantiene una tasa de **50-60 FPS en condiciones normales**, asegurando fluidez inicial (ver Figura 7.19). Sin embargo:

- **El mayor impacto ocurre al hacer clic en el logo**, donde se activan múltiples animaciones simultáneas (desaparición del mundo y aparición del modelo principal), reduciendo el FPS hasta 13.
- **El panel informativo genera un impacto menor**, manteniendo los FPS en torno a 25-30, ya que solo añade un plano con texto sin animaciones complejas.

Capítulo 8

Resultados

Para ofrecer una visión clara del funcionamiento, se explica a continuación cómo el usuario interactúa con la aplicación desde que la abre hasta que finaliza su uso.

El punto de partida es la pantalla de bienvenida, tal como se observa en la Figura 8.1. También vemos cómo ha saltado la notificación de que nos encontramos cerca de un lugar.



Figura 8.1: Pantalla de bienvenida y notificación de proximidad a un lugar.

En la Figura 8.2 se ve el mapa con los lugares visitados en verde y los no visitados en rosa, además de nuestra ubicación.

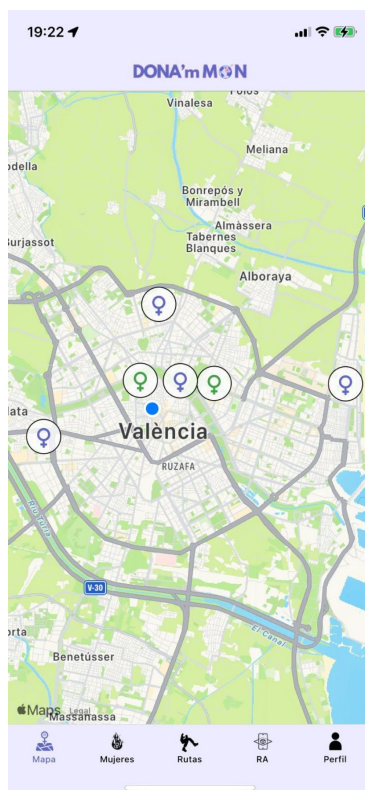


Figura 8.2: Mapa con lugares visitados (verde), no visitados (rosa) y ubicación actual.

Al pulsar sobre un lugar se muestra la información de la mujer y el lugar, además de la distancia (Figura 8.3).



Figura 8.3: Detalle de un lugar: información de la mujer, el lugar y distancia.

En la Figura 8.4 se ve el listado de lugares, y también podemos observar el buscador

por nombre de lugares y de mujeres.

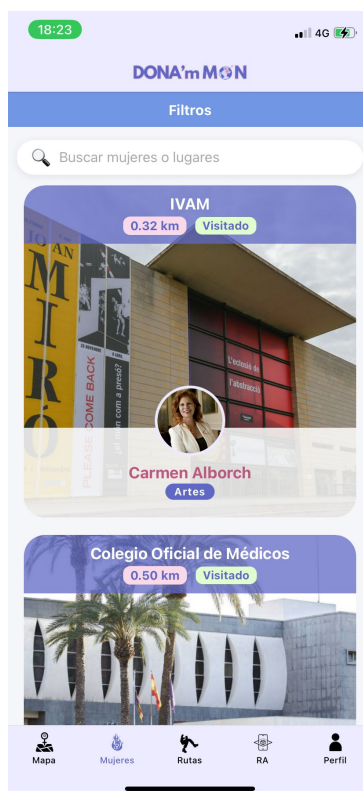


Figura 8.4: Listado de lugares y buscador por nombre de lugares y mujeres.

En la Figura 8.5 observamos los filtros que podemos aplicar tanto sobre el mapa como sobre el listado de lugares.

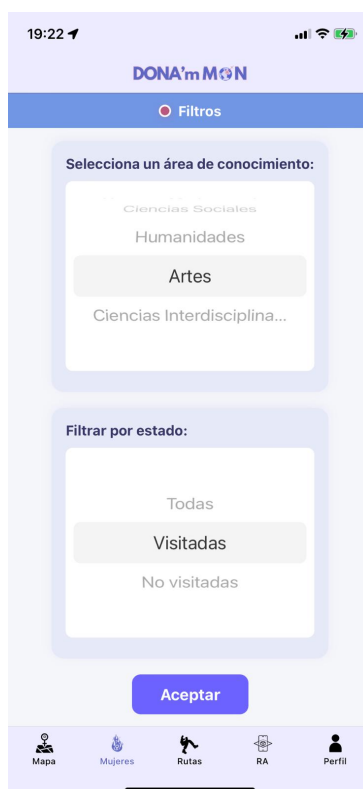
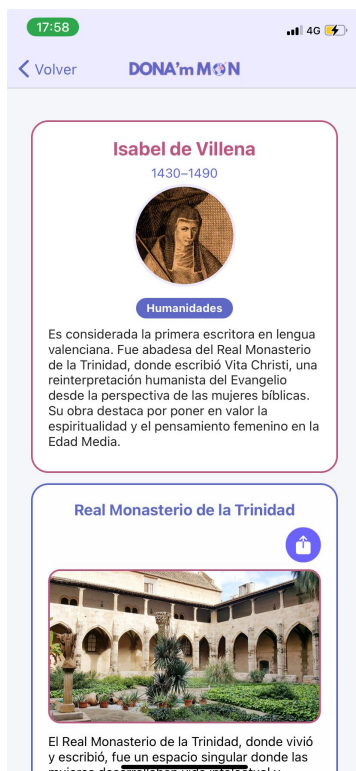
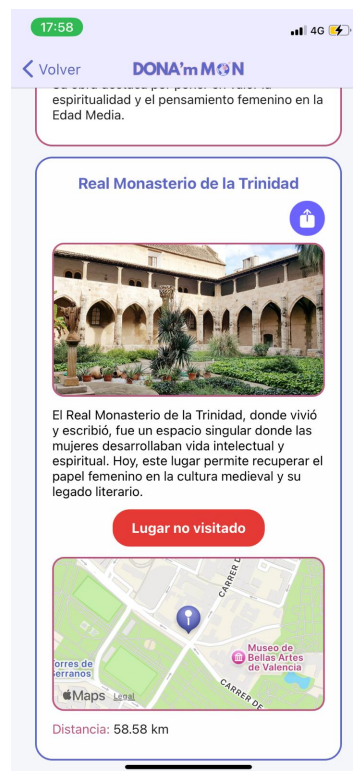


Figura 8.5: Filtros aplicables sobre el mapa y el listado de lugares.

En las Figuras 8.6a y 8.6b se muestra el detalle de un lugar, pudiendo marcar como visitado libremente o compartir el lugar, como se ve en las Figuras 8.7a y 8.7b.

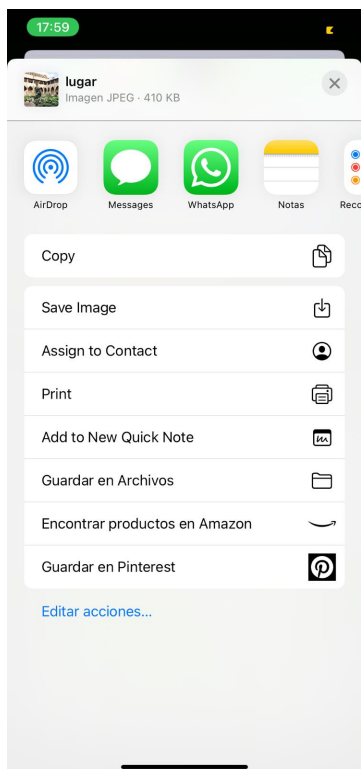


(a) Detalle de un lugar (1).

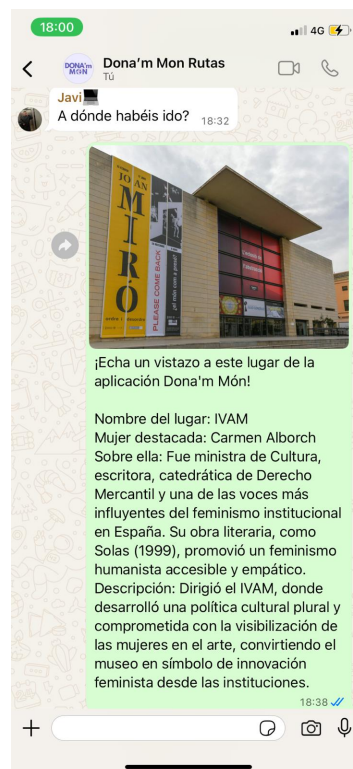


(b) Detalle de un lugar (2).

Figura 8.6: Detalles de un lugar en la aplicación.



(a) Opción para compartir el lugar (1).



(b) Opción para compartir el lugar (2).

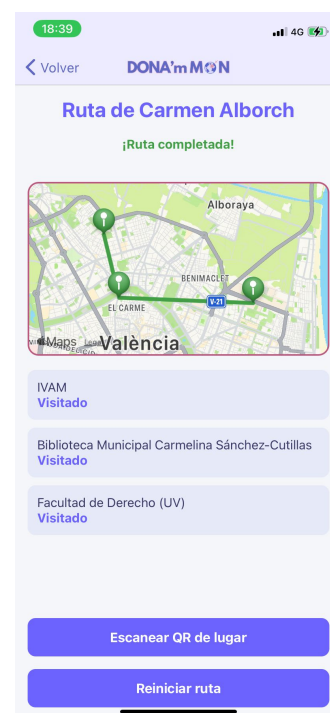
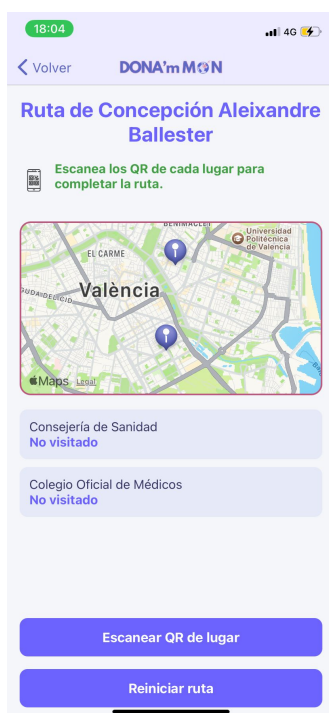
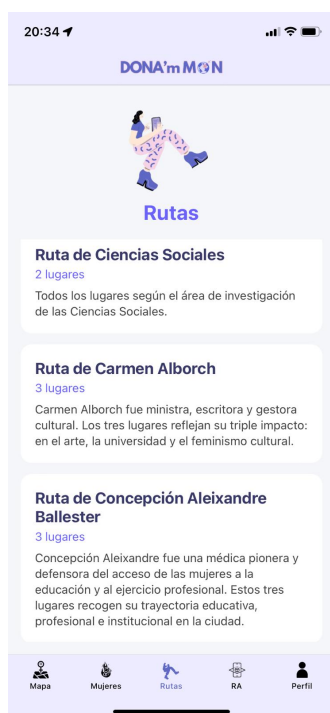
Figura 8.7: Opciones de interacción con el lugar: compartir.

En la Figura 8.8 se observa cómo se muestran las imágenes al pulsar sobre ellas.



Figura 8.8: Visualización de imágenes en modal.

En la Figura 8.9a se muestra la página de rutas, en la Figura 8.9b una ruta incompleta y en la Figura 8.9c una ruta completada escaneando un código QR que se obtiene al llegar al lugar.



(a) Página de rutas.

(b) Ruta incompleta.

(c) Ruta completada.

Figura 8.9: Página de rutas y ejemplos de rutas incompleta y completada.

La Figura 8.10 muestra la pantalla de realidad aumentada, y las Figuras 8.11, 8.12 y 8.13 la experiencia de realidad aumentada.



Figura 8.10: Pantalla de realidad aumentada.



Figura 8.11: Experiencia de realidad aumentada (1).



Figura 8.12: Experiencia de realidad aumentada (2).



Figura 8.13: Experiencia de realidad aumentada (3).

Por último, la Figura 8.14 muestra la pantalla de perfil, donde el usuario puede modificar su nombre, acceder al historial de sus lugares visitados (Figura 8.15), al historial de las rutas completas (Figura 8.16) y cerrar sesión. Si se pulsa en borrar en alguno de los historiales, se actualizará el mapa, el listado o las rutas también.

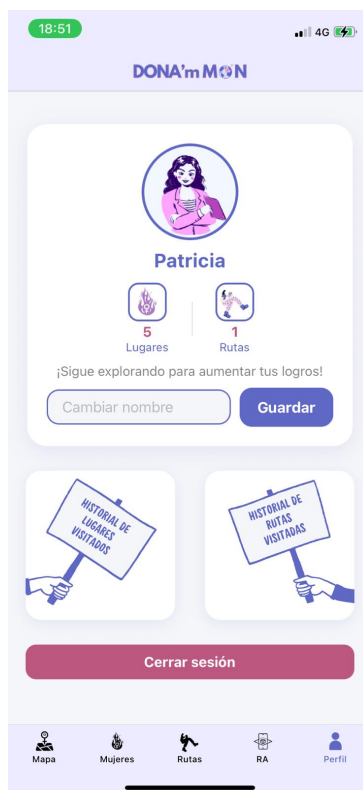


Figura 8.14: Pantalla de perfil del usuario.



Figura 8.15: Historial de lugares visitados.

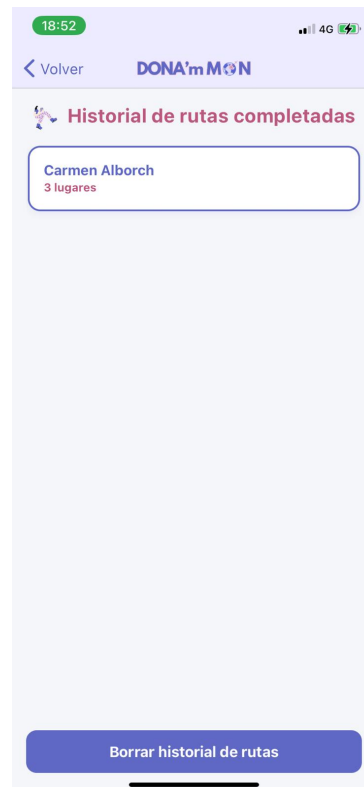


Figura 8.16: Historial de rutas completadas.

Capítulo 9

Conclusiones

9.1. Revisión de costes

Durante el transcurso del desarrollo, ha sido necesario realizar ajustes en la estimación inicial de costes debido a diversas **incidencias técnicas** que afectaron principalmente a la fase de implementación. Aunque el planteamiento general se ha mantenido dentro de lo previsto, ciertos retos específicos obligaron a extender la duración de algunas tareas.

Las principales desviaciones detectadas se relacionan con la **adaptación multiplataforma** y la **tecnología de realidad aumentada**. Por un lado, en dispositivos **Android** fue necesario modificar el comportamiento de funcionalidades como las notificaciones o el despliegue del mapa emergente, ya que no funcionaban correctamente con la implementación inicial. En el caso de **iOS**, surgieron limitaciones con permisos y uso de cámara, lo cual requirió una reestructuración parcial del sistema.

Por otro lado, la carga de **modelos 3D** en la versión web del proyecto presentó más dificultades de las previstas. Aunque se asumió que la plataforma **GitHub Pages** podría gestionar archivos **.glb**, surgieron incompatibilidades y problemas de carga, por lo que se optó por alternativas como **Vercel**. Asimismo, fue necesario reducir el peso de los modelos mediante herramientas de optimización y, en algunos casos, reconstruirlos desde cero para garantizar el **rendimiento adecuado** en dispositivos móviles.

Estas incidencias derivaron en un aumento del tiempo de trabajo estimado para los perfiles técnicos implicados, especialmente el **desarrollador frontend**, el **encargado de la realidad aumentada** y el **modelador 3D**. Se estima una inversión adicional de **10 días de trabajo**, lo que equivale a **80 horas extra** de dedicación.

Cuadro 9.1: Ampliación de jornadas y costes adicionales

Perfil	Días estimados	Días reales	Diferencia	Coste adicional (€)
Desarrollador Frontend	16	20	+4	1.083,68
Desarrollador RA	7	10	+3	461,04
Modelador 3D	15	18	+3	239,52
Total adicional				1.784,24

Este incremento afecta también al cálculo de los **costes indirectos**, dado que estos se calculan como un porcentaje sobre los costes directos. Con los nuevos valores, el **coste total del proyecto** se actualiza de la siguiente manera:

Nuevos costes directos de personal = $27,718,95 + 1,784,24 = \mathbf{29.503,19 \text{ €}}$

Costes indirectos actualizados = $(29,503,19 + 212,40) \times 0,20 = \mathbf{5.943,12 \text{ €}}$

Coste total final = $29,503,19 + 212,40 + 5,943,12 = \mathbf{35.658,71 \text{ €}}$

9.1.1. Comparativa gráfica

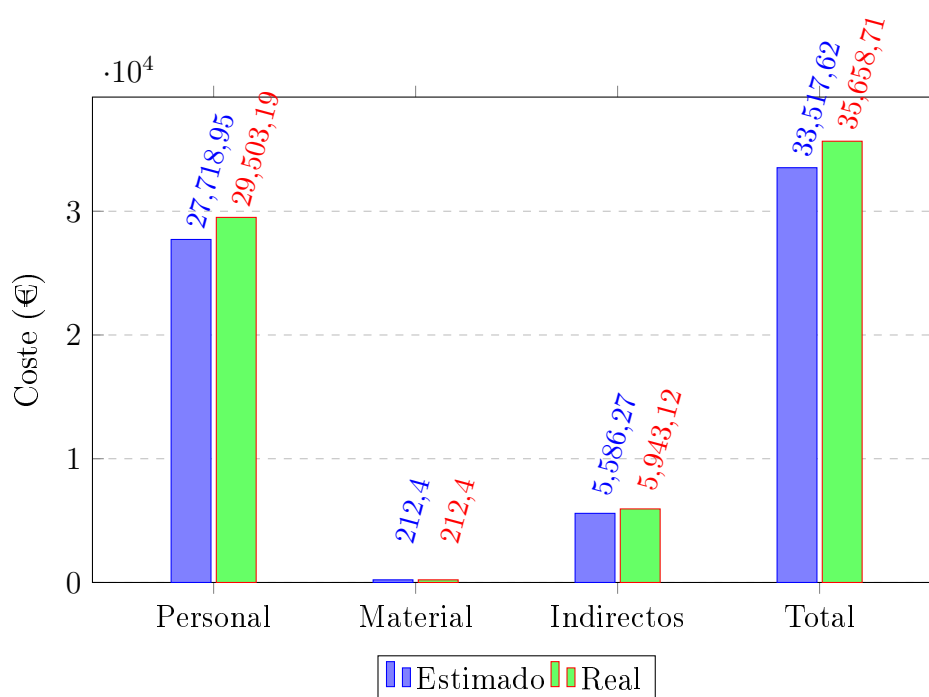


Figura 9.1: Comparación entre el coste estimado y el coste real

9.1.2. Reflexión final

El **coste total final** del proyecto asciende a **35.658,71 €**, lo que supone un incremento del **6,4 %** respecto a la previsión inicial. Esta diferencia se justifica por la necesidad de resolver **desafíos técnicos reales** durante el desarrollo, y se considera razonable para un proyecto con componentes tecnológicos avanzados. Este análisis subraya la importancia de contar con **márgenes de contingencia** al planificar y dimensionar proyectos similares en entornos profesionales.

9.2. Conclusiones

Tras varios meses de trabajo, puedo afirmar que el desarrollo de *DONA'm MÓN* ha supuesto un reto tanto técnico como personal. Habiendo cumplido con todos los objetivos planteados, observamos dos conclusiones diferenciadas.

9.2.1. Conclusiones técnicas

El desarrollo de *DONA'm MÓN* ha demostrado la viabilidad de integrar en un único producto funcionalidades avanzadas como geolocalización, escaneo de códigos QR y realidad aumentada, combinando un *frontend* móvil con React Native y un *backend* en Django con base de datos relacional. Esta integración no solo ha requerido un conocimiento sólido de cada tecnología, sino también la capacidad de resolver problemas derivados de su interoperabilidad, como la gestión de peticiones, la latencia entre cliente y servidor o la carga de modelos 3D optimizados para web.

Uno de los mayores aprendizajes ha sido comprobar la importancia de planificar con flexibilidad: aunque se definió una planificación inicial, la realidad del desarrollo obligó a ajustar tiempos y priorizar funcionalidades críticas, un escenario habitual en proyectos reales. También ha sido clave la gestión de la complejidad: el paso de un prototipo básico a un sistema estructurado y escalable implicó trabajar con patrones de diseño, buenas prácticas en la API REST y un uso responsable de librerías externas.

La experiencia confirma que la combinación de RA y geolocalización en aplicaciones móviles abre un abanico de posibilidades en el ámbito educativo, turístico y cultural. Sin embargo, también plantea retos como la optimización del rendimiento en dispositivos con recursos limitados y la necesidad de ofrecer experiencias inmersivas que sean intuitivas y accesibles para todo tipo de usuarios. Estos aspectos se convierten en líneas claras de mejora para evoluciones futuras.

9.2.2. Conclusiones personales

Realizar este proyecto ha sido una experiencia intensa y, sobre todo, muy enriquecedora. Más allá del resultado final, me quedo con todo lo que he aprendido durante el camino. Una de las cosas que más valoro es haber comprobado lo diferente que es enfrentarse a un proyecto completo respecto a pequeños ejercicios académicos: aquí cada decisión técnica tenía consecuencias, y eso me ha obligado a planificar mejor, anticipar problemas y aprender a priorizar cuando no era posible abarcar todo al mismo tiempo.

También ha sido un reto mantener la motivación cuando aparecían errores inesperados o incompatibilidades entre las tecnologías. Resolverlos me ha enseñado que, en desarrollo, la paciencia y la constancia son tan importantes como el conocimiento técnico. Además, he tenido que aprender a buscar soluciones fuera del temario, explorando documentación oficial, foros y recursos especializados, lo que me ha hecho más autónoma y segura de mí misma a la hora de afrontar problemas complejos.

Por último, este TFG me ha confirmado dos cosas: que me apasiona el desarrollo móvil y que me gustaría seguir creciendo en áreas como la realidad aumentada y la experiencia de usuario. Sé que este es un proyecto que no quiero dar por terminado, me gustaría viéndolo en producción aportando un valor social real, seguir actualizándolo y mejorándolo está dentro de mis planes. Ahora he descubierto mis puntos fuertes, mis áreas de mejora y, sobre todo, el camino profesional que quiero seguir.

9.3. Trabajo futuro

A pesar de que la aplicación *DONA'm MÓN* ya ofrece una experiencia completa y funcional, existen múltiples líneas de trabajo que podrían explorarse en el futuro para

ampliar su alcance, mejorar su calidad y aumentar su impacto social. A continuación, se detallan algunas de las posibles mejoras y ampliaciones:

- **Colaboración con asociaciones locales.** Se plantea establecer contacto con organizaciones feministas como **Por Ti Mujer** y la **Asamblea de Mujeres de Valencia**, con el objetivo de ofrecer la aplicación como herramienta de apoyo para sus rutas temáticas y actividades educativas.
- **Solicitud de nuevos puntos de interés.** Sería relevante habilitar una funcionalidad en la propia aplicación que permita a las personas usuarias proponer nuevos lugares o figuras femeninas relevantes, fomentando así la participación ciudadana y la expansión colaborativa del contenido.
- **Mejoras en la experiencia de realidad aumentada.** Se propone desarrollar interacciones diferenciadas por ubicación, incorporando animaciones específicas, efectos visuales personalizados o minijuegos que refuercen el vínculo entre cada figura femenina y su entorno.
- **Optimización de los modelos 3D.** Es conveniente seguir trabajando en la mejora visual y técnica de los modelos tridimensionales, buscando un equilibrio entre calidad gráfica, rendimiento y peso de los archivos para garantizar una experiencia fluida en todo tipo de dispositivos.
- **Incorporación de una página de contacto.** Añadir un apartado de contacto dentro de la aplicación permitiría a los usuarios enviar sugerencias, reportar errores o comentar su experiencia de uso, facilitando así un canal de retroalimentación directa.
- **Sección de preguntas frecuentes (FAQ).** Un listado de dudas comunes con sus respectivas respuestas ayudaría a resolver posibles bloqueos o malentendidos sin necesidad de asistencia personalizada.
- **Espacio de noticias y novedades.** Sería útil incorporar una sección que informe sobre las actualizaciones de la aplicación, nuevos puntos añadidos o actividades relacionadas con las mujeres destacadas en la plataforma.
- **Panel de administración mejorado y sistema de análisis.** Aunque el proyecto cuenta con el panel básico de Django Admin, sería interesante desarrollar una interfaz administrativa personalizada que integre:
 - Dashboard con estadísticas de uso en tiempo real y métricas de comportamiento de usuarios.
 - Sistema de análisis avanzado que proporcione insights sobre rutas más populares y puntos de interés más visitados.
 - Gestor visual de contenido multimedia, especialmente para modelos 3D.
 - Herramientas de moderación para gestionar propuestas de contenido de usuarios.
 - Sistema de notificaciones push masivas y comunicación con la comunidad.
 - Generación automática de informes de actividad y reportes de uso.
- **Expansión geográfica.** Una línea natural de crecimiento consistiría en adaptar y extender la aplicación a otras ciudades, tanto de España como de otros países,

promoviendo así la visibilidad de figuras femeninas relevantes en diferentes contextos históricos y geográficos.

- **Estrategia de difusión en redes sociales.** Para dar a conocer el proyecto y fomentar su uso, se propone diseñar una campaña de marketing digital basada en plataformas como Instagram o TikTok. Esta estrategia podría incluir vídeos grabados en los puntos de interés reales, mostrando el uso de la app y escaneando los códigos QR, invitando al público a participar y compartir sus experiencias.

Estas líneas de trabajo no solo aportarían nuevas funcionalidades, sino que también contribuirían a la sostenibilidad y escalabilidad del proyecto a largo plazo.

Apéndice A

Apéndice

A.1. Ejemplos del lenguaje de marcado Latex

Antes de empezar a escribir la memoria debes leerle, al menos, los dos primeros capítulos del clásico de Tobias Oetiker [82]¹.

El Apéndice A.1 recoge algunos ejemplos de edición con L^AT_EX.

This document is an example of BibTeX using in bibliography management. Three items are cited: *The L^AT_EX Companion* book [83], the Einstein journal paper [84], and the Donald Knuth's website [85]. The L^AT_EX related items are [83, 85]².

Texto en el párrafo 1.

Texto en el párrafo 2.

Texto en el párrafo 3.

- Consideración 1
- Consideración 2

1. Punto 1
2. Punto 2

A continuación se muestra una ecuación:

$$\int_0^1 \frac{1}{x^2 + 1} dx$$

Podemos incluir imágenes en formato: png, pdf o jpg.

En la figura ?? se muestra un diagrama realizado con <https://www.yworks.com/products/yed>:

¹La última versión puede encontrarse en <http://tobi.oetiker.ch/lshort/lshort.pdf>.

²Esto está tomado de https://www.overleaf.com/learn/latex/Bibliography_management_with_bibtex

Imagen 1

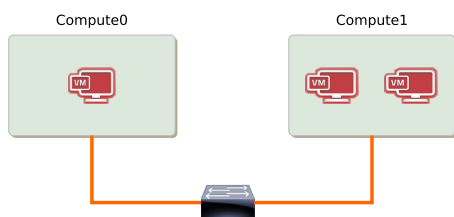
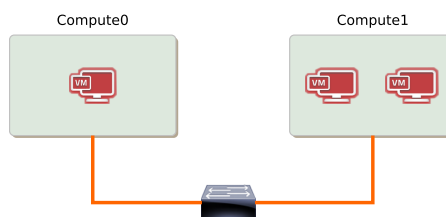


Imagen 2



Este es un ejemplo de una tabla:

Columna 1	Columna 2
1	2

O la misma tabla centrada:

Columna 1	Columna 2
1	2

Para generar el fichero PDF:

```
pdflatex ejemplo-memoria.tex
bibtex ejemplo-memoria
pdflatex ejemplo-memoria.tex
```

También se puede usar `latexmk` que automáticamente regenera la bibliografía.

```
latexmk -pdf ejemplo-memoria.tex
```

Bibliografía

- [1] S92063042. History and evolution of smartphones. <https://medium.com/@s92063042/history-and-evolution-of-smartphones-17987e6f91d1>, 2023. Medium. Accedido: 2025-01-17.
- [2] José de Jesús Cano Rosales. *Desarrollo de una aplicación de realidad aumentada para dispositivos móviles como herramienta de apoyo en el proceso de enseñanza-aprendizaje*. Tesis de grado, Universidad de San Carlos de Guatemala, Guatemala, 2019. Facultad de Ingeniería.
- [3] Ramez Elmasri and Shamkant B. Navathe. *Fundamentals of database systems*. Pearson, 7th edition, 2017.
- [4] PostgreSQL Global Development Group. Postgresql 16 documentation. <https://www.postgresql.org/docs/16/>, 2024.
- [5] MariaDB Foundation. Mariadb vs mysql, 2023.
- [6] Inc. MongoDB. Mongoddb manual, 2024.
- [7] UNESCO. Science report: The race against time for smarter development. <https://unesdoc.unesco.org/ark:/48223/pf0000377433>, 2021. United Nations Educational, Scientific and Cultural Organization.
- [8] Ministerio de Ciencia e Innovación. Científicas en cifras 2021: Estadísticas e indicadores de la (des)igualdad de género en la formación y profesión científica. https://www.ciencia.gob.es/dam/jcr:dc8689c4-2c47-4aaf-97ce-874bd0b5a081/Cientificas_en_Cifras_2021.pdf, 2022. Secretaría General de Investigación.
- [9] Organización de las Naciones Unidas. Transformar nuestro mundo: la agenda 2030 para el desarrollo sostenible. <https://www.un.org/sustainabledevelopment/es/agenda-2030/>, 2015. Consultado el 2 de junio de 2025.
- [10] ONU Mujeres. Objetivo 5: Lograr la igualdad entre los géneros y empoderar a todas las mujeres y niñas. <https://www.unwomen.org/es/news/in-focus/women-and-the-sdgs/sdg-5-gender-equality>, 2024. Consultado el 2 de junio de 2025.
- [11] ONU-Hábitat. Objetivo 11: Lograr que las ciudades sean más inclusivas, seguras, resilientes y sostenibles. <https://www.un.org/sustainabledevelopment/es/cities/>, 2024. Consultado el 2 de junio de 2025.
- [12] Jon Agar. Constant touch: A global history of the mobile phone, 2004. ISBN 978-1840465413.
- [13] Joel West and Jason Dedrick. Globalization and the evolution of the mobile phone industry. *International Journal of Technology and Globalisation*, 5(1/2):65–80, 2010.

-
- [14] Jamie Murphy. Mobile marketing: The future of marketing, or just another fad? *International Journal of Mobile Marketing*, 9(1):47–59, 2014.
 - [15] StatCounter. Mobile operating system market share worldwide. <https://gs.statcounter.com/os-market-share/mobile/worldwide>, 2024.
 - [16] Statista. Number of smartphone users worldwide from 2016 to 2024. <https://www.statista.com/statistics/330695/number-of-smartphone-users-worldwide/>, 2024.
 - [17] AndroidPolice. Android versus ios software updates revisited, 2017.
 - [18] T. Nguyen and J. Smith. Mobile application development trends. *International Journal of Computer Science*, 15(3):45–62, 2018.
 - [19] Gartner. Market guide for multiexperience development platforms, 2022.
 - [20] Data.ai. State of mobile 2023, 2023.
 - [21] App Annie. State of mobile report 2022, 2022.
 - [22] David Parsons and Kathryn MacCallum. *Digital Transformation and Public Services: Societal Impacts in Sweden and Beyond*. Springer, Cham, 2020.
 - [23] Ronald T. Azuma. A survey of augmented reality. *Presence: Teleoperators and Virtual Environments*, 6(4):355–385, 1997. <https://doi.org/10.1162/pres.1997.6.4.355>.
 - [24] Mark Billinghurst, Adrian Clark, and Gun Lee. A survey of augmented reality. *Foundations and Trends in Human–Computer Interaction*, 8(2–3):73–272, 2015. <https://doi.org/10.1561/11000000049>.
 - [25] Steven Feiner, Blair MacIntyre, and Dorée Seligmann. Knowledge-based augmented reality. *Communications of the ACM*, 36(7):53–62, 1993. <https://doi.org/10.1145/159544.159587>.
 - [26] Hirokazu Kato and Mark Billinghurst. Marker tracking and hmd calibration for a video-based augmented reality conferencing system. In *Proceedings 2nd IEEE and ACM International Workshop on Augmented Reality (IWAR’99)*, pages 85–94, 1999. <https://doi.org/10.1109/IWAR.1999.803809>.
 - [27] Daniel Wagner and Dieter Schmalstieg. First steps towards handheld augmented reality. In *Proceedings of the 7th IEEE International Symposium on Wearable Computers*, pages 127–135, 2003. <https://doi.org/10.1109/ISWC.2003.1241409>.
 - [28] Apple. Arkit: Augmented reality for ios. <https://developer.apple.com/augmented-reality/>, 2017.
 - [29] Google. Arcore: Google developers. <https://developers.google.com/ar>, 2018.
 - [30] J. Paavilainen, J. Hamari, J. Stenros, and J. Kinnunen. The pokémon go experience: A location-based augmented reality mobile game goes mainstream. In *Proceedings of CHI 2017*, 2017.
 - [31] Microsoft. Hololens product page. <https://www.microsoft.com/en-us/hololens>, 2016.
 - [32] D. W. F. van Krevelen and R. Poelman. A survey of augmented reality technologies, applications and limitations. *The International Journal of Virtual Reality*, 9(2):1–20, 2010.
 - [33] G. Klein and D. Murray. Parallel tracking and mapping for small ar workspaces. In *ISMAR 2007*, 2007.

- [34] Hiroshi Ishii and Brygg Ullmer. Tangible bits: towards seamless interfaces between people, bits and atoms. In *Proceedings of the ACM SIGCHI Conference on Human Factors in Computing Systems*, pages 234–241, 1997. <https://doi.org/10.1145/258549.258715>.
- [35] M. Billinghurst and H. Kato. Collaborative mixed reality. In *ISMAR 2002*, 2002.
- [36] Apple. Introducing apple vision pro. <https://www.apple.com/apple-vision-pro/>, 2024.
- [37] Women’s legacy. proyecto europeo cofinanciado por la unión europea, 2022. Conselleria d’Educació, Cultura i Esport de la Generalitat Valenciana.
- [38] Dones de ciència: Murals que inspiren, 2020. Las Naves & UPV.
- [39] Callejero con perspectiva de género: iniciativas y actuaciones, 2023. Ayuntamiento de València.
- [40] Asociación Por Ti Mujer. Ruta violeta de mujeres en valencia, 2023. Recuperado de <https://asociacionportimujer.org/ruta-violeta-de-mujeres-en-valencia>.
- [41] Asamblea Feminista de València. Ruta pels espais del patronato de protección a la mujer, 2023. Recuperado de <https://www.feministas.org/ruta-pels-espais-del-patronato-de.html>.
- [42] Asamblea Feminista de València. Ruta de memorias lesbianas en valencia: visibilización y lucha feminista, 2024. Recuperado de <https://www.levante-emv.com/valencia/2024/10/27/mapa-memorias-lesbianas-valencia-lesbianismo-igtbi-ruta-asamblea-feminista-feminismo-110410679.html>.
- [43] R. Guijarro-Garzón et al. Pioneras de la medicina: Concepción aleixandre y la visibilización de la mujer en la ciencia. *Revista de Historia de la Medicina*, 12(2):134–148, 2021.
- [44] Carmen Alborch. *Solas*. Espasa Calpe, 1999.
- [45] Amelia González. *Clara Campoamor: el sufragio femenino en España*. Cátedra, 2006.
- [46] Carmen Rueda Ramos. *Isabel de Villena y la espiritualidad femenina en la Valencia del siglo XV*. Tirant lo Blanch, 2013.
- [47] J. Martínez Pérez. *Médicas en Valencia: mujeres que abrieron camino*. Editorial UV, 2019.
- [48] Vandana Shiva. *Earth Democracy: Justice, Sustainability, and Peace*. South End Press, 2005.
- [49] Google Developers. Google maps platform documentation. <https://developers.google.com/maps/documentation>.
- [50] Mapbox. Mapbox documentation. <https://docs.mapbox.com/>.
- [51] OpenStreetMap Foundation. Openstreetmap wiki. <https://wiki.openstreetmap.org/>.
- [52] React Native Maps Contributors. react-native-maps: React native mapview component for ios + android. <https://github.com/react-native-maps/react-native-maps>.
- [53] Django Software Foundation. Django documentation, 2024.
- [54] Michael Stonebraker and Rick Cattell. 10 rules for scalable performance in ‘simple operation’ datastores. *Communications of the ACM*, 54(6):72–80, 2011.
- [55] Antonio Mele Vincent. *Mastering Django: Core*. Independently published, 2020.

- [56] Django Software Foundation. Using sqlite with django, 2024.
- [57] Apache Software Foundation. Apache cassandra documentation, 2023.
- [58] Viafirma. Implementación de la fórmula haversine en java. <https://www.viafirma.com/es/implementacion-de-la-formula-haversine-en-java/>, 2022. Consultado en 2025.
- [59] Glassdoor. Sueldo de jefe de proyecto. https://www.glassdoor.es/Sueldos/jefe-de-proyectos-sueldo-SRCH_KO0,17.htm, 2025. Consultado el 11 de junio de 2025.
- [60] Glassdoor. Sueldo de desarrollador react native. https://www.glassdoor.es/Sueldos/react-native-developer-sueldo-SRCH_KO0,22.htm, 2025. Consultado el 11 de junio de 2025.
- [61] Glassdoor. Sueldo de desarrollador python/django. https://www.glassdoor.es/Sueldos/django-developer-sueldo-SRCH_KO0,16.htm, 2025. Consultado el 11 de junio de 2025.
- [62] Glassdoor. Sueldo de qa tester. https://www.glassdoor.es/Sueldos/qa-tester-sueldo-SRCH_KO0,9.htm, 2025. Consultado el 11 de junio de 2025.
- [63] Glassdoor. Sueldo de modelador 3d. https://www.glassdoor.es/Sueldos/modelador-de-3d-sueldo-SRCH_KO0,15.htm, 2025. Consultado el 11 de junio de 2025.
- [64] Glassdoor. Sueldo de diseñador gráfico. https://www.glassdoor.es/Sueldos/disenador-grafico-sueldo-SRCH_KO0,17.htm, 2025. Consultado el 11 de junio de 2025.
- [65] Glassdoor. Sueldo de desarrollador de realidad aumentada/virtual. https://www.glassdoor.es/Sueldos/virtual-reality-developer-sueldo-SRCH_KO0,25.htm, 2025. Consultado el 13 de junio de 2025.
- [66] ONU Mujeres. El color morado y su relación con el feminismo. <https://www.unwomen.org/es/news-stories/explainer/2023/03/el-color-morado-y-su-relacion-con-el-feminismo>, 2023. ONU Mujeres.
- [67] Kira Cochrane. Why purple is the colour of international women's day. <https://www.theguardian.com/lifeandstyle/the-womens-blog-with-jane-martinson/2013/mar/08/purple-colour-international-womens-day>, 2013. The Guardian.
- [68] Ana Molina. Guía práctica de teoría del color para diseño gráfico. <https://www.domestika.org/es/blog/1041-teoria-del-color-guia-practica-para-disenadores>, 2010. Domestika.
- [69] Leatrice Eiseman. *Colors for Your Every Mood: Discover Your True Decorating Colors*. Capital Books, 2000.
- [70] Canva Pty Ltd. Canva: Herramienta de diseño gráfico online. <https://www.canva.com/>, 2025. Consultado en 2025.
- [71] Sara Ember. Canción sin miedo - vivir quintana - violin cover by sara ember, 2021. Video en YouTube. Accedido: 19/05/2025.
- [72] Piotr Adam Milewski. Display utf8 text with accents in aframe like á í ñ, 2018. Stack Overflow answer (mar 14, 2018). <https://stackoverflow.com/questions/49276815/display-utf8-text-with-accent-in-aframe-like-%C3%A1-%C3%AD-%C3%B1>.
- [73] A-Frame. A-frame documentation: Models. <https://aframe.io/docs/1.7.0/introduction/models.html>, 2024. Accedido: 17 de julio de 2025.
- [74] Vercel. Vercel documentation: Platform limits. <https://vercel.com/docs/limits>, 2024. Accedido: 17 de julio de 2025.

-
- [75] Mozilla Developer Network. Webxr device api - browser compatibility, 2024. Accedido: 18 julio 2025.
- [76] Can I Use. Webxr device api, 2024. Accedido: 18 julio 2025.
- [77] John Brooke. Sus: A 'quick and dirty' usability scale. *Usability Evaluation in Industry*, pages 189–194, 1996.
- [78] Asus rog strix g15 g513ic-hn004 amd ryzen 7 4800h/16gb/512gb ssd/rtx 3050/15.6". <https://www.pccomponentes.com/asus-rog-strix-g15-g513ic-hn004-amd-ryzen-7-4800h-16gb-512gb-ssd-rtx-3050-156>. Accedido: 19/07/2025.
- [79] iphone 11 - especificaciones técnicas. <https://support.apple.com/es-es/111865>. Accedido: 19/07/2025.
- [80] The Apache Software Foundation. Apache jmeter, 2025. Accedido: 2025-01-14.
- [81] Jiajian Liri. Eruda: Console for mobile browsers. <https://github.com/liriliri/eruda>, 2015. Versión consultada en 2025.
- [82] Tobias Oetiker. The not so short introduction to latex2e. <https://tobi.oetiker.ch/lshort/lshort.pdf>, 2023.
- [83] Frank Mittelbach and Michel Goossens. *The LaTeX Companion*. Addison-Wesley, 2nd edition, 2004.
- [84] Albert Einstein. Zur elektrodynamik bewegter körper, 1905.
- [85] Donald E. Knuth. The texbook. <https://www.ctan.org/pkg/texbook>, 1986.